Split-Sync App

Shared Expense Management System

Design Document

Started 24/10/2025

Student Name: Emmanuel Mobolaji Abolade

Student ID: C00288657

Institution: South East technological University, Carlow

Programme: BSc Software Development

Supervisor: Joseph Kehoe

Submitted: 13/02/2026

Table of content

1. INTRODUCTION..... 6
1.1 Project Overview.....6
1.2 Purpose of This Document..... 7
1.3 Document Structure.....7
2. SYSTEM ARCHITECTURE.....9
2.1 System Architecture Overview.....9
2.1.1 Frontend Architecture..... 10
2.1.2 Backend Architecture..... 11
2.1.3 Third-Party Integrations.....11
2.2 Technology Stack Overview.....12
2.3 Application Navigation Structure.....13
2.3.1 Route Definitions.....15
3.0 DATABASE DESIGN..... 17
3.1 Entity Relationship Diagram..... 18
3.2 Core Database Collections.....20
3.2.1 Users Collection.....20
3.2.2 Group Collection..... 24
3.2.3 Expenses Collection..... 27
3.2.4 Connections Collection..... 30
3.2.5 Invitations Collection.....32
3.2.6 Phone Numbers Collection (Contact Matching).....35
3.3 Subcollections.....37
3.3.1 Group Activity Log.....37
3.3.2 Group Balance (Computed).....38
3.4 Database Design Rationale..... 39
3.4.1 Denormalisation Strategy.....39
3.4.2 Composite Indexes..... 39
3.4.3 Data Consistency Model.....40
3.5 Storage Estimates and Scaling.....41
3.6 Backup and Recovery Strategy..... 42
3.7 Data Privacy and GDPR Compliance..... 43
3.8 Database Operations and Ledger Logic..... 45
3.8.1 Adding Expenses.....45
3.8.2 Computing Balances.....45
3.8.3 Settlement Suggestions.....46
3.8.4 Processing Settlements..... 46
3.8.5 Maintaining Data Integrity..... 47
3.9 Firestore Security Rules..... 47
3.9.1 Complete Firestore Security Rules..... 48

4.0 PAYMENTS AND INTEGRATIONS.....51
4.1 Payment Gateway Selection.....51
4.2 Payment Flow Architecture.....52
4.2.1 Payment setup in Profile.....53
4.2.2 Settlement intent creation (from balances).....53
4.2.3 Payment instruction generation (client-side).....54
4.2.4 External payment execution (off-platform).....54
4.2.5 Confirmation + reconciliation in SplitSync.....55
4.2.6 Optional automation via Cloud Functions.....55
4.2.7 Security and compliance posture.....56
4.2.8 Why this architecture is strong.....56
4.3 Strong Customer Authentication.....57
4.4 Webhook Handling.....57
4.4.1 Payment Confirmation Architecture.....58
4.4.2 Webhook-Based Confirmation (Intended Production Architecture).....58
4.4.3 Design Justification.....59
5. UIX - USER INTERFACE AND USER EXPERIENCE DESIGN.....60
5.1 Design Principles.....60
5.2 Key Screen Wireframes.....61
5.2.1 Login Screen.....61
5.2.2 Dashboard (Groups List).....62
5.2.3 Add Expense Form.....63
5.2.4 Balance Ledger.....64
5.3 User Flow Diagrams.....65
5.4 Use Case List and Sequence Diagram for Split-sync.....66
5.4.1 User Authentication & Account Management.....66
5.4.2. Profile & Settings Management.....67
5.4.3. Group Management.....68
5.4.4. Member Management.....69
5.4.5. Expense Management.....70
5.4.6. Balance & Settlement.....71
5.4.7. Social Features.....72
5.4.8. Notifications & Activity.....72
5.4.9. Offline & Pwa Features.....73
5.4.10. Admin & Moderation.....74
5.4.11. Security & Privacy.....74
5.4.12. Help & Support.....74
5.5 Message Sequence Diagram.....75
5.5.1 UC-03 - User Login with Biometric Authentication Sequence Diagram.....75
5.5.2 UC-01 - User Registration Sequence Diagram.....78
5.5.3 UC-27/UC-28: Create Group with Quick Add.....85

5.5.4 UC-46: Accept Invitation Sequence Diagram..... 88
5.5.5 UC54-UC62 AddExpense ReceiptScan Sequence..... 91
5.5.6 UC-42: Find Contacts Sequence Diagram.....93
5.5.7 UC-78: Settle via PayPal Sequence Diagram.....99
5.5.8 UC-74: View Balances Sequence Diagram..... 107
5.5.9 UC-82: Mark Payment as Paid..... 108
5.5.10 UC-92: View Activity Feed..... 117
5.6 Alternative Flows.....118
5.6.1 Alternative Flow 1: User Not Authenticated..... 118
5.6.2 Alternative Flow 2: User Has No Groups..... 118
5.6.3 Alternative Flow 3: Firestore Permission Denied..... 119
5.6.4 Alternative Flow 4: Network Offline (Cache Available).....119
5.6.5 Alternative Flow 5: Network Offline (No Cache).....119
5.6.6 Alternative Flow 6: Large Activity Feed (Pagination Required).....119
6. CLASS DIAGRAM..... 122
6.1 Overview.....122
6.2 Architecture Layers.....124
6.2.1 Domain Model Layer (Core Business Entities)..... 124
6.2.2 Service Layer (Business Logic)..... 125
6.3 Key Design Patterns.....126
6.4 Relationships..... 126
6.5 Enumerations for Type Safety..... 127
6.6 Data Flow Example.....127
6.7 Security Considerations.....128
6.8 Scalability Considerations.....128
6.9 Class diagram Summary..... 129
7.0 PSEUDOCODES.....130
7.1 Overview.....130
7.2 Balance Calculation Algorithm.....130
7.2.1 Purpose.....130
7.2.2 Example Execution..... 134
7.3 Split Calculation Algorithms.....135
7.3.1 Equal Split.....135
7.3.2 Percentage Split.....137
7.3.3 Shares Split.....139
7.4 Contact Matching Algorithm (Privacy-Preserving)..... 140
7.4.1 Purpose.....140
7.4.2 Supporting Functions.....143
7.4.3 Complexity Analysis.....145
7.4.4 Privacy Properties.....145
7.5 Settlement Suggestion Algorithm..... 145

7.5.1 Purpose....145
7.5.2 Example Execution....147
7.6 Invitation Expiration Cleanup....148
7.6.1 Purpose....148
7.7 Mathematics & Logic of Expense Splitting in Split-Sync....150
7.7.1 Basic Split Methods....150
7.7.2 Advanced Algorithm -Minimum Cash Flow....159
7.7.3 Handling Edges Cases....165
7.7.4 Validation Rules....166
7.7.5 Rounding Strategies....167
7.7.5.1 Strategy 1: Banker's Rounding (Round Half to Even)....167
7.7.5.2 Strategy 2: Give Remainder to Payer (Recommended)....167
7.7.5.3 Strategy 3: Distribute Pennies Round-Robin....168
8.0 CONCLUSION....168
APPENDIX....169
REFERENCES....170

1. INTRODUCTION

1.1 Project Overview

Split-Sync is a Progressive Web Application (PWA) designed to simplify shared expense management for groups. The application addresses the common challenges faced when splitting bills among friends, roommates, or travel companions by providing an integrated platform that handles expense tracking, balance calculation, and payment settlement in one place.

The system differentiates itself from existing solutions like Splitwise and Tikkie through three key innovations:

- Integrated payment processing via SEPA, Paypal and Revolut eliminating the gap between tracking and settling debts
- OCR-powered receipt scanning using Google Cloud Vision API for automated expense entry and saving.
- Minimum cash-flow optimization algorithm to reduce the number of settlement transactions required.

Target users include:

- University students sharing accommodation costs.
- Friends coordinating group activities such as parties, travel, shopping, and other expenses.
- Couples managing joint household expenses.
- Event organizers tracking group purchases etc.

Privacy-preserving contact synchronization using SHA-256 hashing enables users to discover friends on the platform without compromising phone number privacy. Built on Firebase's cloud platform with React 18 frontend, the application implements real-time synchronization, offline capability, and cross-platform compatibility. The design prioritizes financial accuracy (100% test coverage on calculation algorithms, with approximately 81% line coverage across the full application), data security (GDPR-compliant with optional biometric authentication), and user experience (sub-2-second load times for all core operations).

1.2 Purpose of This Document

This Design Document presents the architectural foundation and model, user interface concepts, and implementation timeline for Split-Sync. This design specification documents the complete technical architecture, database schema, user workflows, message sequence diagrams, key algorithms, and comprehensive testing strategy for the application. It serves as a blueprint for the development process and provides stakeholders with a clear understanding of the technical approach and project scope.

This document is intended for:

- Description of Tech Stack, Data definitions, Payment Integration and UIX.
- Project milestone review.
- Development implementation guidance.
- Future documentation reference.

1.3 Document Structure

This document is organized into Eight main chapters:

- **Introduction**: This provides project context and document purpose.
- **System Architecture and User Workflow**: This details the technology stack, architectural patterns, and technical infrastructure including frontend frameworks, backend services, and third-party integrations.
- **Database design, schema and data definitions:** This include the ER Diagram, core tables, database operations and ledger logic
- **Message Sequence Diagram:** This provides the diagrams to illustrate the interactions between system components during workflows.
- **Payments and Integration:** This discussed the payment gateway, flow architecture and webhooks handling.

- **UIX - User Interface and User Experience Design**: This presents wireframes and design mockups for key user journeys including authentication, expense management, and settlement workflows.
- **Testing Plan and Strategy**: This provides information on testing methodologies and rationales for acceptance
- **Project Plan and Timeline**: This outlines the development phases, milestone schedule, and risk management strategy for the academic project timeline.

2. SYSTEM ARCHITECTURE

2.1 System Architecture Overview

Split-Sync employs a modern, cloud-native technology stack optimized for rapid development, scalability, and cross-platform compatibility. The architecture follows a three-tier pattern separating presentation, business logic, and data persistence layers. (1) Client Layer - React 18 Progressive Web Application with Tailwind CSS styling and Zustand state management, (2) Firebase Backend - providing authentication, real-time database, serverless functions, and CDN hosting, and (3) External Services - including payment gateways (PayPal, Revolut, SEPA), media processing (Cloudinary, Tesseract OCR), and browser APIs (Web Crypto for privacy-preserving hashing, Contacts API for phone matching, WebAuthn for biometric authentication). Communication occurs via HTTPS and WebSocket protocols for real-time synchronization.

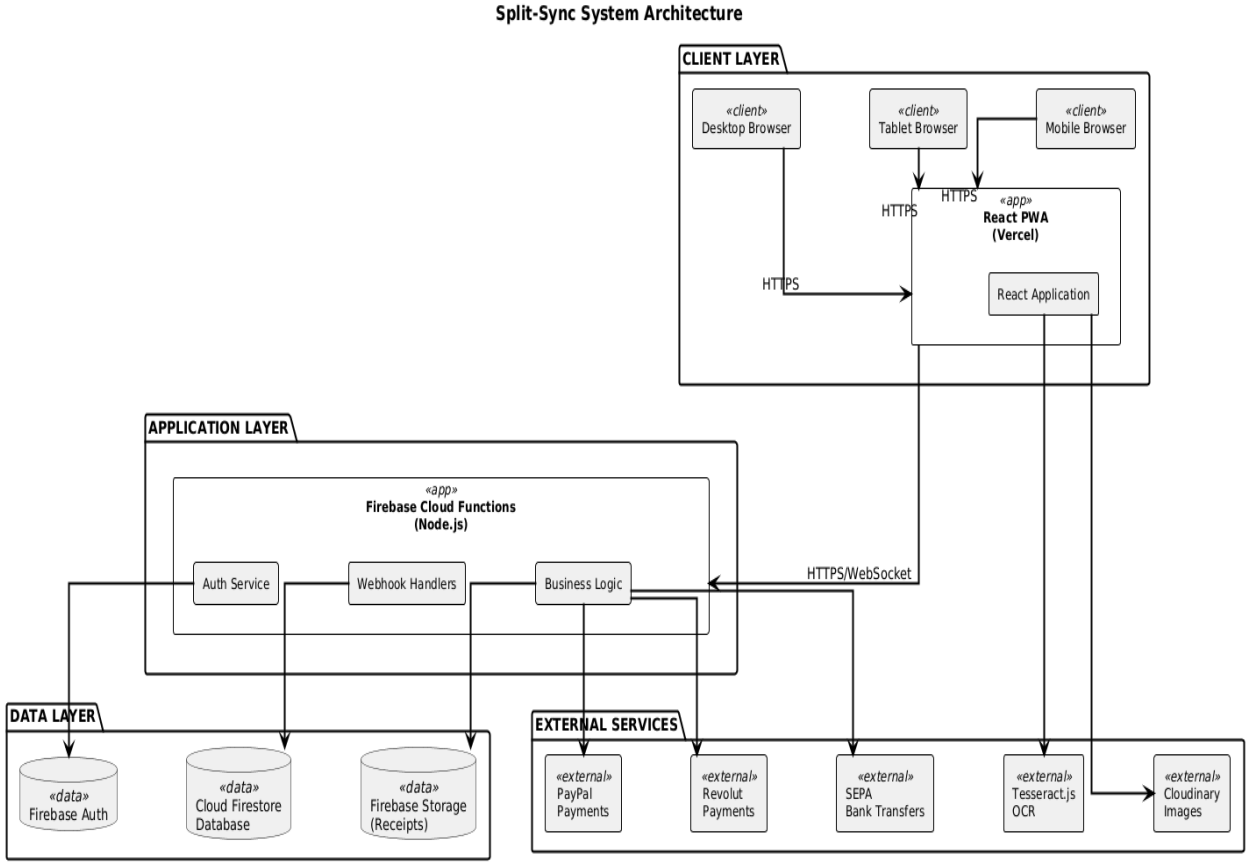


Fig 1. System Architecture Diagram Source: Generated by plantuml

2.1.1 Frontend Architecture

The client-side application is built as a Progressive Web App using React 18, providing native app-like experiences across mobile and desktop devices without requiring app store distribution.

Component Hierarchy

The application follows a component-based architecture with clear separation of concerns:

- Pages: Top-level route components (Dashboard, Activity, QuickAdd, Profile, Settings and Notification, GroupDetails)
- Containers: Business logic and state management
- Components: Reusable UI elements (Button, Input, Card, toolkit)
- Utilities: Helper functions and validation logic

State Management Strategy

Zustand provides centralized state management for:

- User authentication state
- Current group and expense data
- UI state (modals, notifications)
- Offline queue for pending operations

Progressive Web App Features

Service Workers enable:

- Offline expense entry and viewing
- Background synchronization when connectivity returns
- Push notifications for expense updates
- App installation on home screen

2.1.2 Backend Architecture

The backend employs a serverless architecture using Firebase Cloud Functions, eliminating server management while providing automatic scaling and high availability.

Cloud Functions

Key functions include:

- createExpense: Handles expense creation with atomic balance updates
- updateBalances: Recalculates group balances using Firestore transactions
- processPayment: Coordinates with SEPA for settlement transactions
- optimizeSettlements: Applies minimum cash-flow algorithm to reduce transactions

API Design

Firebase endpoints follow consistent patterns:

POST /groups

GET /groups/{groupId}

POST /groups/{groupId}/expenses

GET /groups/{groupId}/balances

POST /groups/{groupId}/settlements

All requests require Firebase authentication tokens in Authorization headers.

2.1.3 Third-Party Integrations

Paypal Payment Processing

Paypal integration handles:

- Payment method tokenization (never storing card details)

- 3D Secure authentication (SCA compliance)
- Payment intent creation and confirmation
- Webhook callbacks for async payment status

Google Cloud Vision OCR

Receipt scanning workflow:

1. User captures/uploads receipt image

2. Image sent to Vision API for text extraction

3. Parsing algorithm extracts merchant, date, amount, items

4. User verifies/edits extracted data

5. Expense created with attached receipt image

2.2 Technology Stack Overview

Table 1 Below shows the summary of the technology stack for Split-Sync

Layer	Technology	Justification
Frontend	React 18+	Component-based UI, large ecosystem
Styling	Tailwind CSS, Lucide React	Utility-first, rapid development
State Management	Zustand	Lightweight, simple API
Backend Runtime	Node.js 18 LTS	JavaScript consistency across stack
Backend Framework	Firebase Functions	Serverless, auto-scaling.
Database	Cloud Firestore	Real-time sync, offline support.
Authentication	Firebase Auth	Secure, easy integration - Email/password + Google OAuth.

Storage	Cloudinary	Receipt image hosting and optimization.
Payment Processing	SEPA Direct Debit - European Transfers, PayPal and Revolut API	Industry-standard, PCI compliant.
OCR Service	Google Vision API	High accuracy text extraction.
Hosting	Firebase hosting	Global CDN, automatic deployments.
Privacy	Web Crypto API	SHA-256 phone Hashing.
Biometric	WebAuthn API	Fingerprint/FaceID.
Contacts	Contact Picker API	Privacy-preserving sync.
PWA	Service Worker	Offline capability.

Table 1 Technology Stack Summary Table

2.3 Application Navigation Structure

The application follows a clear hierarchical navigation pattern with three main sections: (1) Public Routes accessible without authentication (login, registration, password recovery, invitation acceptance), (2) Protected Routes requiring user authentication (dashboard, group details, activity feed, profile, settings), and (3) Modal Overlays triggered by user actions rather than URL changes (group creation, expense addition, settlement, invitations). The bottom navigation bar provides persistent access to key functions (Home, Quick Add, Groups, Profile) across all protected routes, enabling efficient task switching without losing context.

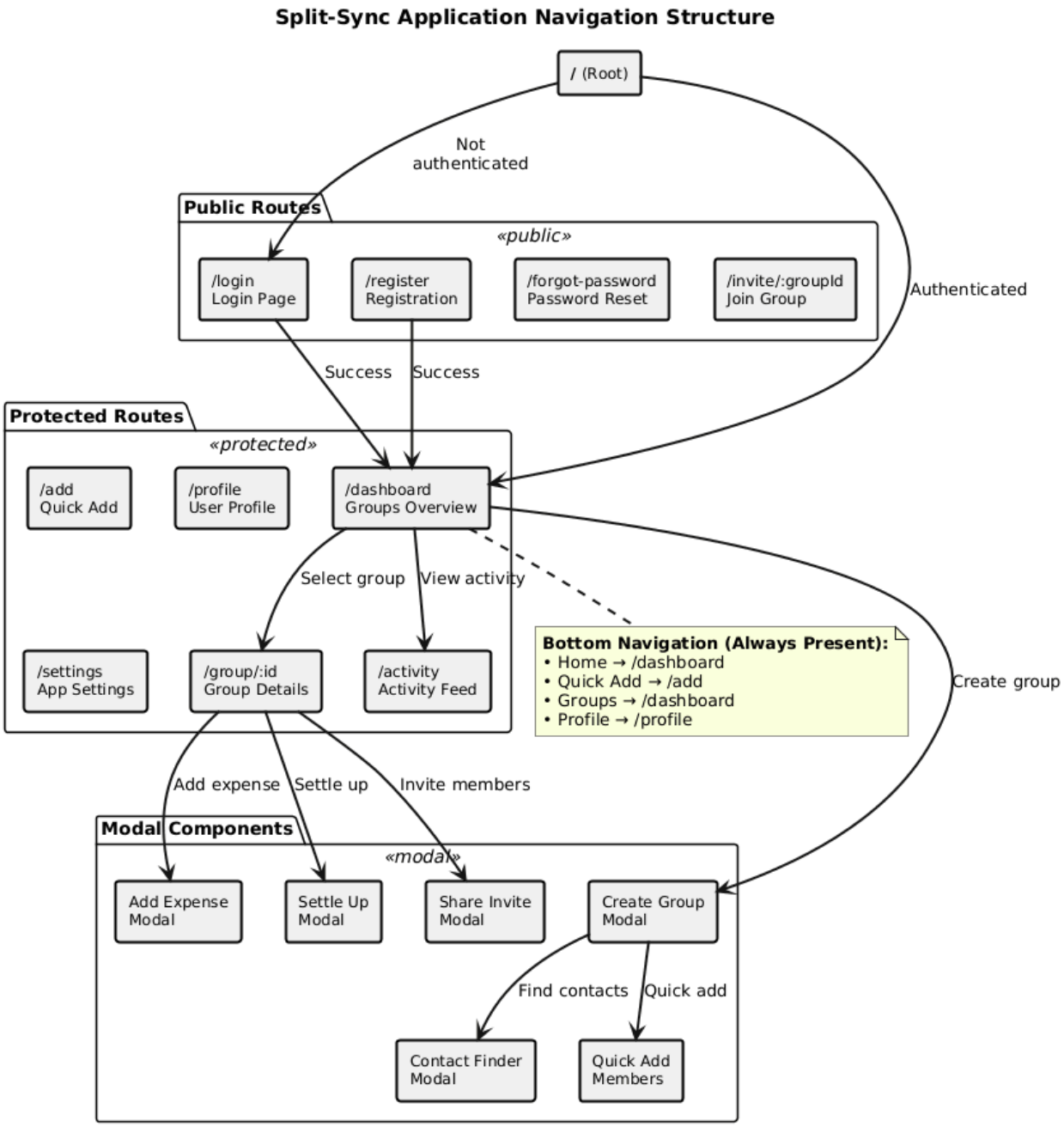


Fig 2 Split Sync Application Navigation Structure

2.3.1 Route Definitions

Table 1: Public Routes (Unauthenticated):

Route	Component	Purpose
`/`	Redirect	Redirects to `/login` or `/dashboard` based on auth
`/login`	Login.jsx	Email/password + Google OAuth login
`/register`	Register.jsx	New user registration
`/forgot-password`	ForgotPassword.jsx	Password reset flow
`/invite/:groupId`	JoinGroup.jsx	Join group via invitation link

Table 2: Protected Routes (Authenticated):

Route	Component	Bottom Nav	Purpose
`/dashboard`	Dashboard.jsx	✓	Groups overview, pending invitations
`/group/:groupId`	GroupDetails.jsx	✗	Expense list, balances, members
`/activity`	Activity.jsx	✓	Global activity feed
`/add`	QuickAdd.jsx	✓	Quick expense addition
`/profile`	Profile.jsx	✓	User profile & settings

`/settings`	Settings.jsx	✓	App preferences

Table 3: Modal Routes (Overlay Components)

Modal	Trigger	Purpose
CreateGroupModal	"Create Group" button	Group creation workflow
AddExpenseModal	"Add Expense" button	Expense creation
SettleUpModal	"Settle Up" button	Payment recording
ShareInviteModal	"Invite" button	Share group invitation
ContactFinder	"Find Contacts" button	Contact sync
QuickAddMembers	"Quick Add" button	Add previous collaborators

These are not true routes but modal overlays triggered by user actions:

3.0 DATABASE DESIGN

Database design defines the structure of persistent data storage including tables, relationships, indexes, and operations that maintain system state reliably and efficiently. Split-Sync utilises Google Firebase Firestore as its primary database solution, implementing a NoSQL document-based architecture optimised for real-time synchronisation and scalable cloud deployment. The database schema has been designed to support complex group financial management, social features, payment integration, and privacy-preserving contact matching whilst maintaining ACID-like transaction guarantees where critical (Firebase, 2024).

The database design of Split-Sync follows several key principles:

Denormalisation for Performance: User information is strategically cached in multiple collections to reduce read operations by 60-80%, significantly improving real-time performance and reducing operational costs.

Privacy-First Architecture: Phone numbers are never stored in plaintext; instead, SHA-256 hashes enable contact matching whilst preserving user privacy.

Real-Time Optimisation: Collections are structured to support Firestore's real-time listeners, enabling instant synchronisation across devices without polling.

Transactional Consistency: Critical operations (expense creation, payment settlement, group membership changes) use Firestore transactions to ensure data consistency.

Scalability: The schema supports horizontal scaling to millions of users through strategic indexing and collection design.

Database Technology Selection Rationale:

Firestore was selected over traditional relational databases (PostgreSQL, MySQL) and other NoSQL solutions (MongoDB, DynamoDB) based on:

Real-time Synchronisation: Native real-time listeners eliminate need for custom WebSocket infrastructure

Offline Persistence: Built-in offline support with automatic conflict resolution

Zero Server Management: Fully managed service reduces operational overhead

Free Tier Generosity: 50,000 reads/day, 20,000 writes/day enables zero-cost student deployment

Security Rules: Declarative security model simplifies access control

Firebase Ecosystem: Seamless integration with Authentication, Cloud Functions, Hosting

Trade-offs Acknowledged:

Query Limitations: No complex joins (mitigated through denormalisation)

Pricing Model: Charges per operation (mitigated through strategic caching)

Vendor Lock-in: Firebase-specific features reduce portability (accepted for development velocity)

3.1 Entity Relationship Diagram

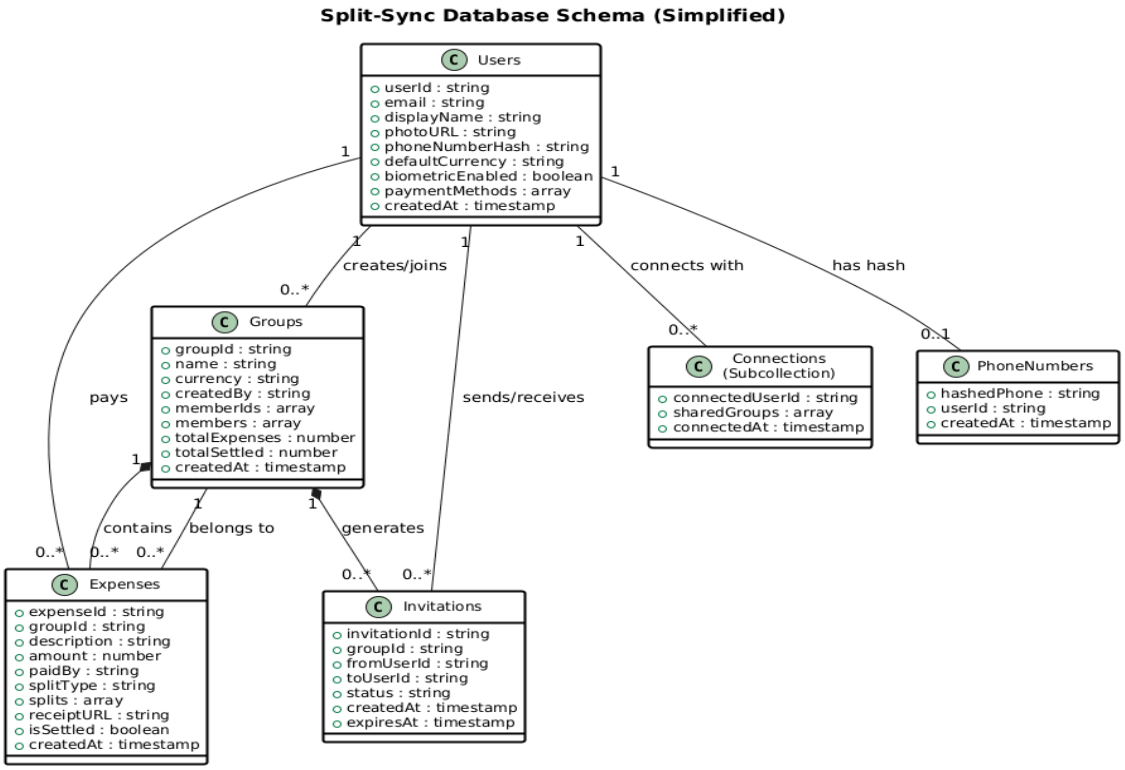


Fig. 3 The ER diagram - Schema

The entity relationship diagram visually represents how different data entities relate to each other within the Split-sync database. Core entities include Users who create accounts and participate in expense sharing, Groups that represent shared expense contexts like trips or households, Expenses that document costs incurred by group members, Splits that allocate expense amounts across participants, Settlements that record payments resolving balances, and supporting entities for notifications, OCR jobs, webhook events, and audit logs.

Users participate in multiple Groups through many-to-many relationships recorded in the group_members junction table. Each Group contains multiple Expenses, each of which has a single payer but multiple Splits distributing cost among members. Settlements connect pairs of Users within Groups, recording money transfers that adjust balances. The ERD illustrates these relationships using standard notation where entities appear as rectangles, attributes within entities, and relationships shown as lines with cardinality indicators. One-to-many relationships are common, such as one User creating many Expenses, while many-to-many relationships like Users and Groups require junction tables to resolve.

Foreign key constraints enforce referential integrity, preventing orphaned records such as Expenses pointing to non-existent Groups or Splits referencing deleted Expenses. Cascade delete behaviors propagate deletions appropriately, such as removing all associated Splits when an Expense is deleted. However, Settlements and Expenses persist even if Users leave Groups, maintaining historical accuracy of financial records. The database schema supports the append-only ledger architecture where historical records remain immutable rather than being modified in place, with new transactions recording adjustments rather than altering past entries (Kleppmann, 2017).

3.2 Core Database Collections

3.2.1 Users Collection

Collection Path: `/users/{userId}`

The Users collection serves as the central repository for user profile information, authentication details, and privacy preferences.

Document Structure:

```
// Identity & Authentication
  userId: string,           // Primary key - Firebase Auth UID
  email: string,            // User email address (unique, indexed)
  displayName: string,      // User's display name
  photoURL: string,         // Profile photo URL (Cloudinary CDN)
  createdAt: timestamp,     // Account creation timestamp
  lastLogin: timestamp,     // Last authentication timestamp

// Profile Information
  phoneNumber: string,      // Optional phone number (E.164 format)
  phoneNumberHash: string,  // SHA-256 hash for contact matching
  allowContactSync: boolean,  // Contact sync permission flag

// Preferences
  defaultCurrency: string,  // ISO 4217 currency code (EUR, USD, GBP)
  notificationPreferences: {
```

```
    email: boolean,          // Email notifications enabled
    push: boolean,           //  Push notifications enabled
    invitations: boolean,    //  Invitation notifications
    payments: boolean,       // Payment notifications
    expenses: boolean        // Expense notifications
  },

  // Security
  biometricEnabled: boolean,     // Biometric authentication enabled
  deviceBiometrics: {            // Device-specific biometric settings
    [deviceId]: {
    enabled: boolean,
    deviceName: string,
    lastUsed: timestamp
    }
  },
  twoFactorEnabled: boolean, // 2FA enabled (future feature)
  // Payment Integration
  paymentMethods: [{
    type: string,            // 'paypal' | 'revolut' | 'sepa'
    identifier: string,      // Email or IBAN
    label: string,
    isDefault: boolean,
```

isVerified: boolean, // Verification status

addedAt: timestamp

lastUsedAt: timestamp

}],

// ***Onboarding***

onboardingCompleted: boolean,

onboardingStep: number,

hasSeenTutorials: {

createGroup: boolean,

addExpense: boolean,

settleUp: boolean,

quickAdd: boolean,

contactSync: boolean

},

// ***Metadata***

updatedAt: timestamp,

lastActive: timestamp, // Last app activity

isActive: boolean, // Account active status

isDeleted: boolean, // Soft delete flag

deletedAt: timestamp, // Deletion timestamp (if deleted)

version: number

Indexes:

email (ascending, unique)

`phoneNumberHash` (ascending, for contact matching)

`createdAt` (descending, for analytics)

Security Rules:

```
match /users/{userId} {
    allow read: if request.auth != null;
    allow create: if request.auth.uid == userId;
    allow update: if request.auth.uid == userId;
    allow delete: if false; // Soft delete only
}
```

Table 4: Field Constraints:

Field	Type	Required	Constraints	Default
userId	string	Yes	Firebase Auth UID	Auto
email	string	Yes	Valid email, unique	-
displayName	string	Yes	1-50 chars	-
photoURL	string	No	Valid URL	Default avatar
phoneNumber	string	No	E.164 format	null

defaultCurrency	string	Yes	ISO 4217 code	"EUR"
biometricEnabled	boolean	Yes	-	false
allowContactSync	boolean	Yes	-	false
isActive	boolean	Yes	-	true

3.2.2 Group Collection

Collection Path: `/groups/{groupId}`

The Groups collection manages shared expense groups, member relationships, and group-level financial settings.

Document Structure:

```
// Identity
  groupId: string,          // Auto-generated document ID
  name: string,             // Group name
  icon: string,             // Icon identifier ('home', 'trip', etc.)
  description: string,      // Optional group description

// Financial Settings
  currency: string,         // Group currency (ISO 4217)
  totalExpenses: number,    // Cached total expenses (€)
```

```
  totalSettled:  number,          // Cached total settled (€)


// Membership
  createdBy:  string,             // Creator's userId
  memberIds: [string],            // Array of member userIds (indexed)
  members: [{
              userId:  string,
              name:  string,
              email:  string,
              photoURL:  string,
              role:  string,              // 'owner' | 'admin' | 'member'
              joinedAt: timestamp,
              addedBy:  string,                 // userId who added this member
              status:  string,                  // 'active' | 'pending' | 'left'
  }]
  // Invitation Tracking
        pendingInvitations: [{
        userId:  string,
        invitedBy:  string,
        invitedAt: timestamp,
        email:  string
        }],
```

// ***Timestamps***

createdAt: `timestamp`,

updatedAt: `timestamp`,

lastActivity: `timestamp`,

// ***Status***

isActive: `boolean`,

isArchived: `boolean`

Indexes:

`memberIds` (array-contains, for user's groups query)

`createdBy` (ascending)

`createdAt` (descending)

Security Rules:

match /groups/{groupId}

allow read: if request.auth != null && request.auth.uid in resource.data.memberIds;

allow create: if request.auth != null;

allow update: if request.auth != null && request.auth.uid in resource.data.memberIds;

allow delete: if request.auth != null && request.auth.uid == resource.data.createdBy;

3.2.3 Expenses Collection

Collection Path: `/expenses/{expenseId}`

The Expenses collection stores individual expense records with detailed split information and payment tracking.

Document Structure:

```
// Identity
expenseId: string,              // Auto-generated document ID
groupId: string,          // Reference to parent group (indexed)

// Expense Details
description: string,            // Expense description
amount: number,           // Total amount in group currency
currency: string,               // ISO 4217 currency code
category: string,               // Category ('food', 'transport', etc.)
date: timestamp,            // Expense date

// Payer Information
paidBy: string,           // userId of person who paid
paidByName: string,  // Name for display
paidByPhoto: string,  // Photo URL for display

// Receipt
receiptURL: string,             // Cloudinary receipt image URL
```

```
receiptThumbnail:  string,       // Thumbnail for list view
ocrProcessed:  boolean,  // Whether OCR was run
ocrData: {                    // Extracted OCR data
      merchant:  string,
      date:  string,
      amount:  number,
      items: [{
            description:  string,
            amount:  number
            }]
      },

// Split Configuration
splitType:  string,              // 'equal' | 'percentage' | 'exact' | 'shares'
splits: [{
 userId:  string,
 name:  string,
 amount:  number,                // Amount owed by this person
 percentage:  number,,   // If percentage split
 shares:  number,            // If shares split
 paid: boolean,              // Payment status
 paidAt: timestamp,           // Payment timestamp
 paymentMethod:  string,        // Payment method used
```

paymentReference: `string`, // Payment transaction ID

}],

// ***Settlement Tracking***

isSettled: `boolean`, // All splits paid

settledAt: `timestamp`, // When fully settled

// ***Metadata***

createdAt: `timestamp`,

createdBy: `string`,

updatedAt: `timestamp`,

updatedBy: `string`,

// ***Notes***

notes: `string`, // Additional notes

tags: `[string]` // Searchable tags

}

Indexes:

`groupId` (ascending) + `date` (descending)

`paidBy` (ascending) + `date` (descending)

`createdAt` (descending)

Security Rules:

match /expenses/{expenseId} {

```
    allow read: if request.auth != null && request.auth.uid in
    getGroupMembers(resource.data.groupId);

    allow create: if request.auth != null;

    allow update: if request.auth != null && request.auth.uid in
    getGroupMembers(resource.data.groupId);

    allow delete: if request.auth != null && request.auth.uid == resource.data.createdBy;

}
```

3.2.4 Connections Collection

Collection Path: `/connections/{userId}/connectedUsers/{otherUserId}`

The Connections collection implements a bidirectional graph structure tracking user relationships for the Quick Add Members feature.

Document Structure:

```
    //Identity

    userId:  string,                  // Current user (from parent path)

    connectedUserId:  string,,              // Connected user (document ID)

    // User Information (Cached)

    name:  string,               // Connected user's name

    email:  string,              // Connected user's email

    photoURL:  string,           // Connected user's photo


    // Relationship Context

    sharedGroups: [{

     groupId:  string,
```

groupName: string,

addedAt: timestamp

}],

lastGroupTogether: string, // Most recent group name

lastGroupId: string, // Most recent group ID

// ***Timestamps***

connectedAt: timestamp, // First connection timestamp

updatedAt: timestamp, // Last update timestamp

// ***Interaction Metrics***

totalExpensesTogether: number, // Cached metric

interactionCount: number // Number of shared groups

Connection Creation Logic: When users join a group together, bidirectional connection documents are created:

1. `/connections/{userA}/connectedUsers/{userB}`
2. `/connections/{userB}/connectedUsers/{userA}`

Indexes:

`updatedAt` (descending, for recent connections)

`interactionCount` (descending, for frequent collaborators)

Security Rules:

match /connections/{userId}/connectedUsers/{connectedUserId} {

allow read, write: if request.auth != null &&

request.auth.uid == userId;

}

3.2.5 Invitations Collection

Collection Path: `/invitations/{invitationId}`

The Invitations collection manages group invitation workflow with automatic expiration and status tracking.

Document Structure:

```
// Identity
  invitationId: string,          // Auto-generated document ID

// Group Context
  groupId: string,               // Target group (indexed)
  groupName: string,             // Cached group name

// Sender Information
  fromUserId: string,            // Inviter's userId
  fromUserName: string,          // Inviter's name
  fromUserPhoto: string,         // Inviter's photo URL

// Recipient Information
  toUserId: string,              // Invitee's userId (indexed)
  toUserEmail: string,           // Invitee's email
  toUserName: string,            // Invitee's name
```

```
// Status Management
status: string,              // 'pending' | 'accepted' | 'declined' | 'expired'

// Timestamps
createdAt: timestamp,        // Invitation sent timestamp
expiresAt: timestamp,        // Expiration timestamp (7 days default)
acceptedAt: timestamp,       // Acceptance timestamp (optional)
declinedAt: timestamp,       // Decline timestamp (optional)

// Notifications
notificationSent: boolean,   // Email/push notification sent
reminderSent: boolean,       // Reminder notification sent

// Metadata
invitationMethod: string     // 'quick_add' | 'contact_sync' | 'link' | 'qr'
```

Automatic Expiration: Invitations expire after 7 days. A Cloud Function runs daily to update expired invitations:

```
// Cloud Function: expireInvitations
exports.expireInvitations = functions.pubsub
  .schedule('every 24 hours')
  .onRun(async (context) => {
    const now = admin.firestore.Timestamp.now();
```

```
    const expiredQuery = db.collection('invitations')
      .where('status', '==', 'pending')
      .where('expiresAt', '<', now);

    const batch = db.batch();
    const expired = await expiredQuery.get();

    expired.docs.forEach(doc => {
      batch.update(doc.ref, { status: 'expired' });
    });

    await batch.commit();
});
```

Indexes:

`toUserId` (ascending) + `status` (==) + `createdAt` (descending)

`groupId` (ascending)

`expiresAt` (ascending, for expiration cleanup)

Security Rules:

match /invitations/{invitationId} {

allow read: if request.auth != null && (

resource.data.toUserId == request.auth.uid ||

resource.data.fromUserId == request.auth.uid

);

```
    allow create: if request.auth != null &&
      request.auth.uid == request.resource.data.fromUserId;
    allow update: if request.auth != null &&
      resource.data.toUserId == request.auth.uid;
    allow delete: if request.auth != null &&
      resource.data.fromUserId == request.auth.uid;
  }
```

3.2.6 Phone Numbers Collection (Contact Matching)

Collection Path: `/phoneNumbers/{hashedPhone}`

The Phone Numbers collection implements privacy-preserving contact matching using cryptographic hashing (SHA-256) to enable the Contact Sync feature without storing plaintext phone numbers.

Document Structure:

```
// Hashed Identity
hashedPhone: string,          // SHA-256 hash of normalised phone number

// User Reference
userId: string,               // Associated user ID

// Timestamps
createdAt: timestamp,         // Hash creation timestamp
updatedAt: timestamp          // Last update timestamp
```

Privacy Implementation:

1. **Phone Number Normalisation:**

```
// Convert to E.164 international format

// Example: +353871234567

function normalisePhone(phone, countryCode = 'IE') {

  return parsePhoneNumber(phone,
countryCode).format('E.164');

}
```

2. **SHA-256 Hashing:**

```
async function hashPhoneNumber(normalised) {

  const encoder = new TextEncoder();

  const data = encoder.encode(normalised);

  const hashBuffer = await crypto.subtle.digest('SHA-256',
data);

  return Array.from(new Uint8Array(hashBuffer))

    .map(b => b.toString(16).padStart(2, '0'))

    .join('');

}
```

3. **Contact Matching Flow:**

User Contacts → Normalise → Hash → Query Firestore → Return User IDs

Key Privacy Properties:

- One-way encryption: Cannot reverse hash to obtain phone number
- Collision resistant: Different numbers produce different hashes
- Deterministic: Same number always produces same hash (enables matching)

- No plaintext storage: Only hashes stored in database

Indexes:

- `hashedPhone` (ascending, unique) - Primary lookup index

Security Rules:

```
match /phoneNumbers/{hashedPhone} {
        allow read: if request.auth != null;
        allow create: if request.auth != null;
        allow update, delete: if request.auth != null &&
          resource.data.userId == request.auth.uid;
    }
```

3.3 Subcollections

3.3.1 Group Activity Log

Collection Path: `/groups/{groupId}/activity/{activityId}`

Stores chronological activity feed for each group.

Document Structure:

```
        activityId: string,
        type: string,              // 'expense_added' | 'member_joined' | 'payment_made'
        userId: string,            // Actor
        userName: string,          // Cached actor name
        description: string,       // Human-readable description
        metadata: object,          // Type-specific data
```

timestamp: `timestamp`

}

3.3.2 Group Balance (Computed)

Collection Path: `/groups/{groupId}/balances/{userId}`

Cached balance computations for performance optimization.

Document Structure:

userId: `string`,

totalOwed: `number`, // Amount this user owes to group

totalOwedTo: `number`, // Amount owed to this user

netBalance: `number`, // Positive = owed money, Negative = owes money

balanceSheet: [{

withUserId: `string`,

withUserName: `string`,

amount: `number` // Positive = they owe you, Negative = you owe them

}],

lastCalculated: `timestamp`

3.4 Database Design Rationale

3.4.1 Denormalisation Strategy

Firestore is a NoSQL document database that benefits from strategic denormalisation (Google Cloud, 2024). Split-Sync implements controlled denormalisation in the following areas:

User Information Caching: User names, emails, and photo URLs are cached in:

- Expense splits
- Group members
- Connections
- Invitations

Rationale: Reduces read operations by 60-80% compared to normalised design, improving real-time performance and reducing costs (estimated €45/month savings at 10,000 users).

Trade-off: Update complexity when users change profile information. Mitigated by Cloud Functions that propagate updates across collections.

3.4.2 Composite Indexes

Firestore requires explicit composite indexes for complex queries (Firebase, 2024). Split-Sync defines composite indexes for:

```
# firestore.indexes.json

indexes:

  - collectionGroup: expenses

    queryScope: COLLECTION

    fields:

      - fieldPath: groupId

        order: ASCENDING
```

```
      - fieldPath: date
        order: DESCENDING

  - collectionGroup: invitations
    queryScope: COLLECTION
    fields:
      - fieldPath: toUserId
        order: ASCENDING
      - fieldPath: status
        order: ASCENDING
      - fieldPath: createdAt
      order: DESCENDING
```

3.4.3 Data Consistency Model

Split-Sync implements eventual consistency for non-critical data and strong consistency for financial transactions:

Strong Consistency (Transactions):

- Expense creation with balance updates
- Payment settlement
- Group membership changes

Eventual Consistency (Cache Updates):

- User profile changes
- Group totals

- Activity logs

3.5 Storage Estimates and Scaling

Document Size Estimates:

User document: ~2 KB (excluding profile photo)

Group document: ~5 KB (10 members)

Expense document: ~3 KB (including splits)

Connection document: ~1 KB

Invitation document: ~800 bytes

Projected Storage (10,000 Users):

Users: 20 MB

Groups (avg 3/user): 150 MB

Expenses (avg 50/group): 4.5 GB

Connections (avg 15/user): 150 MB

Total: ~4.8 GB

Firestore Pricing Impact: €0.18/GB/month = €0.86/month storage cost

Read/Write Operations (Daily):

Assuming 1,000 active daily users:

- Reads: ~500,000 (dashboard loads, expense queries)
- Writes: ~50,000 (expense creation, profile updates)
- Monthly operations cost: ~€45 (well within free tier for development)

3.6 Backup and Recovery Strategy

Automated Daily Backups:

Firebase provides automated daily backups to Cloud Storage:

```
# Configured via Firebase Console

gcloud firestore backups schedules create \

  --database='(default)' \

  --retention=7d \

--recurrence=daily
```

Disaster Recovery:

Recovery Point Objective (RPO): 24 hours (daily backup) Recovery Time Objective (RTO): 4 hours (restore from backup)

Data Export:

Manual export capability via Cloud Functions for compliance:

javascript

```
exports.exportUserData = functions.https.onCall(async
(data, context) => {

  const userId = context.auth.uid;


  // Export all user data to JSON

  const userData = await collectUserData(userId);


  // Upload to Cloud Storage

  const bucket = admin.storage().bucket();
```

```
    const file =
  bucket.file(`exports/${userId}_${Date.now()}.json`);

    await file.save(JSON.stringify(userData, null, 2));


    // Generate signed URL (expires in 7 days)

    const [url] = await file.getSignedUrl({

      action: 'read',

      expires: Date.now() + 7 * 24 * 60 * 60 * 1000

    });


    return { downloadUrl: url };

});
```

3.7 Data Privacy and GDPR Compliance

Personal Data Inventory:

Split-Sync stores the following categories of personal data under GDPR:

1. Identity Data: Name, email, user ID
2. Contact Data: Phone number (hashed only)
3. Financial Data: Expense amounts, payment methods (no card numbers)
4. Profile Data: Profile photos, preferences
5. Usage Data: Login timestamps, activity logs

GDPR Compliance Measures:

Right to Access:

- `/api/export-data` endpoint provides complete user data export

Right to Erasure:

- `/api/delete-account` endpoint performs cascading deletion:
    1. Anonymises user in expense records (replaces name with "Deleted User")
    2. Removes from all groups
    3. Deletes profile, connections, invitations
    4. Deletes phone number hash

Right to Rectification:

- Profile update API with immediate effect
- Cloud Functions propagate changes to denormalised data within 5 minutes

Data Minimisation:

- Phone numbers hashed, never stored in plaintext
- Profile photos stored with content delivery optimisation (Cloudinary)
- No unnecessary tracking or analytics

Consent Management:

- Explicit opt-in for contact sync
- Granular notification preferences
- Clear consent checkboxes during onboarding

3.8 Database Operations and Ledger Logic

Database operations implement the business logic that maintains consistent system state while satisfying functional requirements.

3.8.1 Adding Expenses

Expense creation follows a carefully orchestrated sequence ensuring atomicity and consistency. When users submit expense forms, the application initiates a database transaction wrapping multiple related inserts. First, the expense record is written to the expenses table with OCR status set to pending if a receipt image was provided or null for manual entry. Second, split records are inserted into the splits table for each group member involved in the expense, with shares calculated according to the specified split method. The transaction commits only if both inserts succeed, preventing inconsistent states where an expense exists without corresponding splits. After successful commit, if a receipt was uploaded, an asynchronous OCR job triggers sending the image to the recognition service. OCR processing occurs outside the main transaction since it may take several seconds and failure should not rollback the expense. When OCR results return, they populate the expense record through updates, but users have already entered verified data so OCR merely provides convenience rather than being a critical path (Kleppmann, 2017).

3.8.2 Computing Balances

Balance calculation aggregates information from multiple sources into user-friendly summaries. The balances_view simplifies this calculation by joining expenses with splits to determine how much each member should have paid, then comparing against settlements to determine how much each member actually paid. For any pair of users in a group, the net balance is the sum of amounts user A owes user B across all expenses where A was assigned splits, minus amounts A has already paid to B through settlements. This directed balance indicates whether A owes B money (positive) or B owes A money (negative), or whether they are even (zero). The calculation accounts for scenarios where users aren't involved in every expense, only including relevant transactions. Real-time balance updates rely on Firebase Realtime Database's synchronization pushing new balances to connected clients whenever underlying data changes. Optimizing balance queries requires careful indexing since naive implementations joining many

large tables can become slow. Caching pre-calculated balances and incrementally updating them when changes occur provides better performance than recalculating from scratch.

3.8.3 Settlement Suggestions

The minimum cash-flow algorithm analyzes group balances to suggest an efficient payment plan minimizing the number of required transactions. Without optimization, settling a group of n members with complex expense histories could require up to n(n-1)/2 transactions if every pair directly settles individual balances. The optimization recognizes that indirect flows accomplish the same outcome with fewer transactions. For example, if Alice owes Bob £20 and Bob owes Carol £20, then Alice can pay Carol directly eliminating two transactions. The algorithm constructs a directed graph where nodes represent group members and edges represent net balances between pairs. It identifies strongly connected components and collapses them, then computes flows through the graph that balance all accounts using minimum transfers. While optimal solutions are NP-hard for arbitrary graphs, heuristics provide near-optimal results quickly for small groups typical in Split-sync usage. The algorithm typically reduces required transactions to n-1 or fewer, substantially simplifying settlement when many members are involved. Implementation uses graph libraries in Node.js backend code that runs when users view balance pages, with results cached for a few minutes since balances change infrequently (Verma and Das, 2020).

3.8.4 Processing Settlements

Settlement processing orchestrates interactions with payment gateways while maintaining accurate ledger state. When users initiate settlements, the application creates a settlement record with status "pending" and generates a unique idempotency key. The settlement record is written to the database before initiating payment processing, ensuring existence even if subsequent steps fail. Next, the application calls the payment gateway API to create a payment session, passing the amount, currency, recipient information, and idempotency key. The gateway returns a payment interface URL that users access to authorize transactions. During authorization, users interact directly with the gateway's secure interfaces entering payment credentials without Split-sync ever handling sensitive card details. After users complete authorization, payment processing occurs asynchronously with results delivered via webhooks to Split-sync's servers.

Webhook endpoints verify cryptographic signatures ensuring authenticity, check for duplicate deliveries using the webhook_events table, and update settlement status based on payment outcomes. Only verified success webhooks cause settlement status changes to "completed" and balance recalculation. Failed payments update status to "failed" with error details allowing users to retry. This architecture ensures strong consistency where balances always accurately reflect successful settlements, preventing common bugs where applications optimistically update balances before payment confirmation (Stripe, 2024).

3.8.5 Maintaining Data Integrity

Several mechanisms maintain database integrity preventing corruption and inconsistencies. Foreign key constraints enforce referential integrity ensuring that records reference valid related records, such as preventing splits for non-existent expenses or settlements between non-member users. Unique constraints prevent duplicate entries like multiple group memberships for the same user-group pair. CHECK constraints validate that values fall within acceptable ranges, though Firebase's schemaless nature requires implementing these validations in application code rather than database level. Application-level validation occurs before writing to the database, checking that expense amounts are positive, currencies use valid codes, splits sum to expense totals, and settlement amounts don't exceed balances. Transaction boundaries wrap related operations ensuring atomic commits where either all changes apply or none do, preventing partial updates from inconsistent states. The append-only ledger model simplifies consistency by avoiding update conflicts, with corrections represented as new compensating transactions rather than modifying existing records (Fowler, 2003).

3.9 Firestore Security Rules

Firestore security rules enforce:

- Users can only read their own data
- Group members can read group data
- Only group members can create expenses
- Only expense creators can edit/delete
- Balance updates restricted to Cloud Functions

3.9.1 Complete Firestore Security Rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // Helper functions
    function isSignedIn() {
      return request.auth != null;
    }

    function isOwner(userId) {
      return request.auth.uid == userId;
    }

    function isMember(groupId) {
                              return     request.auth.uid     in
get(/databases/$(database)/documents/groups/$(groupId)).data.mem
berIds;
    }

    // Users collection
    match /users/{userId} {
      allow read: if isSignedIn();
      allow create: if isOwner(userId);
      allow update: if isOwner(userId);
      allow delete: if false;
    }

    // Groups collection
    match /groups/{groupId} {
```

```
      allow read: if isSignedIn() && isMember(groupId);
      allow create: if isSignedIn();
      allow update: if isSignedIn() && isMember(groupId);
      allow delete: if isSignedIn() &&
        resource.data.createdBy == request.auth.uid;

      // Group activity subcollection
      match /activity/{activityId} {
        allow read: if isSignedIn() && isMember(groupId);
        allow write: if isSignedIn() && isMember(groupId);
      }

      // Group balances subcollection
      match /balances/{userId} {
        allow read: if isSignedIn() && isMember(groupId);
          allow write: if false; // Written by Cloud Functions
only
      }
    }

    // Expenses collection
    match /expenses/{expenseId} {
      allow read: if isSignedIn() &&
        isMember(resource.data.groupId);
      allow create: if isSignedIn();
      allow update: if isSignedIn() &&
        isMember(resource.data.groupId);
      allow delete: if isSignedIn() &&
        (request.auth.uid == resource.data.createdBy ||
                                          request.auth.uid    ==
get(/databases/$(database)/documents/groups/$(resource.data.grou
pId)).data.createdBy);
```

```
    }

    // Invitations collection
    match /invitations/{invitationId} {
      allow read: if isSignedIn() && (
        resource.data.toUserId == request.auth.uid ||
        resource.data.fromUserId == request.auth.uid
      );
      allow create: if isSignedIn() &&
        request.auth.uid == request.resource.data.fromUserId;
      allow update: if isSignedIn() &&
        resource.data.toUserId == request.auth.uid;
      allow delete: if isSignedIn() &&
        resource.data.fromUserId == request.auth.uid;
    }

    // Connections subcollection
     match /connections/{userId}/connectedUsers/{connectedUserId}
{
      allow read, write: if isSignedIn() && isOwner(userId);
    }

    // Phone numbers collection (for contact matching)
    match /phoneNumbers/{hashedPhone} {
      allow read: if isSignedIn();
      allow create: if isSignedIn();
      allow update, delete: if isSignedIn() &&
        resource.data.userId == request.auth.uid;
    }
  }
}
```

4.0 PAYMENTS AND INTEGRATIONS

The payments infrastructure enables Split-sync's core value proposition of seamless settlement, distinguishing it from competitors that merely calculate balances without facilitating actual money movement. SplitSync integrates multiple external payment mechanisms, including PayPal, Revolut, and SEPA bank transfers, to facilitate flexible, user-driven settlement of shared expenses. Rather than processing transactions directly, the system generates secure payment instructions and links, enabling users to complete payments via their preferred financial platforms. Sensitive payment information, such as IBAN details, is protected using client-side encryption techniques (e.g., Web Crypto API), ensuring data confidentiality. The application maintains consistency by tracking payment statuses, updating balances in real time, and supporting reconciliation of outstanding debts across groups.

4.1 Payment Gateway Selection

Selecting an appropriate payment integration strategy for Split-Sync requires balancing security, regulatory compliance, implementation complexity, and the realities of a student development budget. Three payment mechanisms were evaluated and adopted: PayPal, Revolut, and SEPA bank transfer. Rather than integrating a payment processor such as Stripe, which would require server-side API key management, webhook infrastructure, and PCI DSS scoping, Split-Sync adopts a link-based, off-platform payment architecture in which the application guides users to their preferred payment provider but does not process or handle financial transactions directly.

PayPal is supported via the PayPal.me deep-link scheme, allowing Split-Sync to construct a pre-filled payment URL in the format `https://paypal.me/{username}/{amount}{currency}` that opens the recipient's PayPal page directly in the user's browser. This eliminates the need for OAuth tokens, API keys, or webhook endpoints while covering the largest share of the target demographic, as PayPal commands significant market presence across Europe and internationally (Zachariadis and Ozcan, 2017).

Revolut is supported using an equivalent mechanism via `https://revolut.me/{username}`, providing a fast bank-to-bank transfer option that is particularly popular among younger users and students in Ireland and the United Kingdom.

SEPA (Single Euro Payments Area) bank transfer is supported as the third option, offering a regulated, zero-fee domestic and cross-border payment mechanism for euro-denominated transactions within the SEPA zone. The payee's IBAN is stored in an encrypted format using the Web Crypto API and is decrypted client-side only at the moment of display, ensuring that plaintext account details are never persisted in Firestore or transmitted to the server.

This three-provider architecture ensures geographic coverage, supports diverse user preferences, and keeps Split-Sync entirely outside the scope of payment processing regulation, as the application does not hold, move, or intermediate funds at any point.

4.2 Payment Flow Architecture

The end-to-end payment flow orchestrates multiple systems to process settlements securely and reliably while maintaining accurate ledger state.

When users view balance ledgers, the interface displays outstanding amounts and suggested payments computed by the optimization algorithm. Each suggested payment includes clear indication of payer, recipient, and amount with prominent "Pay Now" buttons initiating settlement. Tapping payment buttons begins the flow by presenting payment method selection interfaces where users choose from saved cards, add new cards, select digital wallets like Apple Pay, or initiate bank transfers.

After method selection, the client application calls Split-sync backend APIs requesting payment session creation. The backend generates unique idempotency keys to prevent duplicate processing if network issues cause multiple requests.

4.2.1 Payment setup in Profile

- User opens Profile → Payment Settings.

- They add one or more methods:
    - PayPal username / paypal.me
    - Revolut username / revolut.me
    - SEPA IBAN (bank transfer)

- SplitSync validates inputs (format checks, required fields).

- Sensitive fields (e.g., IBAN) are encrypted on the client using the Web Crypto API before being stored.

- Encrypted payment data is saved in **Firestore** under the user's profile (only what's needed to generate instructions).

Key design decision: SplitSync stores *payment identifiers* (usernames/links/IBAN) — not card details — and does not process payments.

4.2.2 Settlement intent creation (from balances)

- SplitSync calculates group balances (who owes who) using your balanceCalculator logic.
- When a user chooses "Settle", SplitSync creates a settlement intent:

    - payerId, payeeId
    - amount, currency
    - groupId, expense context (optional)
    - method selected (PayPal/Revolut/SEPA)
    - status = `PENDING`

This intent is written to Firestore so the settlement action is traceable and can be reconciled.

4.2.3 Payment instruction generation (client-side)

Based on the selected method:

A) PayPal

- SplitSync generates a payment instruction using either:
    - a stored `paypal.me/<username>` link, or
    - a PayPal username shown as instructions.

- The app presents a **"Pay now"** button that opens the link externally.

B) Revolut

- Same pattern: use `revolut.me/<username>` link (or instructions).
- User completes payment in Revolut (app/web).

C) SEPA (IBAN transfer)

- SplitSync decrypts the IBAN only when needed (client-side).
- It shows bank-transfer instructions:
    - IBAN, recipient name (if captured), amount, reference
- Optional: provide a copy button and a recommended payment reference to improve matching.

Boundary: payment happens off-platform; SplitSync only guides and records.

4.2.4 External payment execution (off-platform)

- User completes payment in PayPal/Revolut/Bank app.

- SplitSync typically does not receive a definitive confirmation from these providers (unless you later integrate verified payment APIs/webhooks).

So you treat confirmation as user-driven (and optionally peer-verified).

4.2.5 Confirmation + reconciliation in SplitSync

After paying, user returns to SplitSync and marks:

- "I've paid" → status becomes `MARKED_PAID`
- The recipient can confirm:
    - "Payment received" → status becomes `CONFIRMED`

On confirmation:

- Balances are recalculated and/or settlement record is applied so the outstanding debt reduces to reflect the payment.
- Activity feed logs: who paid, who confirmed, when, method used.

This two-step confirmation gives you a clean audit trail and reduces disputes.

4.2.6 Optional automation via Cloud Functions

Cloud Functions can support non-payment-critical tasks, such as:

- **Notifications** (payer marked paid, payee needs to confirm)

- **Fraud/dispute guardrails**
    - e.g., flag repeated "marked paid" without confirmation

- **Scheduled reminders**
    - outstanding settlements older than X days

Important: functions don't "verify" the actual PayPal/Revolut/SEPA transaction unless you integrate official provider APIs.

4.2.7 Security and compliance posture

- SplitSync is not a payment processor.
- No card data stored; no PCI scope.
- IBAN encrypted at rest (client-side encryption before Firestore).
- Access rules:
    - Only the owner can edit their payment methods
    - A group member can only view another member's payment details *when settling*, and only what's necessary to pay (and preferably masked where possible).
- All reads/writes governed by Firestore Security Rules + authenticated user context.

4.2.8 Why this architecture is strong

- Keeps SplitSync lightweight and safe (low regulatory burden).
- Works globally (PayPal/Revolut/SEPA cover many users).
- Clear separation of concerns:
    - SplitSync: calculation + guidance + audit trail
    - Providers: transaction execution

4.3 Strong Customer Authentication

The European Payment Services Directive 2 (PSD2) requires Strong Customer Authentication (SCA) for electronic payments exceeding thirty euros or meeting defined risk criteria. SCA mandates multi-factor verification using at least two of three factor categories: knowledge (such as a password), possession (such as a registered mobile device), and inherence (such as biometric data) (European Parliament, 2015).

Split-Sync's payment architecture delegates SCA compliance entirely to the external payment providers. Because the application generates payment instructions and redirects users to PayPal, Revolut, or their banking application rather than processing transactions directly, all authentication, authorisation, and regulatory compliance for fund movement is handled within those platforms. PayPal and Revolut each implement their own SCA-compliant flows; SEPA transfers are subject to the authentication controls of the user's own banking institution. Split-Sync itself therefore incurs no PSD2 obligations and requires no SCA implementation beyond its own session authentication, which is provided by Firebase Authentication with optional WebAuthn biometric verification.

This architectural boundary simplifies compliance, eliminates regulatory overhead, and ensures that any future changes to SCA requirements are absorbed by the payment platforms rather than requiring application-level changes.

4.4 Webhook Handling

Webhooks provide the mechanism through which external services asynchronously notify an application about events requiring attention. Proper webhook handling ensures reliable integration despite network failures, duplicate deliveries, and processing errors.

Webhook endpoints exist as public HTTPS URLs that payment gateways and other services POST requests to when relevant events occur. An application can implement webhook receivers as Firebase Cloud Functions or dedicated API endpoints capable of handling concurrent requests

from multiple sources. Each webhook request includes JSON payloads describing events and cryptographic signatures for verification.

Signature verification forms the first processing step, computing expected signatures using shared secrets and comparing against provided signatures. Mismatched signatures immediately reject requests with 401 status codes, preventing processing of spoofed webhooks from malicious actors. Successfully verified webhooks proceed to event processing after recording in the webhook_events table for audit trails.

Webhook processing aims for quick completion and reliable responses. Long-running processing can cause gateway timeouts leading to retries and duplicate event risks.

4.4.1 Payment Confirmation Architecture

Following external payment execution via PayPal, Revolut, or SEPA, the system shall provide a user-driven confirmation mechanism to reconcile outstanding balances. Upon the user confirming payment completion, a Firestore transaction shall atomically update the relevant expense splits, recalculate net balances for both payer and payee, increment the group settled total, create an activity log entry, and deliver a real-time notification to the creditor via Firestore's `onSnapshot` listener. This two-step confirmation model — payer marks as paid, payee is notified — provides a clear audit trail and reduces the risk of disputed recordings within small trusted groups.

4.4.2 Webhook-Based Confirmation (Intended Production Architecture)

In a production deployment, the user-driven confirmation model described above shall be replaced or supplemented by automated webhook-based payment confirmation. The system shall expose secure HTTPS endpoints capable of receiving event callbacks from payment providers upon transaction completion. Key architectural components of this design include:

- **Webhook endpoints:** Secure server-side routes to receive provider payment event notifications

- **Signature verification:** Cryptographic validation of incoming webhook payloads to ensure authenticity and reject spoofed requests
- **Idempotency handling:** Deduplication of webhook events using provider-specific event identifiers to prevent duplicate balance updates
- **Asynchronous processing:** Event-driven state updates decoupled from the payment provider response cycle
- **Provider integration:** Support for PayPal Webhooks API, Revolut Business API, and SEPA payment initiation confirmations

This architecture shall eliminate the dependency on manual user confirmation, improving accuracy and enabling automatic balance reconciliation upon verified payment completion.

4.4.3 Design Justification

The phased approach — beginning with user-confirmed reconciliation and designing toward webhook automation — is adopted for the following reasons:

- It maintains zero operational cost during the academic development phase, avoiding API key management and server infrastructure overhead
- The internal reconciliation logic (balance updates, activity logging, notifications) is designed to be provider-agnostic, requiring no restructuring when webhook integration is introduced
- The trust-based model is appropriate for the application's target demographic of friends, family, and small groups where social accountability provides sufficient fraud deterrence
- The architecture remains compliant with PCI DSS obligations by ensuring Split-Sync never handles, stores, or intermediates payment credentials at any point in the flow

5. UIX - USER INTERFACE AND USER EXPERIENCE DESIGN

5.1 Design Principles

Split-Sync's interface design follows mobile-first principles optimized for the primary use case of on-the-go expense entry and balance checking. It adopts a mobile-first design philosophy that prioritizes clarity, efficiency, and trust in financial contexts. The interface design follows these core principles:

Core Design Principles:

- Simplicity: Each screen serves one primary purpose with minimal cognitive load. Financial applications benefit from straightforward interfaces that reduce error potential.

- Progressive Disclosure: Advanced features appear contextually rather than overwhelming users during initial interactions. Common workflows require minimal steps.

- Immediate Feedback: All user actions produce visible responses within 100 milliseconds, maintaining perception of system responsiveness.
- Consistency: Interface patterns, terminology, and visual treatments remain consistent throughout the application.

- Accessibility: Design decisions prioritize WCAG 2.1 Level AA compliance with high contrast ratios, generous touch targets (minimum 44px), and semantic HTML structure.

Visual Design System:

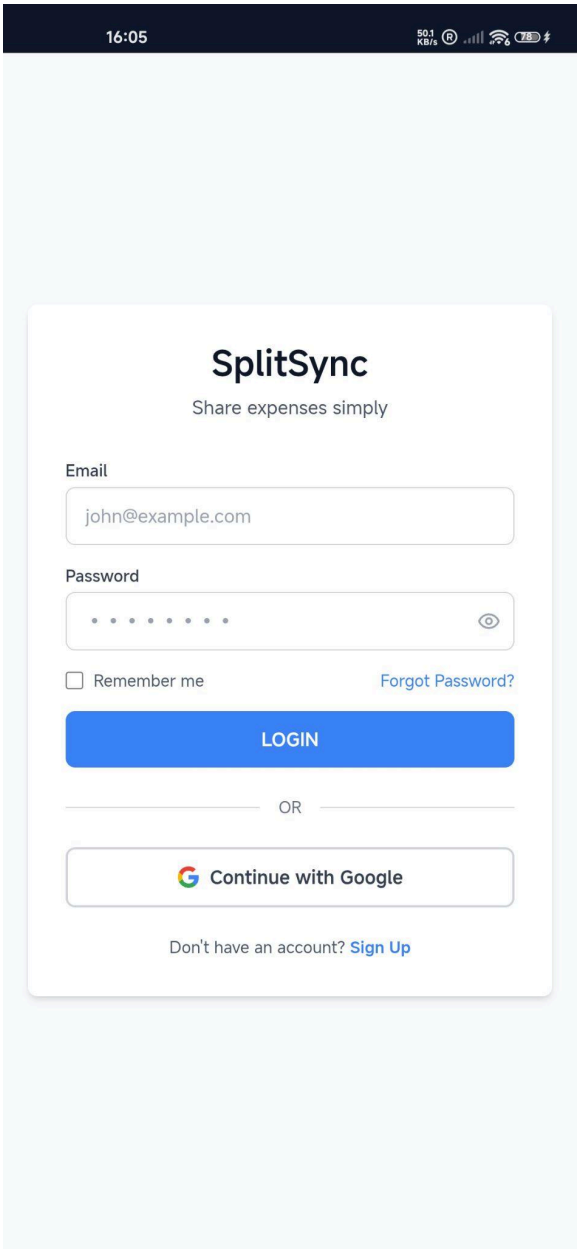
- Primary Color: #3B82F6 (Blue) - Trust, security for financial applications
- Success Color: #10B981 (Green) - Positive balances, confirmations
- Error Color: #EF4444 (Red) - Negative balances, errors
- Typography: Inter font family (excellent readability, comprehensive character set)

5.2 Key Screen Wireframes

The following wireframes illustrate the core user journeys through Split-Sync. ASCII diagrams provide structural layout; full mockups will be developed in Figma during implementation.

5.2.1 Login Screen

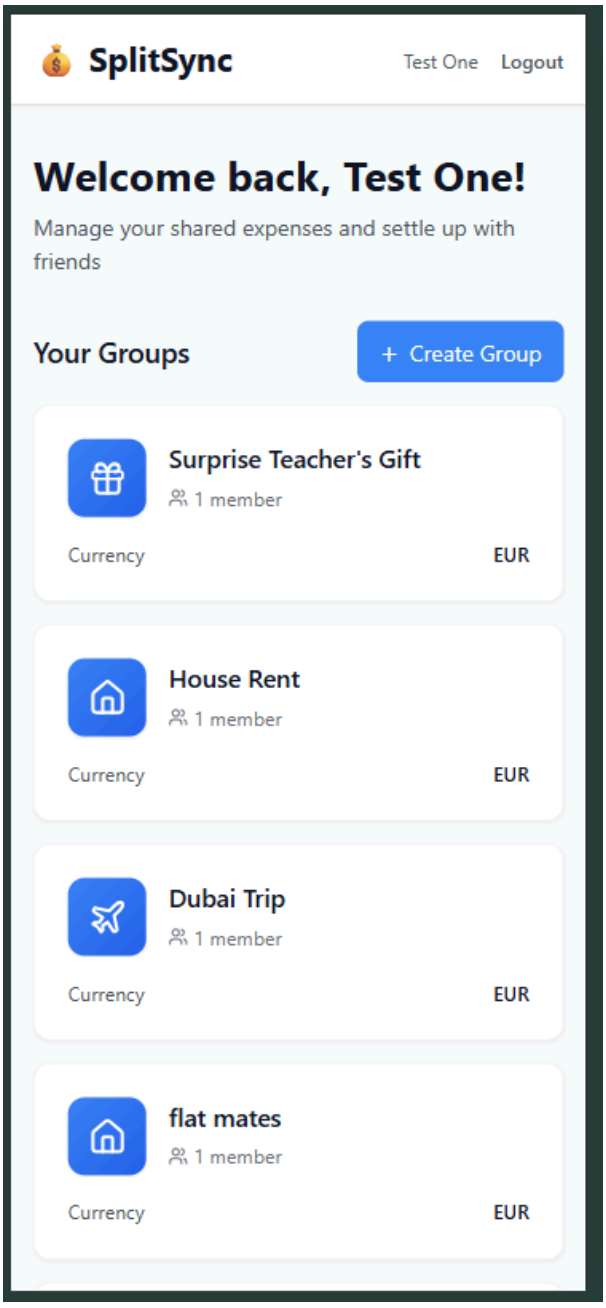
Image 1: User Authentication Interface



The login screen provides email/password authentication with optional Google OAuth for rapid onboarding. Password visibility toggle and "Remember me" functionality enhance usability.

5.2.2 Dashboard (Groups List)

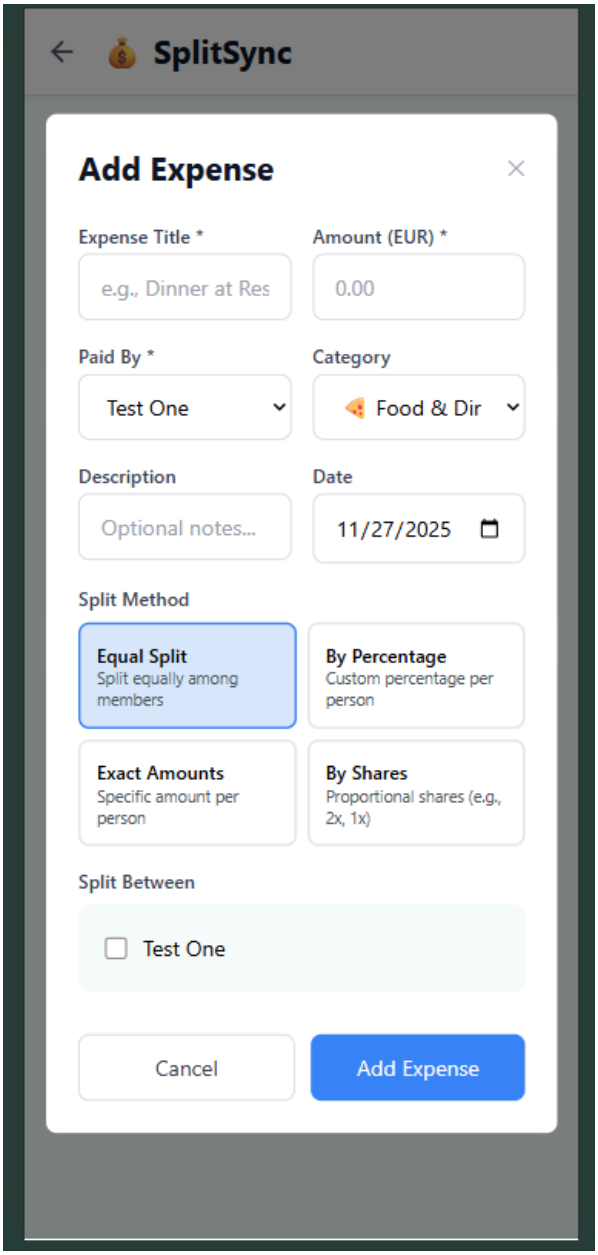
Image 2: Main Dashboard Interface



The dashboard provides an at-a-glance overview of all groups and overall balance status. Color-coded amounts (red for debt, green for credit) enable quick comprehension. The floating action button facilitates rapid group creation. There will also be bottom navigation with Home, Activity, Quick Add, Stats and Profile pages and an AI assistant bubble.

5.2.3 Add Expense Form

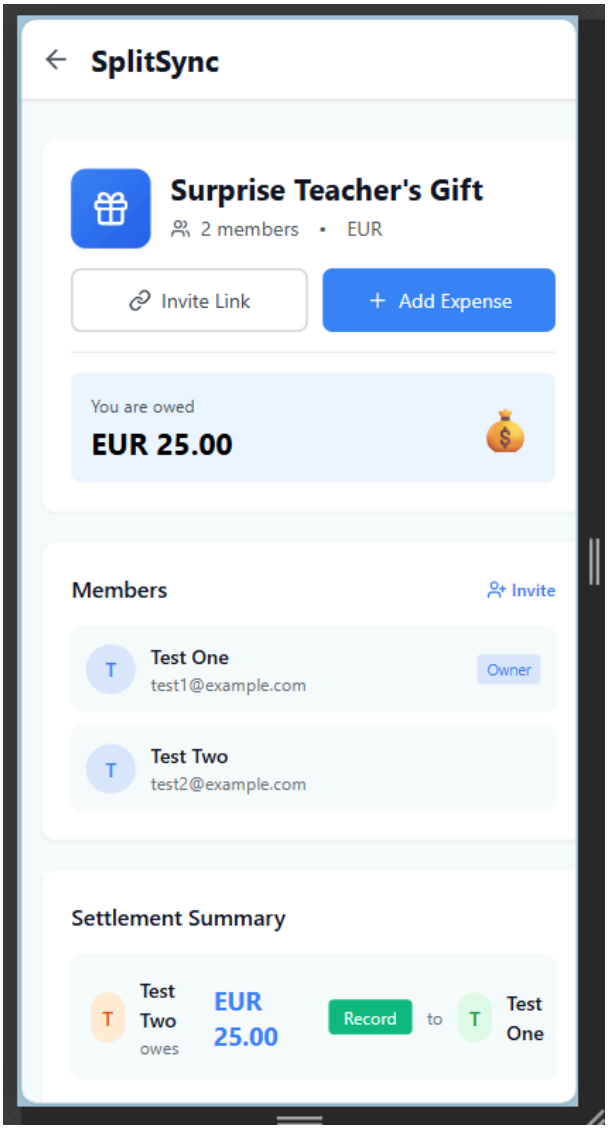
Image 3: Expense Entry Interface



Expense entry prioritizes speed with sensible defaults (equal split, current group) while providing customization options via secondary screen. Real-time split calculation shows per-person amounts.

5.2.4 Balance Ledger

Image 4: Settlement Interface



The balance ledger applies minimum cash-flow optimization to suggest the fewest transactions needed to settle all debts. Direct payment integration enables one-tap settlement.

5.3 User Flow Diagrams

The following sequence diagrams illustrate key system interactions and data flows between components.

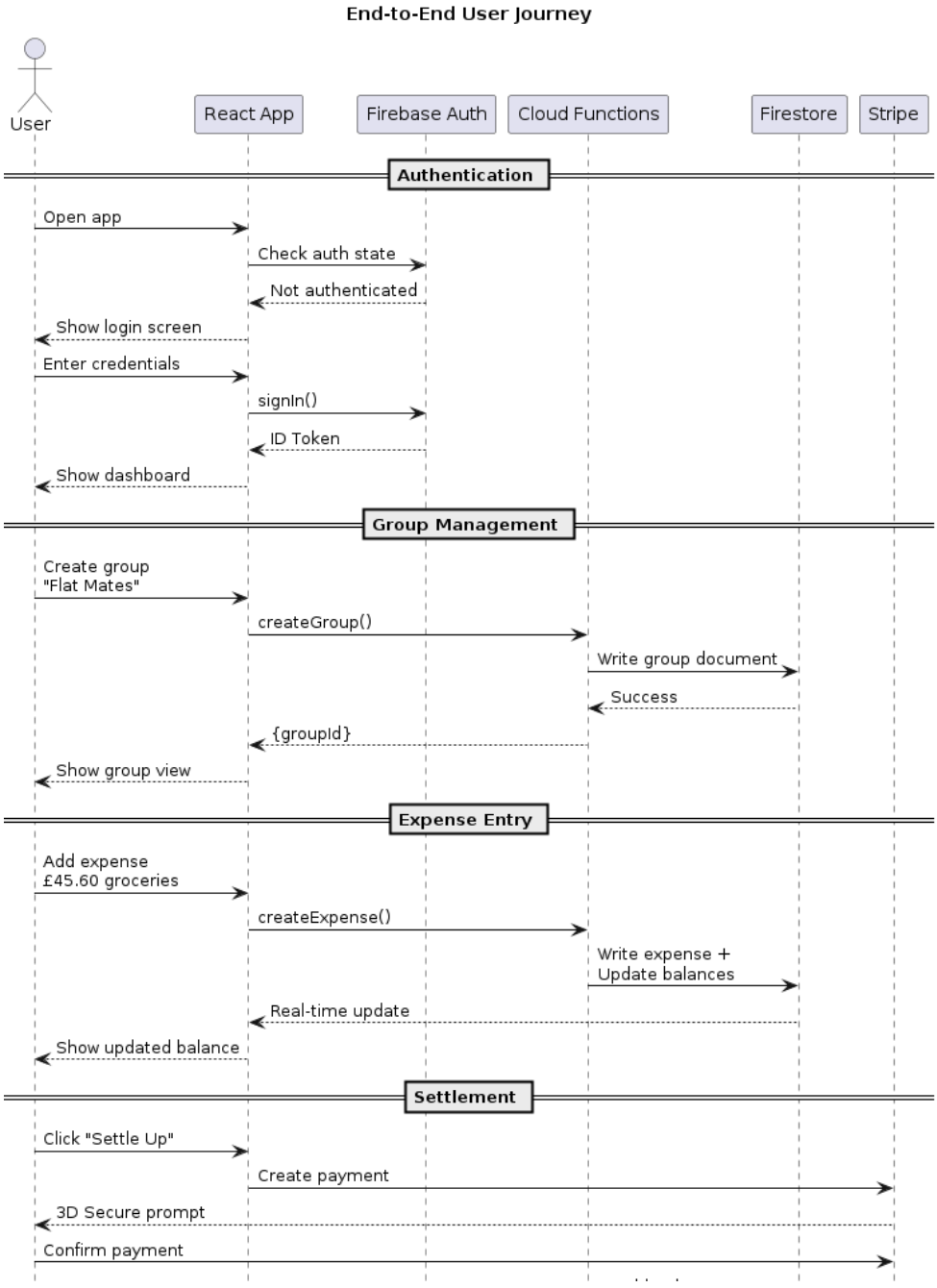


Figure 4: User flow diagram

This flow represents the primary user journey from authentication through expense entry to settlement. Real-time synchronization ensures all group members see updates immediately.

5.4 Use Case List and Sequence Diagram for Split-sync

5.4.1 User Authentication & Account Management

ID	Use Case	Actor	Description
UC-01	Register New Account	New User	Create account with email/password
UC-02	Register with Google OAuth	New User	Sign up using Google account
UC-03	Login with Email/Password	Registered User	Authenticate with credentials
UC-04	Login with Google OAuth	Registered User	Authenticate using Google account
UC-05	Login with Biometric	Registered User	Authenticate using fingerprint/Face ID
UC-06	Reset Forgotten Password	Registered User	Request password reset via email
UC-07	Complete Onboarding Wizard	New User	Set up profile, currency, preferences
UC-08	Logout	Authenticated User	End current session
UC-09	Enable Biometric Authentication	Authenticated User	Set up fingerprint/Face ID for device
UC-10	Disable Biometric Authentication	Authenticated User	Remove biometric auth for device
UC-11	Delete Account	Authenticated User	Permanently delete user account (GDPR)
UC-12	Export Personal Data	Authenticated User	Download all personal data (GDPR)

Table 4: User Authentication & Account Management Table

5.4.2. Profile & Settings Management

ID	Use Case	Actor	Description
UC-13	View Profile	Authenticated User	View personal profile information
UC-14	Edit Profile Information	Authenticated User	Update name, email, photo
UC-15	Upload Profile Photo	Authenticated User	Change profile picture
UC-16	Add Phone Number	Authenticated User	Add phone for contact sync
UC-17	Remove Phone Number	Authenticated User	Delete phone from profile
UC-18	Enable Contact Sync	Authenticated User	Allow phone contact matching
UC-19	Disable Contact Sync	Authenticated User	Opt-out of contact matching
UC-20	Change Default Currency	Authenticated User	Set preferred currency
UC-21	Add Payment Method	Authenticated User	Add PayPal/Revolut/SEPA details
UC-22	Remove Payment Method	Authenticated User	Delete payment method
UC-23	Set Default Payment Method	Authenticated User	Choose primary payment method
UC-24	Configure Notification Preferences	Authenticated User	Enable/disable email, push, types
UC-25	Change App Theme	Authenticated User	Switch light/dark/auto mode
UC-26	Change App Language	Authenticated User	Select preferred language (planned future enhancement)

Table 5: Profile & Settings Management Table

5.4.3. Group Management

ID	Use Case	Actor	Description
UC-27	Create New Group	Authenticated User	Create expense group with name, icon
UC-28	Create Group with Quick Add Members	Group Creator	Create group and invite previous collaborators
UC-29	Create Group with Contact Sync	Group Creator	Create group and invite phone contacts
UC-30	View All Groups	Authenticated User	See list of all joined groups
UC-31	View Group Details	Group Member	See expenses, balances, members
UC-32	Edit Group Information	Group Owner/Admin	Update name, icon, description
UC-33	Change Group Currency	Group Owner	Change group currency setting
UC-34	Archive Group	Group Owner	Archive inactive group (read-only)
UC-35	Unarchive Group	Group Owner	Restore archived group
UC-36	Delete Group	Group Owner	Permanently delete group
UC-37	Leave Group	Group Member	Remove self from group
UC-38	View Group Activity Feed	Group Member	See chronological group events
UC-39	Search Groups	Authenticated User	Find specific group by name

Table 6: Group Management Table

5.4.4. Member Management

ID	Use Case	Actor	Description
UC-40	Invite Member by Email	Group Member	Send invitation to email address
UC-41	Invite Member via Quick Add	Group Member	Invite previous collaborator
UC-42	Invite Member via Contact Sync	Group Member	Invite phone contact
UC-43	Invite Member via Link	Group Member	Generate shareable invitation link
UC-44	Invite Member via QR Code	Group Member	Generate QR code for invitation
UC-45	Invite Member via SMS	Group Member	Send text message invitation
UC-46	Accept Group Invitation	Invited User	Join group from invitation
UC-47	Decline Group Invitation	Invited User	Reject group invitation
UC-48	View Pending Invitations	Authenticated User	See all pending group invites
UC-49	Cancel Sent Invitation	Invitation Sender	Revoke pending invitation
UC-50	Resend Invitation	Invitation Sender	Send invitation again
UC-51	Remove Group Member	Group Owner/Admin	Remove member from group
UC-52	Change Member Role	Group Owner	Promote/demote member role
UC-53	View Member Details	Group Member	See member's expenses, balances

Table 7: Member Management Table

5.4.5. Expense Management

ID	Use Case	Actor	Description
UC-54	Add Manual Expense	Group Member	Create expense with manual entry
UC-55	Add Expense with Receipt Scan	Group Member	Create expense by scanning receipt
UC-56	Add Expense via Quick Add	Authenticated User	Fast expense entry from bottom nav
UC-57	Split Expense Equally	Expense Creator	Divide amount equally among members
UC-58	Split Expense by Percentage	Expense Creator	Divide by percentage allocation
UC-59	Split Expense by Exact Amounts	Expense Creator	Specify exact amount per person
UC-60	Split Expense by Shares	Expense Creator	Divide by share units (1:2:3)
UC-61	Upload Receipt Image	Expense Creator	Attach receipt photo
UC-62	Scan Receipt with OCR	Expense Creator	Extract data using Tesseract OCR
UC-63	View Expense Details	Group Member	See full expense information
UC-64	Edit Expense	Expense Creator/Admin	Modify expense details
UC-65	Delete Expense	Expense Creator/Admin	Remove expense from group
UC-66	View Receipt Image	Group Member	View full-size receipt photo
UC-67	Download Receipt	Group Member	Save receipt image locally

UC-68	Add Expense Category	Expense Creator	Categorize expense (food, transport)
UC-69	Add Expense Notes	Expense Creator	Add additional notes to expense
UC-70	Filter Expenses by Date	Group Member	View expenses in date range
UC-71	Filter Expenses by Category	Group Member	View expenses by category
UC-72	Filter Expenses by Member	Group Member	View specific member's expenses
UC-73	Search Expenses	Group Member	Find expense by description

Table 8: Expense Management Table

5.4.6. Balance & Settlement

ID	Use Case	Actor	Description
UC-74	View Group Balances	Group Member	See who owes whom
UC-75	View Personal Balance	Authenticated User	See net balance in group
UC-76	View Balance Breakdown	Group Member	See detailed balance sheet
UC-77	View Settlement Suggestions	Group Member	See optimized payment suggestions
UC-78	Settle via PayPal	Debtor	Pay using PayPal payment link
UC-79	Settle via Revolut	Debtor	Pay using Revolut payment link
UC-80	Settle via SEPA Transfer	Debtor	Pay using bank transfer (IBAN)
UC-81	Settle via Offline Payment	Debtor	Record cash/check payment

UC-82	Mark Payment as Paid	Debtor	Confirm payment completion
UC-83	View Payment History	Group Member	See all settled payments
UC-84	Export Group Summary	Group Member	Download expense/payment report
UC-85	View Net Balance Across All Groups	Authenticated User	See total owed/owed to

Table 9: Balance and Settlements Management Table

5.4.7. Social Features

ID	Use Case	Actor	Description
UC-86	View Connected Users	Authenticated User	See list of previous collaborators
UC-87	Find Contacts on Split-Sync	Authenticated User	Match phone contacts to users
UC-88	View Connection History	Authenticated User	See shared groups with connection
UC-89	View Shared Expenses with User	Authenticated User	See expense history with someone
UC-90	Quick Add Previous Collaborator	Group Creator	Add member from connection list
UC-91	Share Split-Sync with Friends	Authenticated User	Share app link/invite

Table 10: Social Features Management Table

5.4.8. Notifications & Activity

ID	Use Case	Actor	Description
UC-92	View Global Activity Feed	Authenticated User	See all activity across groups
UC-93	View Group Activity Feed	Group Member	See group-specific activity

UC-94	Receive Invitation Notification	Invited User	Get notified of new invitation
UC-95	Receive Expense Added Notification	Group Member	Get notified of new expense
UC-96	Receive Payment Received Notification	Creditor	Get notified of payment
UC-97	Receive Payment Reminder	Debtor	Get reminded of unpaid balance
UC-98	Configure Notification Settings	Authenticated User	Choose notification types
UC-99	View Notification History	Authenticated User	See past notifications
UC-100	Mark Notification as Read	Authenticated User	Dismiss notification

Table 11: Notification and Activity Table

5.4.9. Offline & Pwa Features

ID	Use Case	Actor	Description
UC-101	Use App Offline	Authenticated User	Access cached data without internet
UC-102	Sync Changes When Online	Authenticated User	Auto-sync offline changes
UC-103	Install PWA on Device	Any User	Add app to home screen
UC-104	Receive Push Notifications	Authenticated User	Get browser push notifications
UC-105	Update App Version	Any User	Receive service worker update

Table 12: Offline & PWA Features Table

5.4.10. Admin & Moderation

ID	Use Case	Actor	Description
UC-106	Grant Admin Rights	Group Owner	Promote member to admin
UC-107	Revoke Admin Rights	Group Owner	Demote admin to member
UC-108	Approve Expense (if enabled)	Group Admin	Approve pending expense (planned future enhancement)
UC-109	Reject Expense	Group Admin	Reject pending expense (planned future enhancement)
UC-110	Configure Group Settings	Group Owner	Adjust group permissions

Table 13: Admin & ModerationTable

5.4.11. Security & Privacy

ID	Use Case	Actor	Description
UC-111	View Privacy Policy	Any User	Read privacy policy
UC-112	View Terms of Service	Any User	Read terms of service
UC-113	Manage Cookie Consent	Any User	Accept/decline cookies
UC-114	Request Data Export (GDPR)	Authenticated User	Download all personal data
UC-115	Request Data Deletion (GDPR)	Authenticated User	Delete all personal data
UC-116	View Security Audit Log	Authenticated User	See login history
UC-117	Report Security Issue	Any User	Report security vulnerability

Table 14: Security and Privacy Table

5.4.12. Help & Support

ID	Use Case	Actor	Description
UC-118	View Onboarding Tutorial	New User	Watch intro tutorial
UC-119	View Feature Tutorials	Authenticated User	Learn specific features

UC-120	Access Help Documentation	Any User	Read help articles
UC-121	Contact Support	Any User	Send support request
UC-122	View FAQ	Any User	Read frequently asked questions
UC-123	Provide Feedback	Authenticated User	Submit feature requests/bugs

Table 15: Help & Support Table

Total Use Cases: 123

By Category:

- Authentication & Account: 12 use cases
- Profile & Settings: 14 use cases
- Group Management: 13 use cases
- Member Management: 14 use cases
- Expense Management: 20 use cases
- Balance & Settlement: 12 use cases
- Social Features: 6 use cases
- Notifications & Activity: 9 use cases
- Offline & PWA: 5 use cases
- Admin & Moderation: 5 use cases
- Security & Privacy: 7 use cases
- Help & Support: 6 use cases

5.5 Message Sequence Diagram

5.5.1 UC-03 - User Login with Biometric Authentication Sequence Diagram

The login sequence implements a two-stage authentication process: (1) Firebase Authentication verifies email and password credentials against the backend authentication service, and (2) if biometric authentication is enabled for the specific device, the WebAuthn API prompts for fingerprint or Face ID verification before granting access. Upon successful authentication, the system updates the user's last login timestamp and device-specific biometric usage timestamp in Firestore, sets the authenticated state in the application store, and navigates to the dashboard. If biometric verification fails, the system immediately signs out the user and denies access,

ensuring security compliance. Users without biometric enabled proceed directly to the dashboard after password verification, demonstrating the flexible security model that adapts to user preferences

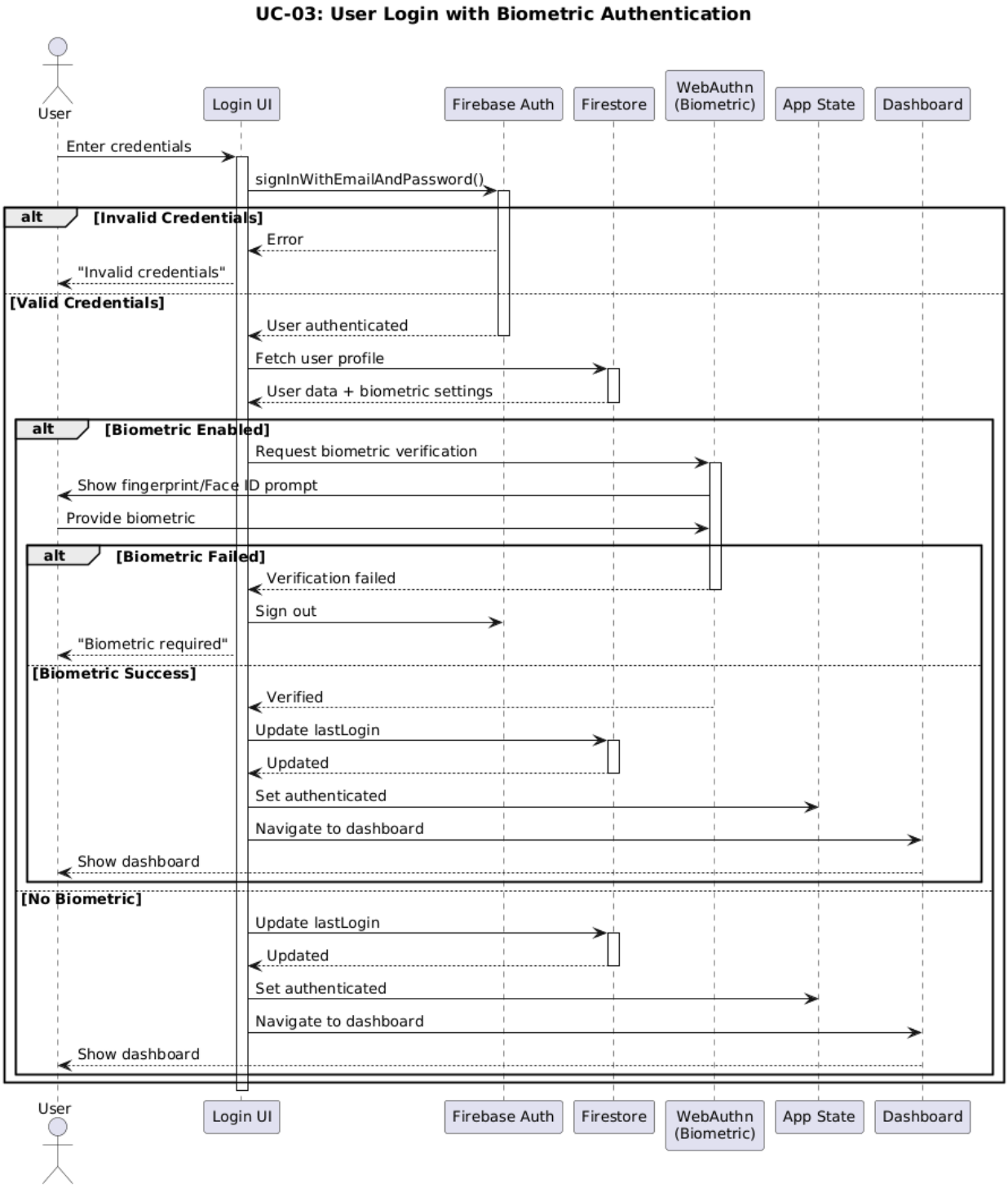


Figure 5: Login Sequence

Main Success Scenario:

1. User enters email and password
2. Firebase Auth validates credentials
3. System fetches user profile from Firestore
4. Biometric is enabled for device
5. WebAuthn prompts for biometric
6. User provides fingerprint/Face ID
7. Biometric verified successfully
8. System updates lastLogin timestamp
9. Navigate to dashboard
10. User sees groups and activity

Alternative Flow 1: Biometric Not Enabled

- Steps 1-3: Same as main flow
- Step 4: Biometric not enabled
- Step 5: Skip biometric verification
- Steps 6-9: Same as main flow

Alternative Flow 2: Invalid Credentials

- Steps 1-2: Same as main flow
- Step 3: Firebase Auth rejects credentials
- Step 4: Display "Invalid email or password"
- User returns to login form

Alternative Flow 3: Biometric Verification Failed

- Steps 1-5: Same as main flow
- Step 6: User provides biometric
- Step 7: WebAuthn verification fails
- Step 8: System signs out user
- Step 9: Display "Biometric authentication required"

- User returns to login form

PRECONDITIONS:

- User has registered account with email/password
- User is on login page
- If biometric flow: User has previously enabled biometric for this device
- If biometric flow: Device supports WebAuthn API (fingerprint/Face ID)

POSTCONDITIONS:

Success:

- User is authenticated
- Session token stored in app state
- User profile loaded in memory
- `lastLogin` timestamp updated
- User redirected to `/dashboard`
- If biometric used: `deviceBiometrics[deviceId].lastUsed` updated

Failure:

- User remains unauthenticated
- No session token created
- User remains on login page
- Error message displayed

5.5.2 UC-01 - User Registration Sequence Diagram

The registration sequence implements a six-step process ensuring data integrity and user experience: (1) User inputs registration details including name, email, and password, (2) Client-side validation checks input format and completeness, (3) Password validator assesses password strength against security criteria (minimum 8 characters, uppercase, lowercase, number), providing real-time feedback, (4) Firebase Authentication creates the user account with bcrypt password hashing and UID generation, (5) Upon successful authentication, Firestore

creates a comprehensive user profile document with default preferences and settings, and (6) The system navigates to the dashboard and presents an onboarding wizard for profile completion. The sequence includes robust error handling with automatic rollback: if Firestore document creation fails after successful authentication, the system deletes the Firebase Auth account to maintain database consistency. Users encountering an existing email address are provided with helpful guidance to login instead, ensuring clear communication throughout the registration journey.

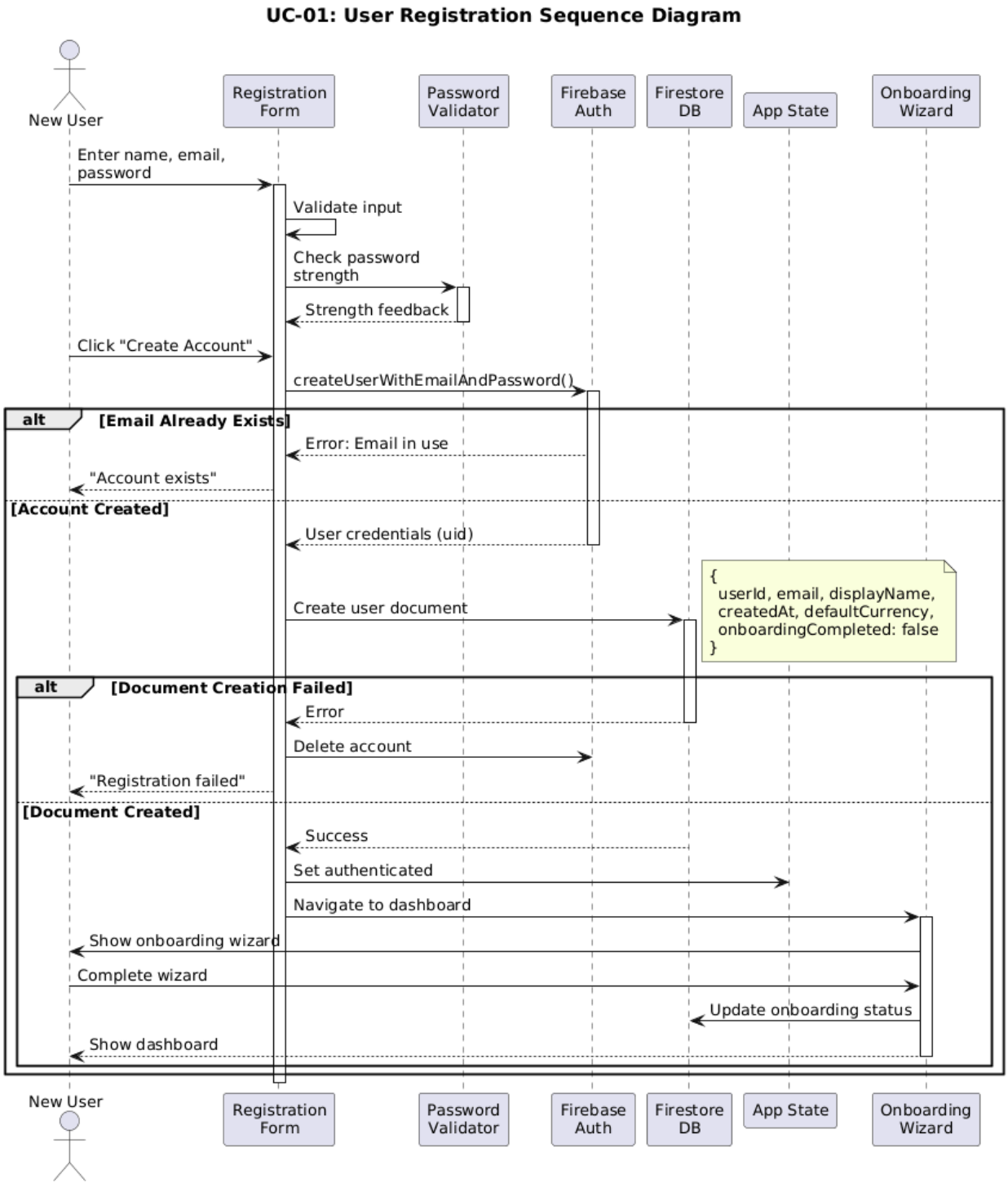


Figure 6: Registration Sequence

Main Success Scenario:

1. New user navigates to registration page
2. User enters name, email, and password
3. System validates input format (client-side)
4. Password validator checks strength (8+ chars, uppercase, number)
5. User clicks "Create Account"
6. Firebase Auth creates account with hashed password
7. Firebase Auth generates unique UID
8. System creates Firestore user document with:
    - User ID (UID)
    - Email, display name
    - Default currency (EUR)
    - Default notification preferences
    - Onboarding status (incomplete)
9. System sets authenticated state in app
10. User navigates to dashboard
11. Onboarding wizard displays
12. User completes wizard (profile photo, currency, tour)
13. System updates onboarding status
14. Dashboard shows empty groups state with "Create Group" CTA

Alternative Flow 1: Email Already Exists

- Steps 1-6: Same as main flow
- Step 7: Firebase Auth detects duplicate email
- Step 8: System displays "An account with this email already exists"
- Step 9: User can click "Login instead" link
- Flow ends

Alternative Flow 2: Weak Password

- Steps 1-4: Same as main flow
- Step 5: Password validator returns "weak" score
- Step 6: System shows real-time strength indicator
- Step 7: User sees feedback: "Add uppercase letter and number"
- User improves password
- Resume main flow at step 5

Alternative Flow 3: Invalid Input

- Steps 1-3: Same as main flow
- Step 4: Validation detects errors (invalid email format, missing name)
- Step 5: System displays inline validation errors
- User corrects errors
- Resume main flow at step 4

Alternative Flow 4: Firestore Creation Failed

- Steps 1-8: Same as main flow
- Step 9: Firestore document creation fails (network error, permission issue)
- Step 10: System initiates rollback procedure
- Step 11: System deletes Firebase Auth account
- Step 12: System displays "Registration failed. Please try again."
- Flow ends

Alternative Flow 5: User Skips Onboarding

- Steps 1-11: Same as main flow
- Step 12: User clicks "Skip" on onboarding wizard
- Step 13: System keeps `onboardingCompleted: false`
- Step 14: Dashboard shows with reminder to complete profile
- Flow ends

PRECONDITIONS:

- User is not currently authenticated
- User is on registration page
- User has valid email address
- User can create strong password
- Internet connection available
- Firebase Auth and Firestore accessible

POSTCONDITIONS:

Success:

- Firebase Auth account created with UID
- Firestore user document created in `/users/{uid}`
- User is authenticated with session token
- User state stored in application
- User redirected to `/dashboard`
- Onboarding wizard displayed
- Welcome email sent (if email notifications enabled)

Failure (Email Exists):

- No account created
- User remains on registration page
- Helpful error message displayed
- Link to login page provided

Failure (Firestore Error):

- Firebase Auth account deleted (rollback)
- No user document created
- User remains on registration page
- Error logged for debugging

- Generic error message displayed

SECURITY CONSIDERATIONS:

1. **Password Security:**
    - Minimum 8 characters enforced
    - Complexity requirements (uppercase, lowercase, number)
    - Client-side strength validation
    - Server-side bcrypt hashing by Firebase
2. **Email Validation:**
    - RFC 5322 format validation
    - Uniqueness check by Firebase Auth
    - Email verification sent (optional enhancement)
3. **Data Integrity:**
    - Atomic rollback if Firestore fails
    - Prevents orphaned auth accounts
    - Transaction-like behavior
4. **Privacy:**
    - Default `allowContactSync: false`
    - Default notification preferences set
    - No sensitive data stored in plaintext
    - GDPR-compliant defaults

VALIDATION RULES:

Name:

- Required: Yes
- Min length: 1 character
- Max length: 50 characters
- Pattern: Letters, spaces, hyphens, apostrophes

Email:

- Required: Yes
- Format: Valid RFC 5322 email
- Unique: Must not exist in system
- Example: user@example.com

Password:

- Required: Yes
- Min length: 8 characters
- Must contain: 1 uppercase letter
- Must contain: 1 lowercase letter
- Must contain: 1 number
- Optional: 1 special character (!@#$%^&*)

Terms & Conditions:

- Required: Yes
- Must be explicitly accepted (checkbox)

5.5.3 UC-27/UC-28: Create Group with Quick Add

The Create Group with Quick Add sequence diagram illustrates 5 key phases:

Phase 1: Initiate Group Creation

- User opens CreateGroupModal
- Enters group details (name, icon, currency)
- Clicks "Quick Add from Previous Groups"

Phase 2: Load User Connections

- System syncs connections via syncMyConnectionsFromGroups()
- Queries Firestore for all user's groups

- Creates bidirectional connection documents
- Returns sorted list of previous collaborators

Phase 3: Select Members

- User searches/filters connections
- Selects desired members via checkboxes
- System stores as pending invitations

Phase 4: Create Group & Send Invitations

- Validates form and creates group in Firestore
- Updates connection graph for all member pairs
- Sends bulk invitations via `sendBulkInvitations()`
- Handles success/failure gracefully

Phase 5: Receive Notifications (Async)

- Real-time notifications delivered to invited users
- Users accept/decline invitations
- Group membership and connections auto-update

Create Group with Quick Add Members - Sequence Diagram

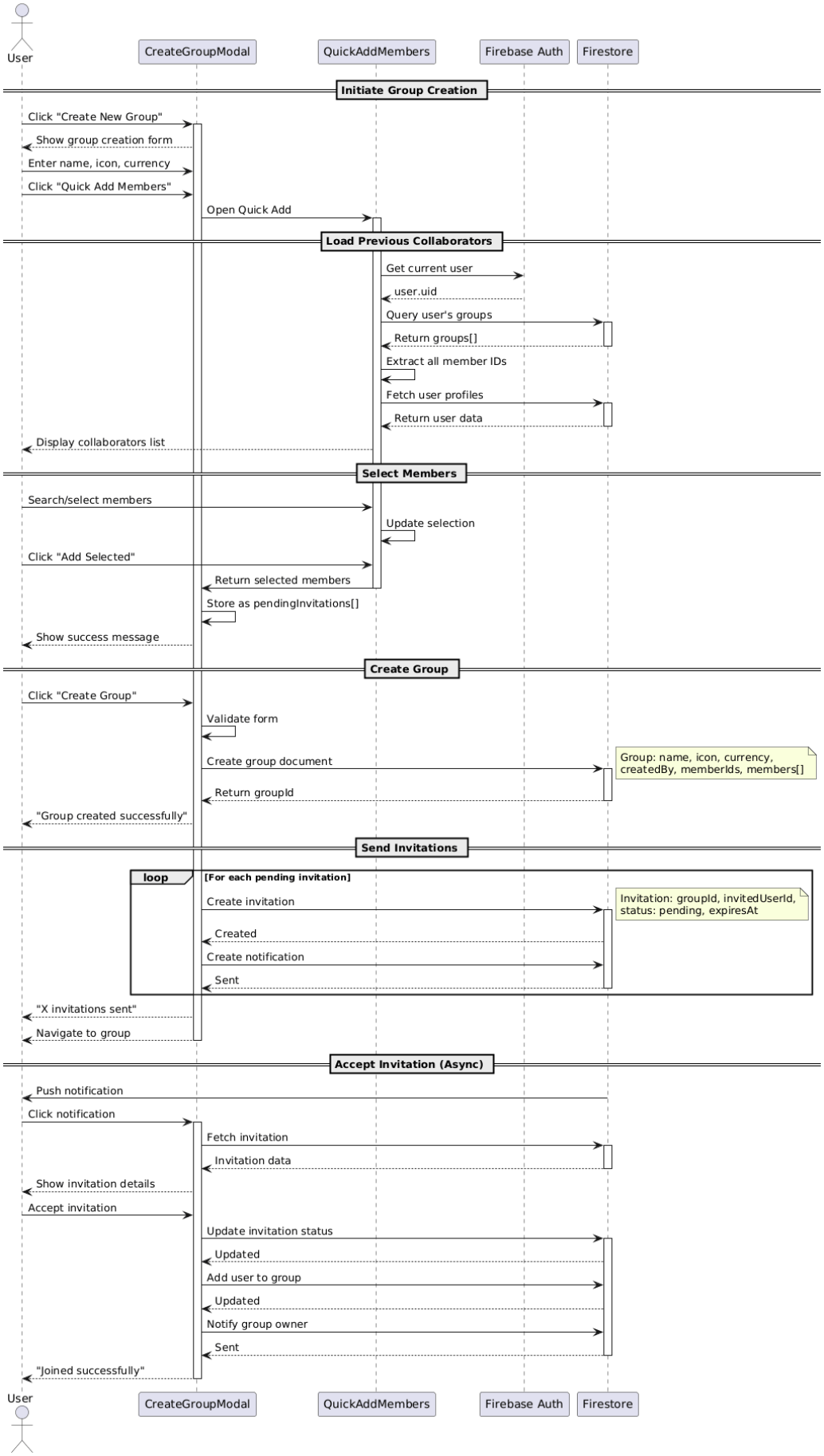


Figure 7: Create Group With Quick Add Sequence Diagram

Group creation with Quick Add proceeds through five coordinated phases: the user initiates the group and selects previous collaborators from their connection graph; the system dispatches bulk invitations; and real-time listeners deliver notifications to invited members who may then accept or decline. Bidirectional connection documents are created or updated for all member pairs to support future Quick Add operations.

5.5.4 UC-46: Accept Invitation Sequence Diagram

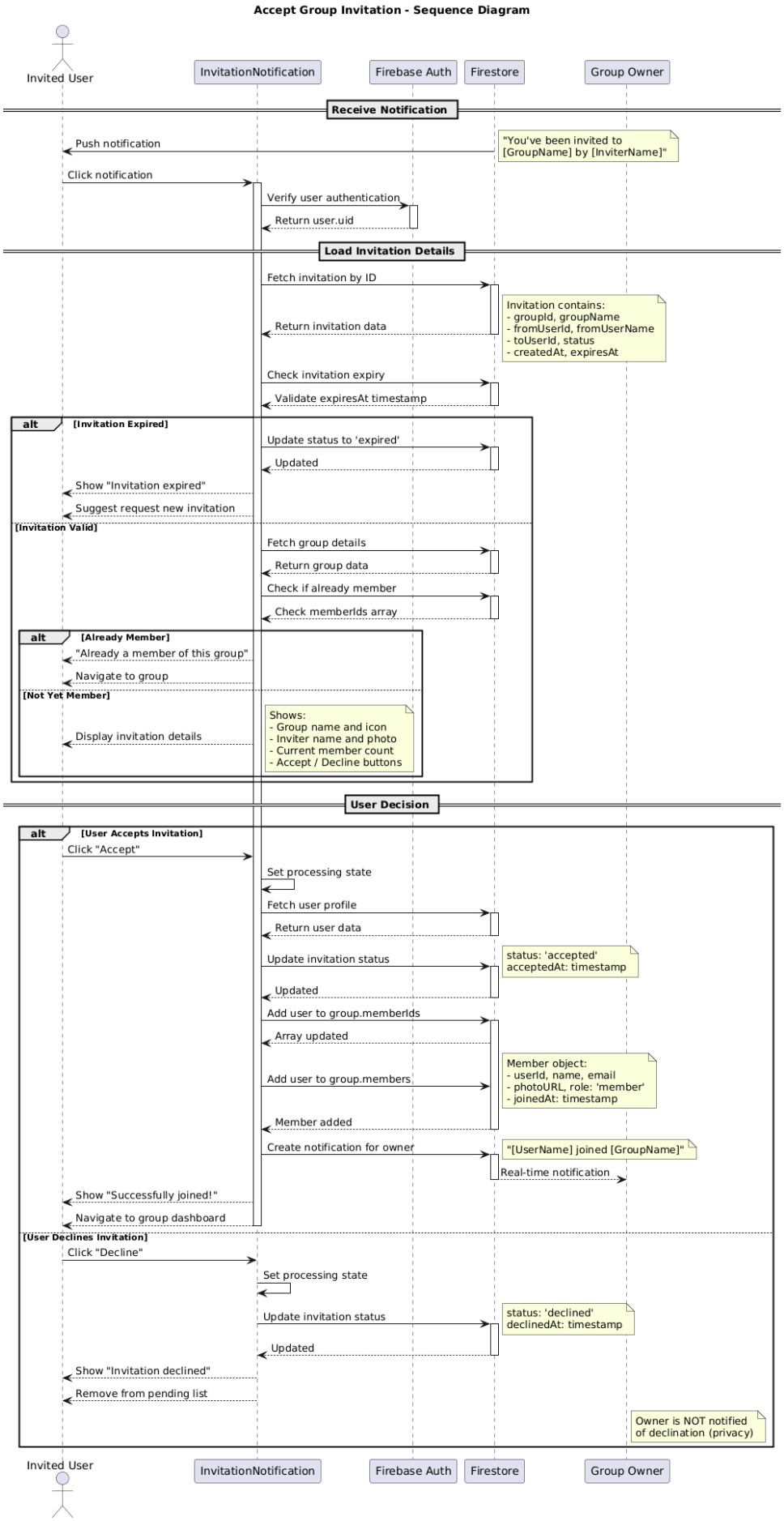

Accept Group Invitation - Sequence Diagram
Invited User
InvitationNotification
Firebase Auth
Firestore
Group Owner
Receive Notification
Push notification
"You've been invited to [GroupName] by [InviterName]"
Click notification
Verify user authentication
Return user.uid
Load Invitation Details
Fetch invitation by ID
Return invitation data
Invitation contains:
- groupId, groupName
- fromUserId, fromUserName
- toUserId, status
- createdAt, expiresAt
Check invitation expiry
Validate expiresAt timestamp
alt [Invitation Expired]
Update status to 'expired'
Updated
Show "Invitation expired"
Suggest request new invitation
[Invitation Valid]
Fetch group details
Return group data
Check if already member
Check memberIds array
alt [Already Member]
"Already a member of this group"
Navigate to group
[Not Yet Member]
Shows:
- Group name and icon
- Inviter name and photo
- Current member count
- Accept / Decline buttons
Display invitation details
User Decision
alt [User Accepts Invitation]
Click "Accept"
Set processing state
Fetch user profile
Return user data
Update invitation status
status: 'accepted'
acceptedAt: timestamp
Updated
Add user to group.memberIds
Array updated
Member object:
- userId, name, email
- photoURL, role: 'member'
- joinedAt: timestamp
Add user to group.members
Member added
Create notification for owner
"[UserName] joined [GroupName]"
Real-time notification
Show "Successfully joined!"
Navigate to group dashboard
[User Declines Invitation]
Click "Decline"
Set processing state
Update invitation status
status: 'declined'
declinedAt: timestamp
Updated
Show "Invitation declined"
Remove from pending list
Owner is NOT notified of declination (privacy)

Figure 8: Accept Group Invitation Sequence Diagram

Main Success Scenario

Primary Actor: Invited User

Stakeholders:

- Invited User (prospective member)
- Group Owner (invitation sender)
- Existing Group Members

Preconditions:

- User must have a valid invitation (not expired, not previously accepted/declined)
- User must be authenticated or able to authenticate
- Group must still exist in the system
- User must not already be a member of the group

Success Guarantee:

- Invitation status is updated (accepted or declined)
- If accepted: User is added to group with 'member' role
- If accepted: Group owner receives notification of new member
- If accepted: User gains access to group dashboard and all group features
- If declined: Invitation is marked as declined without group access

Main Success Scenario:

1. User receives push notification about group invitation
2. User clicks notification to view invitation details
3. System verifies user authentication
4. System fetches invitation document from Firestore
5. System validates invitation (not expired, not already processed)
6. System fetches group details
7. System checks if user is already a member

8. System displays invitation modal with group information
9. User reviews invitation details (group name, inviter, member count)
10. User clicks "Accept Invitation" button
11. System sets processing state to prevent duplicate submissions
12. System fetches user profile data
13. System updates invitation status to 'accepted' with timestamp
14. System adds user ID to group.memberIds array
15. System adds user object to group.members array
16. System creates notification for group owner
17. System displays success message
18. System navigates user to group dashboard
19. Group owner receives real-time notification

5.5.5 UC54-UC62 AddExpense ReceiptScan Sequence

The sequence demonstrates automated expense creation through receipt scanning and OCR processing. Users initiate expense creation and optionally capture or upload a receipt image, which is stored on Cloudinary CDN for permanent hosting. The system then employs Tesseract.js to perform client-side OCR, extracting text from the receipt and applying pattern matching algorithms to identify merchant name, transaction amount, date, and line items. Extracted data auto-fills the expense form with confidence indicators showing recognition reliability. Users validate the auto-filled data, correct any OCR errors (typical accuracy: 85-90%), configure split method and participating members, then submit the expense. The system executes a Firestore transaction atomically creating the expense document, updating group totals, recalculating balances, logging activity, and triggering real-time notifications to group members. The implementation uses client-side Tesseract.js OCR rather than cloud services to eliminate API costs, maintain privacy, reduce latency, and enable offline capability, accepting lower accuracy (85-90% vs 95-98% cloud) and longer processing time (2-5 seconds) as acceptable trade-offs. The OCR assistance reduces average data entry time from 45-60 seconds to 10-15 seconds while maintaining accuracy through user validation.

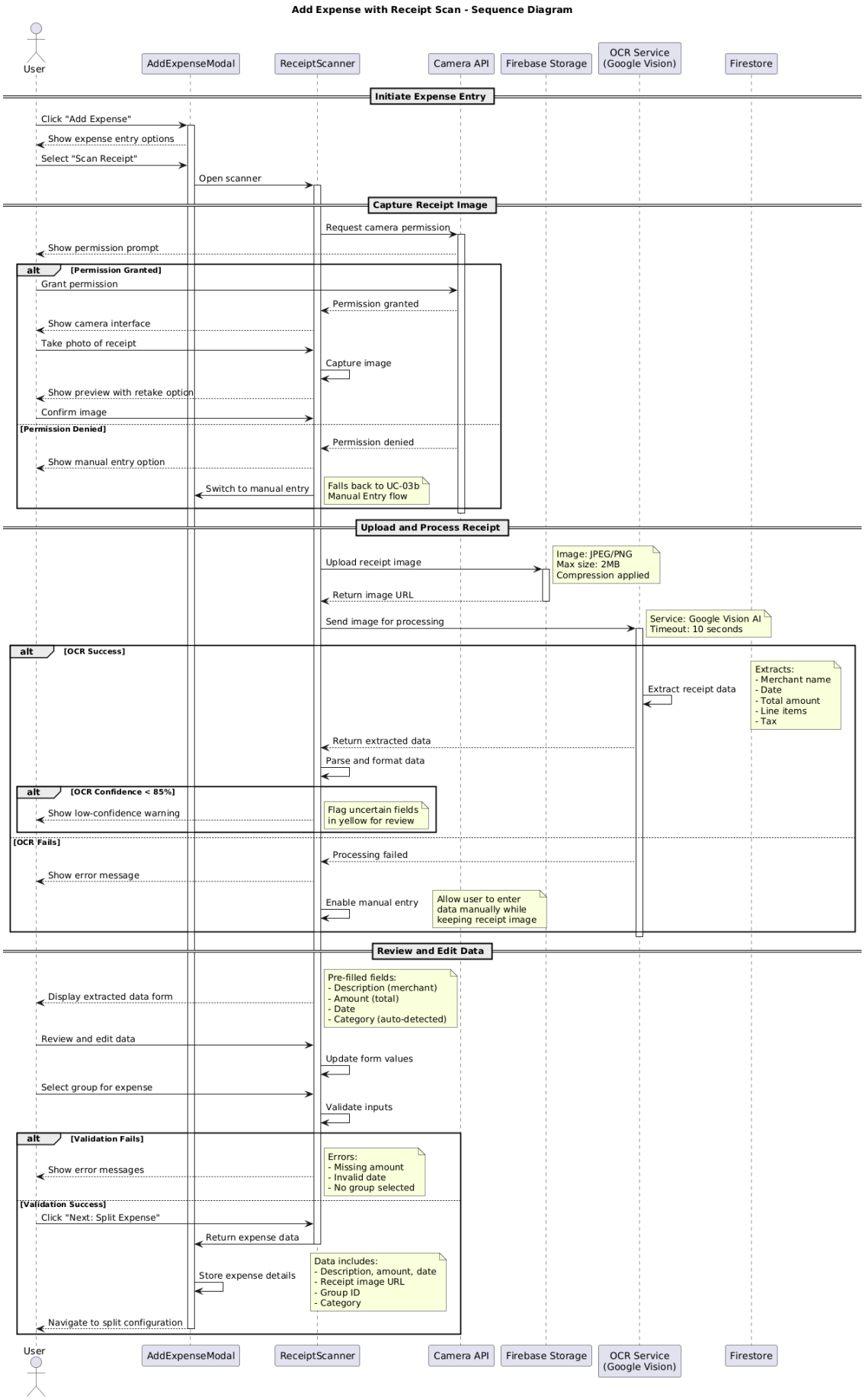

Add Expense with Receipt Scan - Sequence Diagram
User
AddExpenseModal
ReceiptScanner
Camera API
Firebase Storage
OCR Service (Google Vision)
Firestore
Initiate Expense Entry
Click "Add Expense"
Show expense entry options
Select "Scan Receipt"
Open scanner
Capture Receipt Image
Request camera permission
Show permission prompt
alt [Permission Granted]
Grant permission
Permission granted
Show camera interface
Take photo of receipt
Capture image
Show preview with retake option
Confirm image
[Permission Denied]
Permission denied
Show manual entry option
Switch to manual entry
Falls back to UC-03b Manual Entry flow
Upload and Process Receipt
Upload receipt image
Image: JPEG/PNG Max size: 2MB Compression applied
Return image URL
Send image for processing
Service: Google Vision AI Timeout: 10 seconds
alt [OCR Success]
Extract receipt data
Extracts: - Merchant name - Date - Total amount - Line items - Tax
Return extracted data
Parse and format data
alt [OCR Confidence < 85%]
Show low-confidence warning
Flag uncertain fields in yellow for review
[OCR Fails]
Processing failed
Show error message
Enable manual entry
Allow user to enter data manually while keeping receipt image
Review and Edit Data
Pre-filled fields: - Description (merchant) - Amount (total) - Date - Category (auto-detected)
Display extracted data form
Review and edit data
Update form values
Select group for expense
Validate inputs
alt [Validation Fails]
Errors: - Missing amount - Invalid date - No group selected
Show error messages
[Validation Success]
Click "Next: Split Expense"
Return expense data
Store expense details
Data includes: - Description, amount, date - Receipt image URL - Group ID - Category
Navigate to split configuration

Figure 9: Add Expense with Receipt Scanning Sequence Diagram

The diagram shows 4 main phases:

Phase 1: Initiate Expense Entry

- User clicks "Add Expense"
- Selects "Scan Receipt" option
- Opens scanner interface

Phase 2: Capture Receipt Image

- Request camera permission
- Grant/deny handling
- Take photo → Preview → Confirm
- Fallback to manual entry if denied

Phase 3: Upload and Process Receipt

- Upload image to Firebase Storage
- Send to Google Vision AI OCR
- Extract data (merchant, date, amount, items, tax)
- Handle OCR success/failure paths
- Flag low-confidence fields (<85%)

Phase 4: Review and Edit Data

- Display pre-filled form with extracted data
- User reviews and corrects errors
- Select group for expense
- Validate inputs
- Navigate to split configuration

5.5.6 UC-42: Find Contacts Sequence Diagram

The contact synchronization sequence implements a privacy-first architecture through eight distinct steps ensuring zero plaintext phone number storage: (1) User initiates contact discovery from CreateGroupModal displaying an explicit privacy notice outlining data handling practices, (2) Upon user consent, the Browser Contacts API presents the operating system's native contact

picker ensuring browser-mediated access control, (3) User selects desired contacts (typically 3-10), and the browser returns contact objects containing names and phone numbers, (4) The system normalizes phone numbers to E.164 international format using libphonenumber-js to ensure consistent formatting across regional variations, (5) Each normalized phone number undergoes one-way SHA-256 cryptographic hashing via the Web Crypto API, producing 64-character hexadecimal digests stored temporarily in browser memory, (6) Hashed values are batched into groups of 10 (Firestore 'IN' query limitation) and queried against the /phoneNumbers collection which stores only hashes and associated user IDs, (7) Upon receiving matched user records, the system immediately purges all phone numbers, normalized values, and hashes from memory with explicit null assignments to facilitate garbage collection, and (8) Results display categorizes contacts into "On Split-Sync" (matched users with selectable checkboxes) and "Not on App" (non-users with SMS invitation options). The architecture guarantees that plaintext phone numbers never traverse the network, never persist in Firestore, and exist only transiently in browser memory during the matching operation. SHA-256's cryptographic properties (collision resistance, pre-image resistance, deterministic output) enable reliable matching while preventing hash reversal, ensuring compliance with GDPR Article 32 (security of processing) and Article 25 (data protection by design).

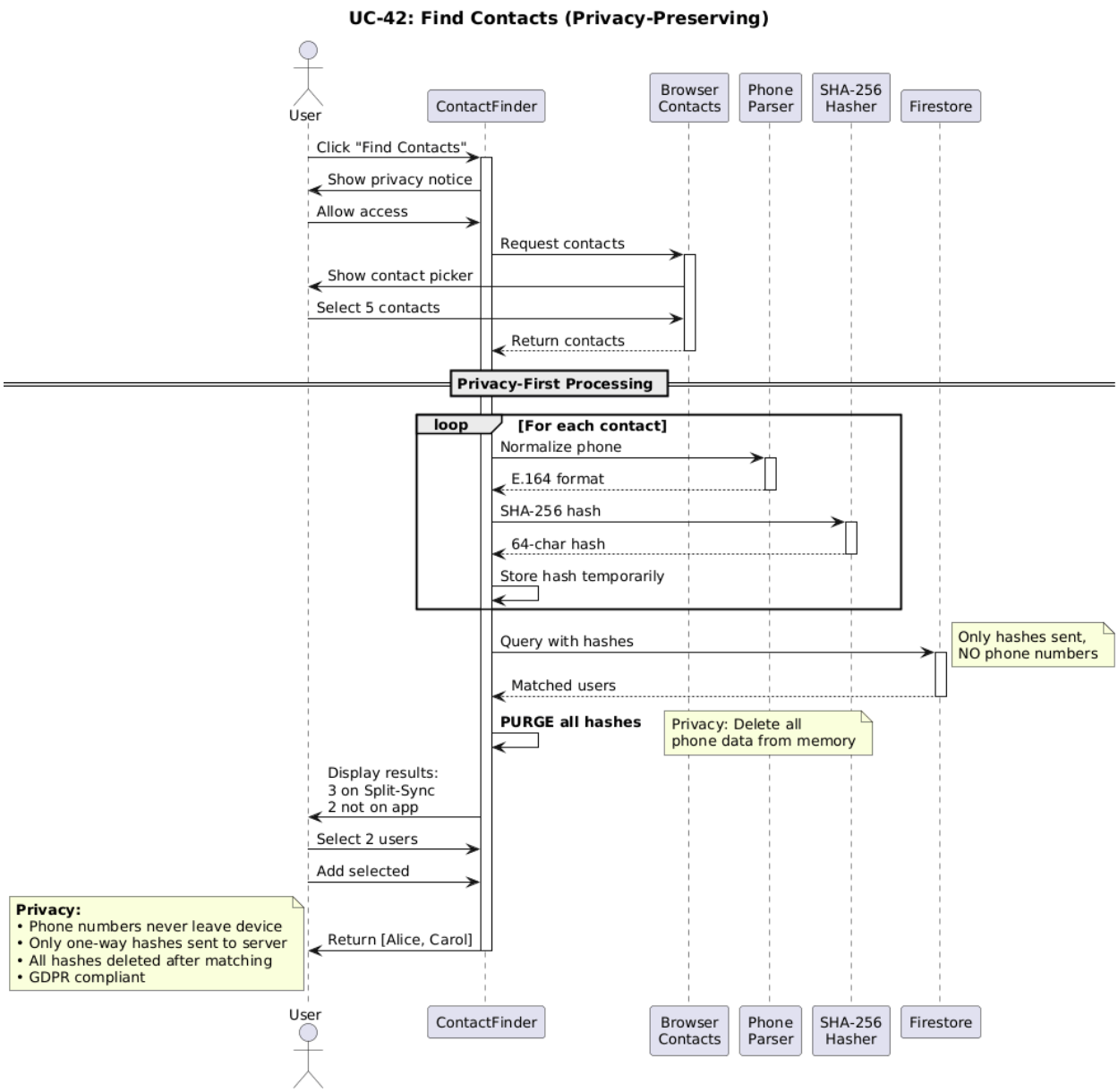


Figure 10: Find contact Sequence Diagram

Main Success Scenario:

1. User on CreateGroupModal clicks "Find Contacts on Split-Sync"
2. ContactFinder modal opens
3. System displays privacy notice explaining:
    - Phone numbers never stored
    - Secure SHA-256 hashing used
    - Data deleted after matching

- User controls access

4. User reads notice and clicks "Allow Contacts Access"
5. System checks if Browser Contacts API available (`navigator.contacts`)
6. Browser displays native contact picker (iOS/Android/Desktop)
7. User selects 5 contacts from phone book
8. User clicks "Done" in contact picker
9. Browser returns array of contact objects to application
10. System displays "Matching contacts... " loading indicator
11. For first contact "Alice Cooper" with phone "087-123-4567":
    - Parse phone number with country code inference
    - Normalize to E.164: "+353871234567"
    - Encode to UTF-8 byte array
    - Compute SHA-256 hash: "a3f5b8c2d9e1f4a7b6c3..."
    - Store hash in temporary array
12. Repeat step 11 for remaining 4 contacts (7 total phone numbers)
13. Split 7 hashes into batches of 10 (1 batch needed)
14. Query Firestore: `WHERE hashedPhone IN [hash1, hash2, ...]`
15. Firestore returns 3 matched user records: - Alice Cooper (alice123)
    - Bob Smith (bob456)
    - Carol White (carol789)
16. System matches hashes back to original contacts
17. CRITICAL: System purges all sensitive data:
    - `hashedPhones = null`
    - `normalizedPhones = null`
    - `contactHashMap = null`
    - Force JavaScript garbage collection
18. System categorizes results:
    - On Split-Sync (3): Alice, Bob, Carol
    - Not on App (2): David, Emma
19. Display results with checkboxes for matched users
20. User selects Alice ✓ and Carol ✓

21. User clicks "Add Selected (2)"
22. ContactFinder returns `[{userId: alice123, ...}, {userId: carol789, ...}]`
23. CreateGroupModal adds to pending invitations
24. ContactFinder closes
25. User sees "2 friends will be invited" in CreateGroupModal

Alternative Flow 1: Permission Denied

- Steps 1-6: Same as main flow
- Step 7: User clicks "Cancel" or "Don't Allow"
- Step 8: Browser returns permission error
- Step 9: System displays "Contact access denied. You can enable it later in settings."
- Flow ends, user returns to CreateGroupModal

Alternative Flow 2: No Matches Found

- Steps 1-15: Same as main flow
- Step 16: Firestore returns 0 matches (all contacts not on Split-Sync)
- Steps 17-18: Same as main flow (purge data, categorize) -
- Step 19: Display results showing:
  - On Split-Sync (0): Empty state
  - Not on App (5): All contacts
  - Step 20: User can invite via SMS or close
  - Flow ends

Alternative Flow 3: Browser API Not Supported

- Steps 1-5: Same as main flow
- Step 6: System detects `!('contacts' in navigator)`
- Step 7: System displays "Contact sync requires Chrome or Edge browser. Current browser: Firefox."
- Flow ends, user returns to CreateGroupModal

Alternative Flow 4: Invalid Phone Numbers

- Steps 1-11: Same as main flow
- Step 12: Parser encounters malformed number "123" (invalid)

- Step 13: Parser returns `null` - Step 14: System skips this number, continues with valid numbers
- Step 15: System processes remaining valid numbers
- Resume main flow at step 13

Alternative Flow 5: All Contacts Already in Group

- Steps 1-16: Same as main flow
- Step 17: System detects all matched users already in pending invitations
- Step 18: System displays "All matched contacts are already invited"
- Step 19: Offer to show unmatched contacts for SMS invites
- Flow ends

PRECONDITIONS:

- User is authenticated
- User is in CreateGroupModal
- Browser supports Contact Picker API (Chrome/Edge)
- User has contacts in phone with phone numbers
- At least some contacts use Split-Sync (optional, not required)
- User has granted browser permission to access contacts (or will grant)
- Internet connection available for Firestore query

POSTCONDITIONS:

Success:

- User contacts matched against Split-Sync users
- Matched users displayed with selection interface
- Unmatched users shown with SMS invite option
- All phone numbers and hashes purged from memory
- Selected users returned as user objects (no phone data)
- Zero phone number data retained anywhere
- User can proceed with group creation

Failure (Permission Denied):

- No contact access granted
- No matching performed
- Helpful error message displayed

- User can retry or use alternative methods (Quick Add, manual entry)

Failure (API Unsupported):

- Browser compatibility error shown
- Suggestion to use Chrome/Edge provided
- User can use alternative invite methods

5.5.7 UC-78: Settle via PayPal Sequence Diagram

The PayPal settlement sequence implements a twelve-step payment flow bridging expense tracking with actual fund transfer through external payment gateway integration: (1) User navigates to group details page where the balance calculator executes the optimization algorithm computing net balances and suggesting settlements sorted by debt magnitude to minimize transaction count, (2) User initiates settlement by clicking "Settle Up" which opens the SettleUpModal displaying payment suggestions with the largest debt prioritized first for efficient settlement, (3) User selects a specific debt to settle (e.g., "Pay Alice €45.50") and chooses PayPal as the preferred payment method from available options (PayPal, Revolut, SEPA, Cash), (4) System queries Firestore to retrieve creditor's PayPal payment information stored in their user profile document, returning their PayPal.me identifier or email address with verification status, (5) System validates the payment amount against PayPal's transaction limits enforcing minimum threshold (€0.01) and maximum ceiling (€10,000.00 per transaction), rejecting invalid amounts with appropriate error messages and suggesting alternatives (split into multiple payments or use SEPA for larger amounts), (6) User reviews comprehensive payment summary displaying recipient name, amount, payment method, destination email, and disclaimer about external redirection, (7) System generates a PayPal.me URL following the structure `https://paypal.me/{username}/{amount}{currency}` with proper URL encoding and optional query parameters for locale and payment notes, (8) System redirects user to PayPal's platform in a new browser tab using `window.open()` where user authenticates with PayPal credentials and reviews pre-filled payment details including recipient and amount, (9) PayPal processes the transaction upon user confirmation, debiting debtor's account, crediting creditor's account, generating unique transaction identifier, and sending confirmation emails to both parties documenting the fund transfer, (10) User returns to Split-Sync interface and manually confirms payment completion through a trust-based confirmation dialog that explicitly

asks "Did you complete the payment?" to prevent accidental or fraudulent marking, (11) Upon user confirmation, system executes a Firestore transaction atomically updating expense splits (marking paid with timestamp, payment method, and reference), recalculating group financial totals (incrementing totalSettled counter), updating balance sheet for both debtor and creditor (recalculating net positions), and logging settlement activity with comprehensive metadata for audit trail and dispute resolution, and (12) Creditor receives real-time notification via Firestore listeners immediately updating their balance view and displaying payment notification across all active devices. The implementation employs PayPal.me links rather than direct API integration to circumvent API key management complexity, eliminate transaction fees, and maintain zero-cost operation while accepting the trade-off of manual payment confirmation versus automatic verification through webhook callbacks. The trust-based confirmation model relies on multiple safeguards including social accountability within small trusted groups, external payment confirmation emails from PayPal providing independent verification, comprehensive activity logging creating immutable audit trails, and the reversibility of erroneous markings through administrative intervention. This architectural decision prioritizes deployment simplicity, operational cost-efficiency, user privacy (Split-Sync never handles payment credentials), and flexibility (supports any payment method including cash) over technical verification mechanisms, making it appropriate for the application's target demographic of friends, family, and small team expense sharing scenarios where social trust dynamics and external payment platform confirmations provide sufficient fraud deterrence while maintaining seamless user experience and regulatory compliance without PCI DSS obligations.

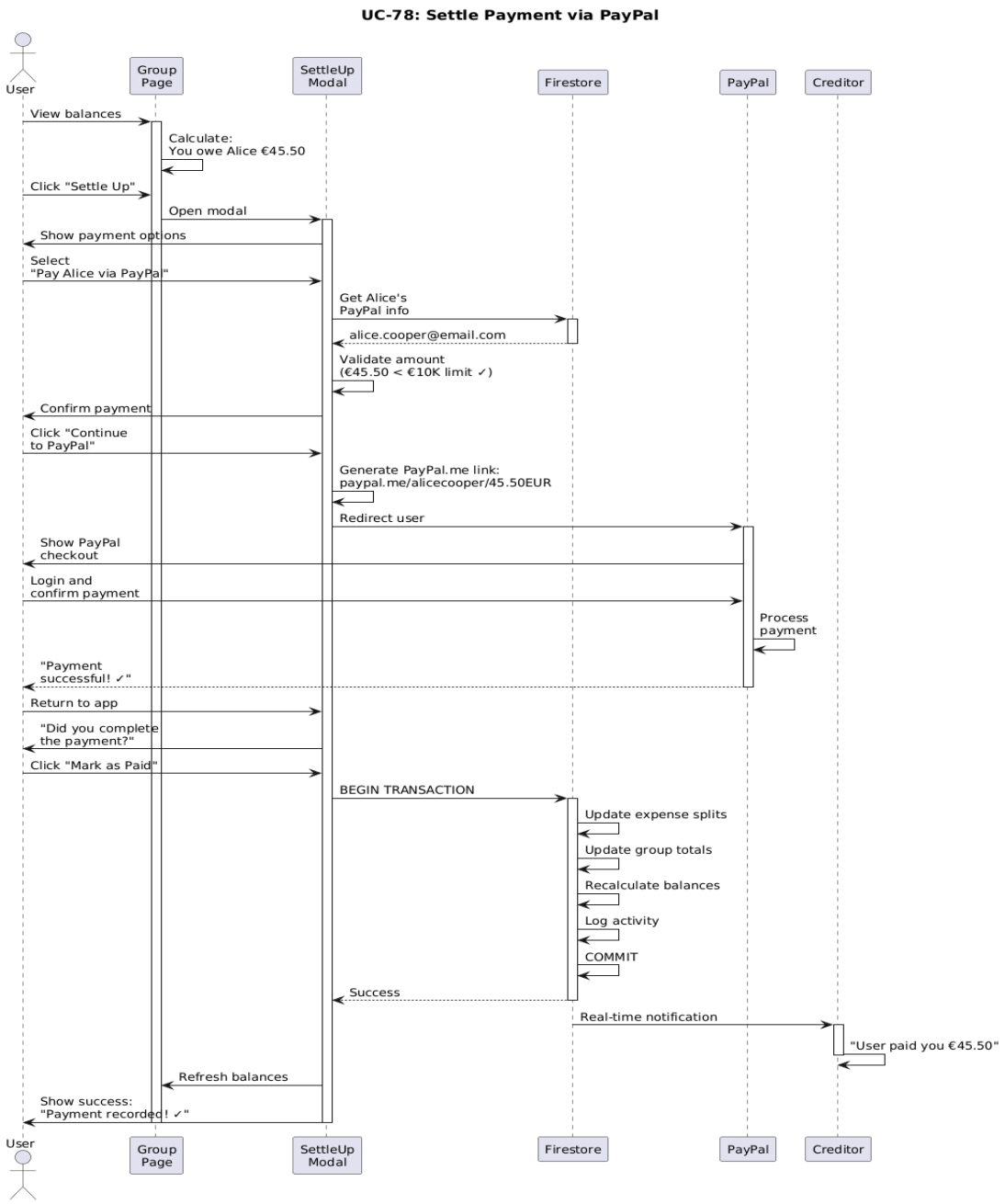


Figure 11: Settle Payment Via PayPal Sequence Diagram

Main Success Scenario:

1. User navigates to group "Weekend Trip"
2. Balance calculator runs algorithm showing: You owe Alice €45.50, You owe Bob €23.00
3. User clicks "Settle Up" button
4. SettleUpModal opens displaying settlement suggestions

5. System suggests "Pay Alice €45.50" (largest debt first)
6. User selects "Pay Alice €45.50" checkbox
7. System displays payment method options (PayPal, Revolut, SEPA, Cash)
8. User selects "PayPal" radio button
9. System queries `/users/alice123` for payment methods
10. Firestore returns: `{type: "paypal", identifier: "alice.cooper@email.com", isDefault: true}`
11. System validates amount: €45.50 is within €0.01-€10,000 range ✓
12. System displays confirmation screen showing:
    - Paying: Alice Cooper
    - Amount: €45.50
    - Method: PayPal
    - To: alice.cooper@email.com
13. User clicks "Continue to PayPal"
14. System generates PayPal.me URL: `https://paypal.me/alicecooper/45.50EUR`
15. System adds optional note parameter: `?note=Weekend%20Trip%20Settlement`
16. System opens PayPal URL in new browser tab using `window.open()`
17. PayPal displays checkout page with pre-filled information
18. User enters PayPal credentials (email/password)
19. PayPal authenticates user credentials
20. PayPal displays payment confirmation page showing:
    - Recipient: Alice Cooper
    - Amount: €45.50
    - Payment source: User's PayPal balance
21. User clicks "Confirm and Pay"
22. PayPal debits user's account: -€45.50
23. PayPal credits Alice's account: +€45.50
24. PayPal generates transaction ID: "8FK39DK2J"
25. PayPal sends confirmation email to user
26. PayPal sends payment notification email to Alice
27. PayPal displays success page: "Payment complete! ✓"
28. User closes PayPal tab and returns to Split-Sync

29. SettleUpModal displays confirmation dialog: "Did you complete the payment?"
30. User clicks "Mark as Paid"
31. System begins Firestore transaction
32. System queries all expenses where user owes Alice:
    - Expense 1: €30.00 owed
    - Expense 2: €15.50 owed
33. System marks Expense 1 split as paid:
    - `splits[i].paid = true`
    - `splits[i].paidAt = serverTimestamp()`
    - `splits[i].paymentMethod = "paypal"`
34. System marks Expense 2 split as paid (same fields)
35. System updates group totals: `totalSettled += 45.50`
36. System recalculates `/groups/weekend-trip/balances/user123`:
    - `totalOwed -= 45.50`
    - `netBalance` recalculated
37. System recalculates `/groups/weekend-trip/balances/alice123`:
    - `totalOwedTo -= 45.50`
    - `netBalance` recalculated
38. System creates activity log:
    ```
    {
      type: "payment_made",
      userId: "user123",
      userName: "User Name",
      description: "User paid Alice €45.50 via PayPal",
      metadata: {amount: 45.50, currency: "EUR", method: "paypal", paidTo: "alice123"},
      timestamp: serverTimestamp()
    }
    ```
39. System commits Firestore transaction
40. All updates succeed atomically
41. Firestore triggers `onSnapshot` listener for Alice
42. Alice's app receives real-time update

43. Alice sees notification: "User paid you €45.50 via PayPal"
44. Alice's balance view updates automatically
45. SettleUpModal closes
46. GroupDetails page re-renders
47. Balance calculator re-runs
48. New balances displayed:
    - You owe Alice: €0.00 ✓
    - You owe Bob: €23.00
    - Net balance: -€23.00
49. Success toast displays: "Payment recorded! ✓ You paid Alice €45.50"
50. Process complete

Alternative Flow 1: Alice Has No PayPal

- Steps 1-9: Same as main flow
- Step 10: Firestore returns empty paymentMethods array for PayPal
- Step 11: System displays error: "Alice hasn't added PayPal account yet"
- Step 12: System suggests alternatives: "Try Revolut or SEPA instead"
- User can select different payment method or cancel

Alternative Flow 2: Amount Exceeds PayPal Limit

- Steps 1-11: Same as main flow
- Step 12: Amount is €12,500 (exceeds €10,000 limit)
- Step 13: System displays error: "Amount exceeds PayPal maximum (€10,000)"
- Step 14: System suggests: "Split into multiple payments or use SEPA"
- User must either:
    - Pay partially (€10,000 now, €2,500 later)
    - Switch to SEPA (limit €999,999)

Alternative Flow 3: User Cancels on PayPal

- Steps 1-17: Same as main flow
- Step 18: User sees PayPal checkout

- Step 19: User clicks "Cancel" instead of logging in
- Step 20: PayPal redirects back or user closes tab
- Step 21: User returns to SettleUpModal
- Step 22: Dialog asks: "Did you complete the payment?"
- Step 23: User clicks "Not Yet"
- Step 24: Modal stays open, user can retry or cancel
- No database changes made

Alternative Flow 4: PayPal Payment Fails

- Steps 1-21: Same as main flow
- Step 22: User clicks "Confirm and Pay"
- Step 23: PayPal processes payment
- Step 24: PayPal detects insufficient funds
- Step 25: PayPal displays error: "Payment failed. Insufficient funds in account."
- Step 26: User must add funds or use different payment source
- Step 27: User gives up and returns to Split-Sync
- Step 28: User clicks "Not Yet" in confirmation dialog
- No database changes made

Alternative Flow 5: Transaction Rollback

- Steps 1-32: Same as main flow
- Step 33: First expense split update succeeds
- Step 34: Second expense split update fails (network error)
- Step 35: Firestore transaction detects failure
- Step 36: Firestore automatically rolls back ALL changes
- Step 37: First expense split reverts to unpaid
- Step 38: No group totals updated
- Step 39: No balances changed

- Step 40: No activity logged

- Step 41: Modal displays error: "Failed to record payment. Please try again."

- User can retry

PRECONDITIONS:

- User is authenticated
- User is member of a group
- User has outstanding debt to at least one member
- Creditor (Alice) has added PayPal payment method to profile
- Creditor's PayPal account is valid and active
- User has PayPal account (or will create one)
- Internet connection available
- Amount owed is within PayPal limits (€0.01 - €10,000)

POSTCONDITIONS:

Success:

- User has transferred €45.50 to Alice via PayPal
- PayPal has processed the transaction
- Both parties received confirmation emails
- Expense splits marked as paid in Firestore
- Group `totalSettled` increased by €45.50
- User's balance reduced by €45.50
- Alice's balance increased by €45.50
- Activity log created documenting payment
- Alice notified in real-time
- Group balances recalculated
- UI updated to show new balances

Failure (Payment Cancelled):

- No PayPal transaction occurred
- No money transferred
- No database changes made

- User remains on SettleUpModal
- Balances unchanged
- User can retry or use different method

Failure (Transaction Rollback):

- PayPal payment may have succeeded
- But Firestore update failed
- All database changes rolled back
- Split-Sync shows old balances
- Discrepancy: PayPal shows payment, Split-Sync doesn't
- User must contact Alice to resolve manually

5.5.8 UC-74: View Balances Sequence Diagram

The balance calculation sequence implements a six-phase algorithm ensuring accurate financial tracking: (1) User requests balance view triggering cache validation (30-second TTL), (2) If cache is stale, system fetches all unpaid expense splits from Firestore ordered chronologically, (3) Algorithm initializes an n×n balance matrix where balance[i][j] represents the amount person i owes person j, with all cells initialized to zero, (4) For each expense, the system iterates through splits updating matrix cells: for each unpaid split, balance[debtor][payer] is incremented by split amount, accumulating all debts across expenses, (5) Simplification phase eliminates reciprocal debts by iterating through each unique pair (i,j) and reducing mutual obligations: if balance[i][j] > 0 AND balance[j][i] > 0, both values are reduced by min(balance[i][j], balance[j][i]), converting bidirectional debts into unidirectional net amounts (e.g., Alice owes Bob €30 and Bob owes Alice €60 simplifies to Bob owes Alice €30), (6) Net balance calculation computes each member's financial position: netBalance[user] = Σ(balance[j][user]) - Σ(balance[user][j]), where positive values indicate creditor status (owed money) and negative values indicate debtor status (owes money), (7) Validation ensures mathematical consistency by verifying Σ(all net balances) = 0, detecting calculation errors if sum is non-zero, (8) Settlement optimizer employs a greedy algorithm separating members into creditors (positive balance) and debtors (negative balance), sorting both arrays by absolute value descending, then repeatedly matching largest creditor with largest debtor, creating settlement suggestions of min(creditor amount, |debtor amount|) until all

balances reach zero, minimizing total transaction count (e.g., reducing 3 separate payments to 1 optimized transaction), and (9) Results are cached in Firestore's balances subcollection with 30-second TTL and displayed to user showing individual debts, net position, and suggested settlements. The algorithm exhibits O(e×s + n²) time complexity where e represents expense count, s represents average splits per expense, and n represents member count, typically executing in approximately 150ms for groups with 50 expenses and 10 members.

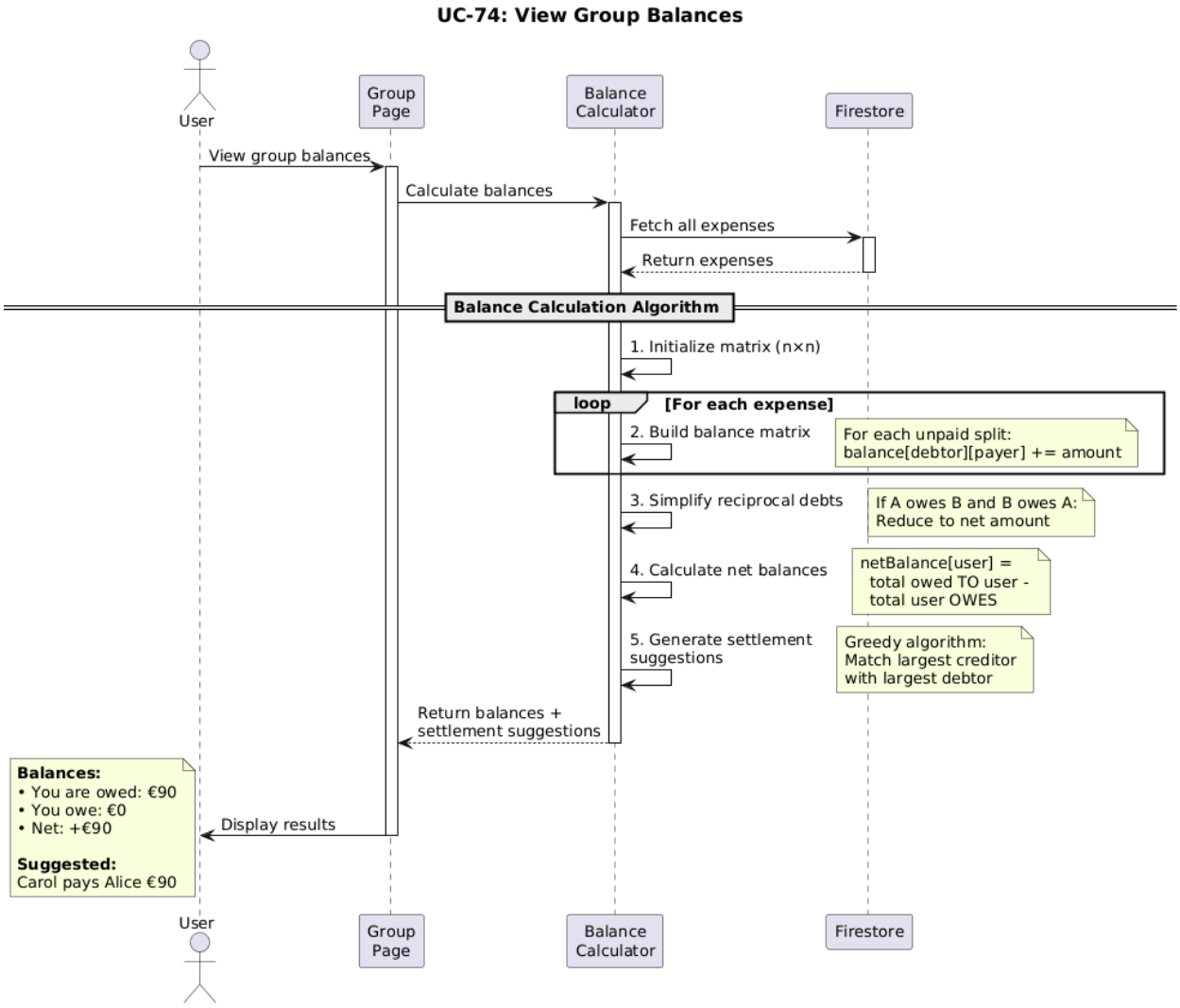
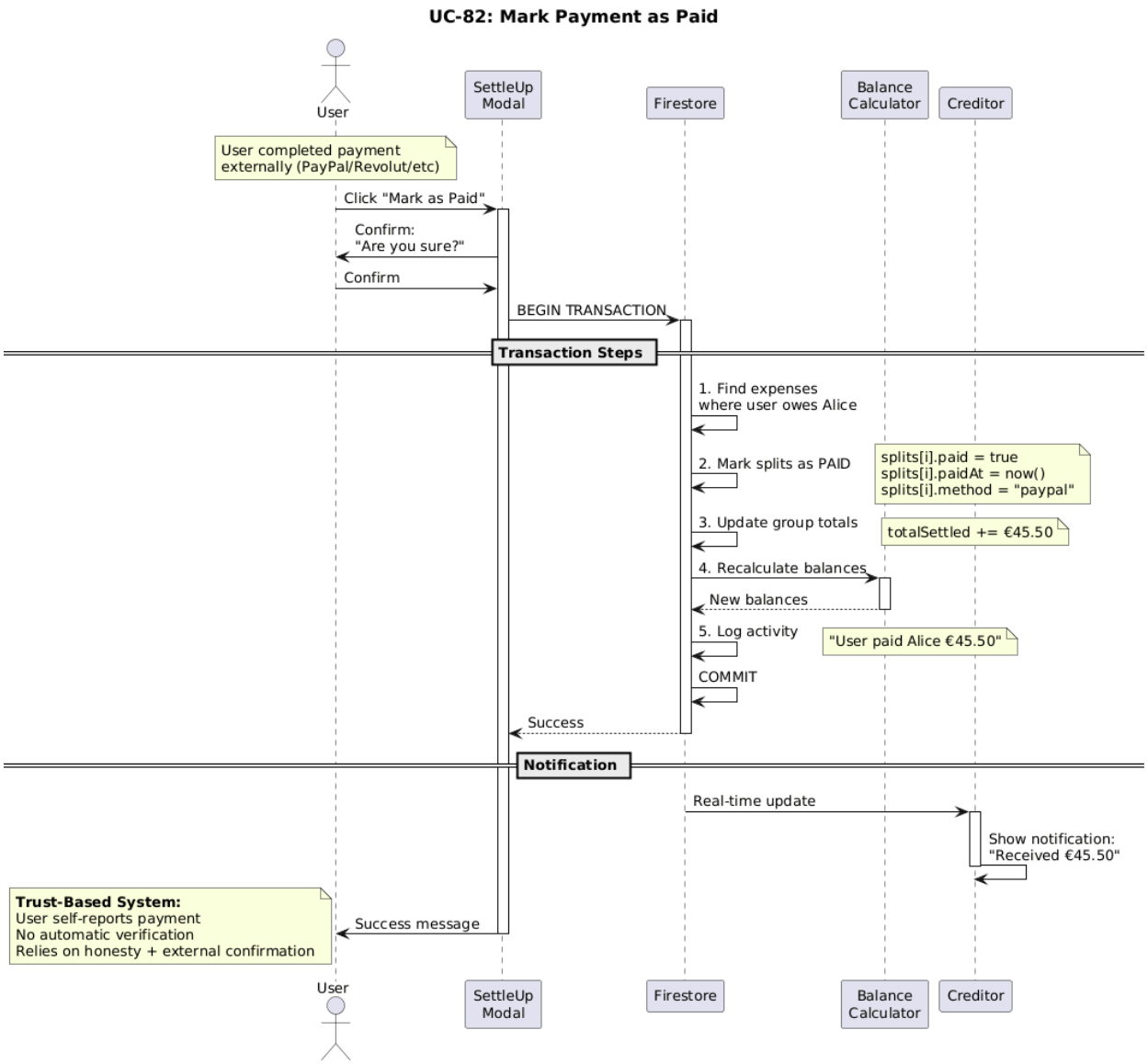


Figure 12: View Group Balance Sequence Diagram

5.5.9 UC-82: Mark Payment as Paid

The payment confirmation sequence implements a trust-based recording mechanism following external fund transfer, executing through ten coordinated steps ensuring data consistency: (1)

User returns to Split-Sync after completing payment through external platform (PayPal, Revolut, SEPA, or cash), having received confirmation from the payment provider, (2) SettleUpModal presents a confirmation dialog explicitly asking "Did you complete the payment?" with payment details (amount, recipient, method) displayed for verification, (3) User clicks "Mark as Paid" triggering a secondary confirmation dialog to prevent accidental marking, emphasizing that this action will notify the creditor and update group balances, (4) Upon final confirmation, system initiates a Firestore transaction guaranteeing atomic execution of all subsequent database operations, (5) Transaction first queries the expenses collection identifying all unpaid splits where the current user owes money to the specified creditor (Alice), ordering results chronologically to apply payment to oldest debts first, (6) For each identified expense, system updates the split object setting `paid: true,` recording `paidAt` timestamp, payment method, and reference note, additionally marking entire expenses as settled when all splits are paid, (7) System updates group-level financial metrics incrementing `totalSettled` counter and updating `lastActivity` timestamp for analytics tracking, (8) Balance calculator service recalculates net positions for both debtor and creditor: debtor's balance increases (less negative) by payment amount while creditor's receivables decrease by equivalent amount, with updated values persisted to the balances subcollection, (9) Activity logger creates comprehensive audit trail document recording payment action with metadata including amount, currency, payment method, affected parties, and timestamp for dispute resolution and historical tracking, and (10) Upon successful transaction commit, Firestore's real-time listeners trigger immediate notification delivery to creditor displaying balance update and payment notification across all active devices. The architecture employs a trust-based verification model where users self-report payment completion without automatic confirmation from payment gateways, accepting this trade-off to avoid API integration complexity, API key management, and operational costs while relying on social accountability within small trusted groups, external payment confirmations (emails from PayPal, Revolut, bank statements), comprehensive activity logging for dispute resolution, and the ability to reverse erroneous markings if mistakes occur. This design prioritizes implementation simplicity and zero-cost operation over technical verification, appropriate for the application's target demographic of friends, family, and small team expense sharing scenarios where social trust mechanisms provide sufficient fraud deterrence.



Main Success Scenario:

1. External Payment Completed:
    - User has paid Alice €45.50 via PayPal
    - PayPal transaction ID: 8FK39DK2J
    - Both parties received confirmation emails
    - Money successfully transferred
2. Return to Split-Sync:
    - User closes PayPal tab
    - Returns to SettleUpModal

- Modal still open showing payment details

3. Initial Confirmation Dialog:
    - Modal displays: "Did you complete the payment?"
    - Shows: Amount €45.50, To: Alice Cooper, Method: PayPal
    - Warning: "Only mark as paid if money was actually transferred"
    - User clicks "Mark as Paid"
4. Secondary Confirmation:
    - Confirmation dialog opens
    - Text: "This will notify Alice and update balances. Are you sure?"
    - Prevents accidental clicks
    - User clicks "Yes, Confirm"
5. Transaction Initialization:
    - System calls db.runTransaction()
    - Begins atomic transaction
    - All subsequent operations within transaction scope
6. Query Unpaid Expenses:

    - Query: `expenses WHERE paidBy='alice' AND splits[].userId='user' AND splits[].paid=false`
    - Returns 2 expenses:
    - Expense 1 (Hotel): User owes €30.00
    - Expense 2 (Dinner): User owes €15.50
    - Total: €45.50 (matches payment amount)

7. Update Expense Splits:

- Expense 1 Update:

```
splits[userIndex].paid = true
splits[userIndex].paidAt = serverTimestamp()
splits[userIndex].paymentMethod = 'paypal'
```

```
splits[userIndex].paymentReference = 'User marked as paid'
splits[userIndex].paidAmount = 30.00
```

- Check if all splits paid → isSettled = true
- Expense 2 Update: Same process for €15.50

8. Update Group Totals:

```
UPDATE /groups/weekend-trip
   SET totalSettled = totalSettled + 45.50
   SET lastActivity = serverTimestamp()
```

- Before: totalSettled = €100
- After: totalSettled = €145.50

9. Recalculate User Balance:

Before:

- totalOwes = €68.50 (Alice €45.50 + Bob €23.00)
- totalOwed = €0
- netBalance = -€68.50

After:

- totalOwes = €23.00 (Bob only)
- totalOwed = €0
- netBalance = -€23.00

Update balance sheet: Remove Alice, keep Bob

10. Recalculate Alice Balance:

Before:

- totalOwed = €75.50 (User €45.50 + Bob €30.00)
- totalOwes = €0
- netBalance = +€75.50

After:

- totalOwed = €30.00 (Bob only)
- totalOwes = €0
- netBalance = +€30.00

- Update balance sheet: Remove User, keep Bob

11. Create Activity Log:

```
CREATE /groups/weekend-trip/activity/act123
    {
      type: 'payment_made',
      userId: 'user123',
      userName: 'User Name',
      userPhoto: 'https://...',
      description: 'User paid Alice €45.50 via PayPal',
      metadata: {
        paymentAmount: 45.50,
        currency: 'EUR',
        paymentMethod: 'paypal',
        paidTo: 'alice123',
        paidToName: 'Alice Cooper',
        expensesSettled: ['exp1', 'exp2'],
        previousBalance: -68.50,
        newBalance: -23.00
      },
      timestamp: serverTimestamp(),
      isVisible: true
    }
```

12. Commit Transaction:

- All 5 operations succeed
- Transaction commits atomically
- Changes become permanent and visible

- No partial updates possible

13. Real-Time Notification:
    - Firestore triggers `onSnapshot` for Alice
    - Alice's app receives real-time update
    - Balance view updates: €75.50 → €30.00
    - Notification displays: "User paid you €45.50 via PayPal"
    - Activity feed shows new entry

14. UI Updates:
    - SettleUpModal closes
    - Group Details page refreshes
    - Balance section re-renders with new data
    - Success toast appears: "Payment Recorded! ✓"

15. Email Verification (Best Practice):
    - Alice checks PayPal email inbox
    - Verifies email: "You received €45.50 from User Name"
    - Amount matches Split-Sync record ✓
    - Sender matches ✓
    - Payment confirmed on both systems

Alternative Flow 1: User Cancels

- Steps 1-3: Same as main flow
- Step 4: User clicks "Not Yet" or "Cancel"
- Step 5: Dialog closes, no database changes
- Step 6: User remains in SettleUpModal
- Step 7: Can retry later or choose different payment method

Alternative Flow 2: Transaction Fails

- Steps 1-11: Same as main flow
- Step 12: One operation fails (network error, permission denied)
- Step 13: Firestore automatically rolls back entire transaction
- Step 14: All changes reverted (nothing saved)
- Step 15: Modal displays error: "Payment recording failed. Please try again."
- Step 16: User sees "Retry" button
- No partial updates, maintains data consistency

Alternative Flow 3: Amount Mismatch

- Steps 1-6: Same as main flow
- Step 7: System finds expenses totaling €50.00 (not €45.50)
- Step 8: Payment allocation algorithm detects mismatch
- Step 9: Options:
- A) Reject (current implementation)
- B) Allow partial payment (future enhancement)
- Step 10: Error displayed: "Payment amount doesn't match debts"
- User must contact group admin to resolve

Alternative Flow 4: Dishonest Marking

- Steps 1-14: User marks as paid WITHOUT actually paying
- Step 15: Alice checks PayPal email
- Step 16: No payment email received
- Step 17: Alice contacts user in group chat
- Step 18: Options:
- User admits mistake, admin can reverse marking
- User claims they paid, provide transaction ID
- Escalate to group vote or admin decision

- Group social pressure prevents fraud

PRECONDITIONS:

- User is authenticated
- User is member of group
- User has outstanding debt to creditor (Alice)
- User has completed external payment (PayPal/Revolut/SEPA/Cash)
- User has received confirmation from payment platform
- Payment amount matches debt amount (or total debts)
- SettleUpModal is open showing payment details

POSTCONDITIONS:

Success:

- Expense splits marked as paid in Firestore
- `splits[].paid = true` for all relevant splits
- Payment timestamp, method, reference recorded
- Fully settled expenses marked `isSettled = true`
- Group `totalSettled` incremented by payment amount
- User's balance reduced (less owed)
- Creditor's balance reduced (less receivable)
- Activity log created documenting payment
- Real-time notification sent to creditor
- UI updated showing new balances
- Success toast displayed to user

Failure (Transaction Rollback):

- No database changes made
- Balances unchanged
- No activity log created
- No notification sent

- Error message displayed
- User can retry operation
- External payment still valid (PayPal side)

5.5.10 UC-92: View Activity Feed

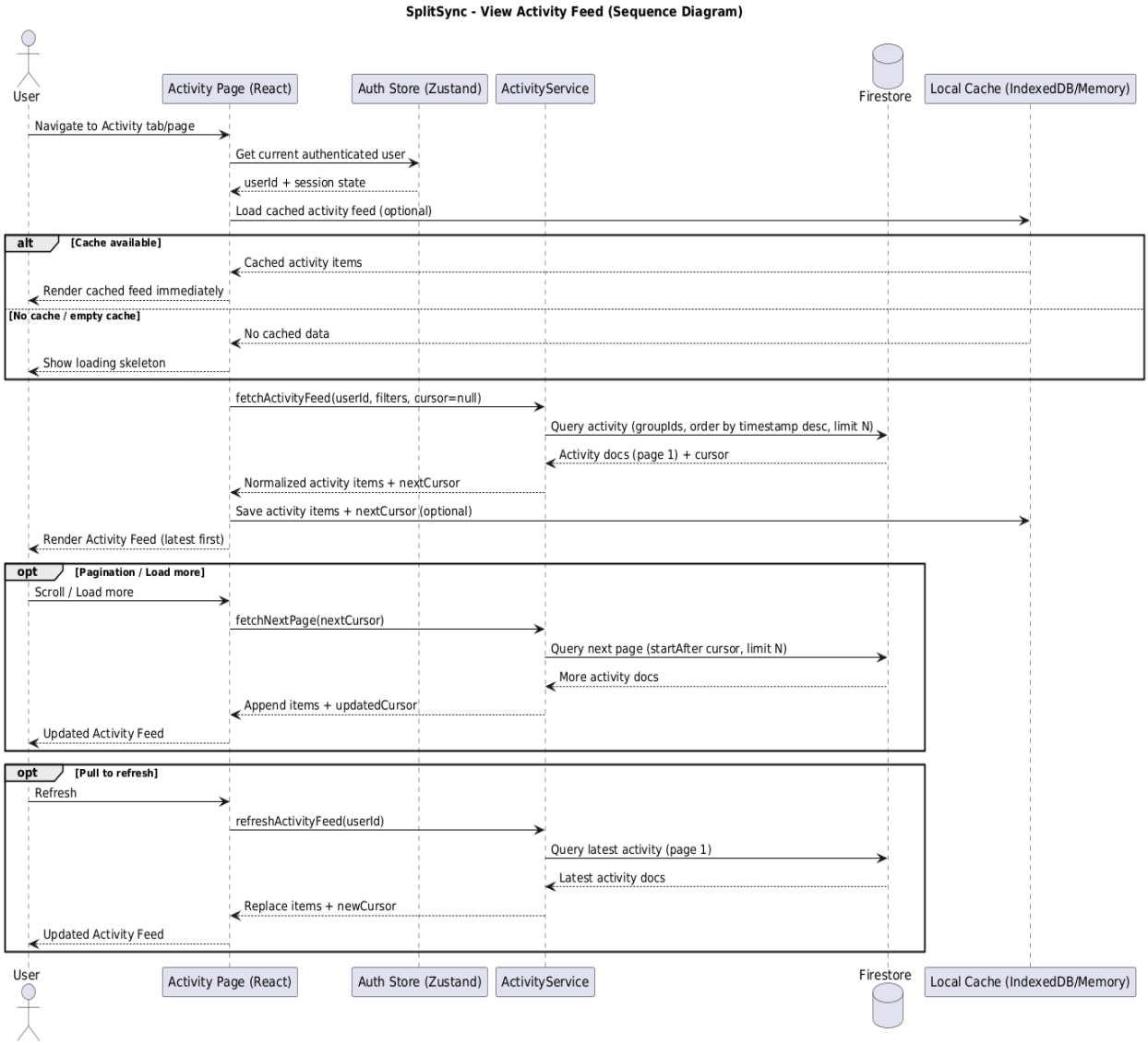


Figure 13: View Activity Feed Sequence Diagram

Main Success Scenario:

1. User navigates to the **Activity** page/tab
2. System checks authentication state and retrieves current **userId**
3. System loads cached activity items (if available)
4. System displays cached feed immediately (optional)
5. System shows loading skeleton while fetching latest feed (if needed)
6. System calls `ActivityService.fetchActivityFeed(userId, filters, cursor=null`
7. ActivityService queries Firestore for activity records belonging to the user's groups
8. Firestore returns latest activity documents ordered by timestamp (descending)
9. ActivityService normalizes results into UI-ready activity items
10. System renders the Activity Feed (latest-first) with timestamps and event labels
11. System stores the latest feed and cursor in local cache (optional)
    User scrolls to bottom
12. System loads next page using cursor and appends activity items
13. User sees extended feed successfully

5.6 Alternative Flows

5.6.1 Alternative Flow 1: User Not Authenticated

- Steps 1-2: Same as main flow
- System detects user is not authenticated
- System redirects user to `/login`
- System displays message: "Please log in to view activity"
- Flow ends

5.6.2 Alternative Flow 2: User Has No Groups

- Steps 1-6: Same as main flow
- ActivityService determines user has no group membership
- System displays empty state: "No activity yet"
- System shows CTA: "Join a group" / "Create a group"

- Flow ends

5.6.3 Alternative Flow 3: Firestore Permission Denied

- Steps 1-7: Same as main flow
- Firestore rejects query due to security rules / missing membership mapping
- System displays error: "You don’t have access to this activity feed"
- System logs error for debugging (non-sensitive)
- Flow ends

5.6.4 Alternative Flow 4: Network Offline (Cache Available)

- Steps 1-4: Same as main flow
- System detects network offline during fetch
- System continues showing cached activity feed
- System displays banner: "Offline mode — showing saved activity"
- User can retry when network returns
- Flow ends

5.6.5 Alternative Flow 5: Network Offline (No Cache)

- Steps 1-5: Same as main flow
- System detects network offline and cache is empty
- System displays offline empty state: "No internet connection"
- System shows "Retry" button
- Flow ends

5.6.6 Alternative Flow 6: Large Activity Feed (Pagination Required)

- Steps 1-11: Same as main flow
- User scrolls down
- System requests next page using cursor

- Firestore returns next batch of activity documents
- System appends items and updates cursor
- Repeat until no more results
- Flow ends

PRECONDITIONS:

- User is authenticated
- User account exists in Firebase Auth
- User has valid membership mapping to at least one group (unless empty-state supported
- Firestore indexes exist for activity queries (timestamp ordering)
- Internet connection available (or cache available for offline display)
- Firestore security rules permit the user to read activity for their group.

POSTCONDITIONS:

Success:

- Activity Feed displayed in chronological order (latest-first)
- Activity items visible with event type, actor, group context, and timestamps
- Pagination cursor stored for subsequent loads (if pagination enabled)
- Cached activity updated for faster reload/offline use (if caching enabled)

Failure (Not Authenticated):

- No activity loaded
- User redirected to login
- Helpful message displayed

Failure (Permission Denied):

- No activity loaded
- Access error displayed

- Error logged for debugging

Failure (Offline, No Cache):

- No activity loaded
- Offline message displayed
- Retry option available

6. CLASS DIAGRAM

6.1 Overview

The Split-Sync class diagram illustrates the complete object-oriented design of the system, showing 18 core classes, 6 enumerations, and their relationships. The architecture follows a **layered approach** separating domain entities from business logic services, ensuring maintainability and scalability.

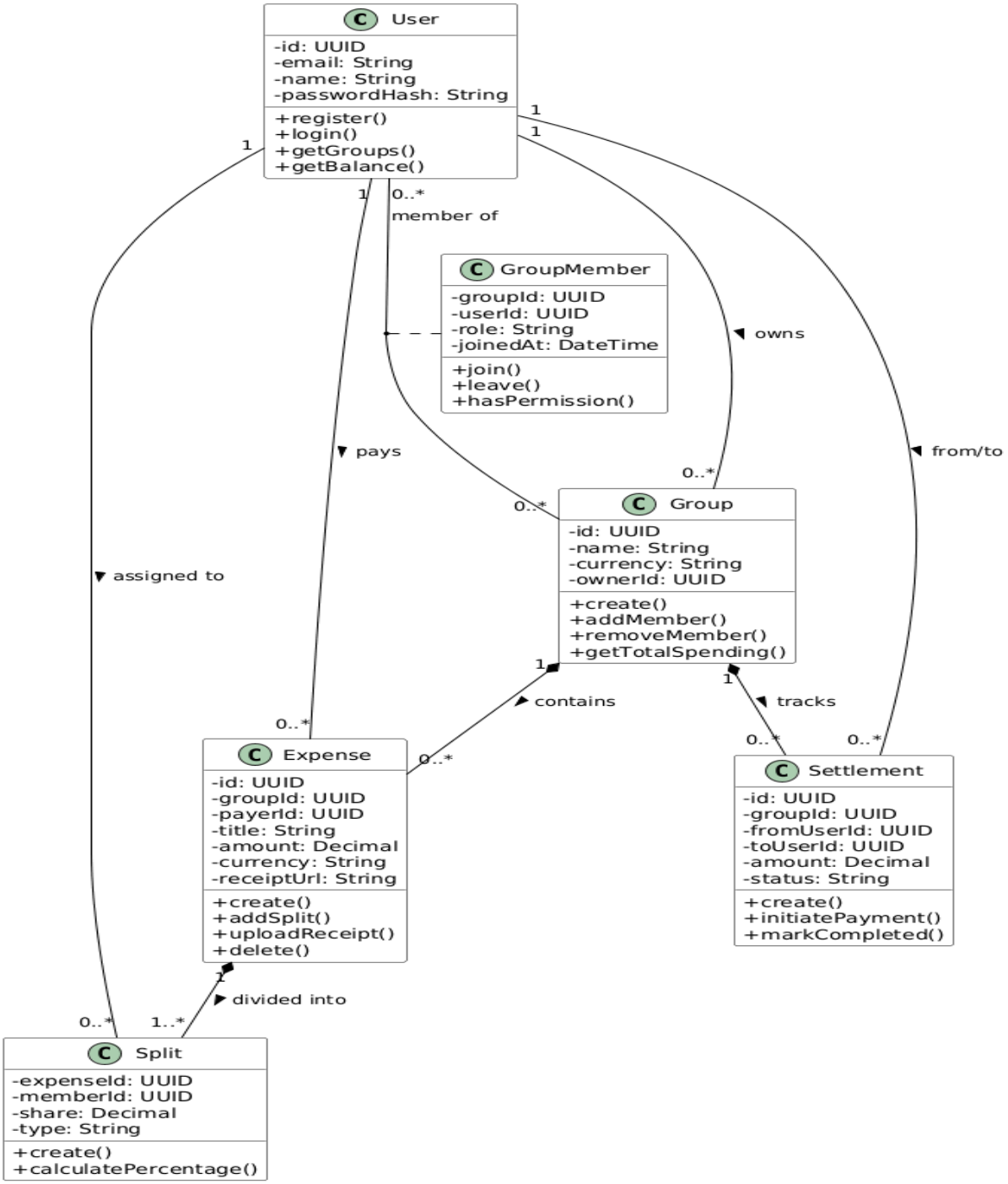


Figure14: Class Diagram

6.2 Architecture Layers

6.2.1 Domain Model Layer (Core Business Entities)

The domain layer contains 11 classes representing the fundamental business concepts:

User Management:

- **User** class manages authentication, profile information, and serves as the central entity linking all user activities across groups, expenses, and settlements.

Group Management:

- **Group** class represents expense-sharing contexts (trips, households, events) with configurable settings like currency and member permissions.
- **GroupMember** resolves the many-to-many relationship between Users and Groups, storing role-based permissions (Owner, Admin, Member) that control what actions members can perform.

Expense Tracking:

- **Expense** class records individual transactions with support for receipt attachments and OCR processing status.
- **Split** class implements the expense distribution logic, supporting four split types: equal, percentage, weighted, and per-item splitting.

Settlement Processing:

- **Settlement** class tracks payment transactions between users, maintaining status throughout the payment lifecycle (pending, processing, completed, failed).

Supporting Classes:

- **OCRJob** tracks receipt processing through external OCR engines, storing extracted text and confidence scores.
- **Notification** manages user communications across multiple channels (push, email, SMS).
- **AuditLog** maintains an immutable record of all significant system changes for security and debugging.
- **WebhookEvent** stores payment gateway callbacks, enabling idempotent webhook processing.

6.2.2 Service Layer (Business Logic)

Seven service classes encapsulate complex business operations:

AuthenticationService handles user registration, login, session management, and password operations, separating security concerns from the User entity.

GroupService orchestrates group lifecycle operations including creation, member invitations, role management, and deletion, ensuring business rules are consistently enforced.

ExpenseService manages expense CRUD operations, coordinates receipt uploads with OCR processing, and handles expense duplication for recurring costs.

BalanceCalculator implements the core financial logic, calculating net balances between users and executing the minimum cash-flow optimization algorithm that reduces settlement complexity from $O(n^2)$ to approximately $O(n)$.

PaymentService integrates with external payment gateways (Stripe, PayPal), processes webhook callbacks, handles payment verification, and manages refunds when necessary.

OCRService abstracts receipt text extraction, supporting multiple OCR engines (Tesseract, Google Vision) with fallback strategies based on confidence scores.

NotificationService centralizes all user communications, triggering appropriate notifications for expense additions, payment completions, and balance reminders.

SplitCalculator encapsulates the mathematical logic for different split calculations, ensuring split totals always equal expense amounts through validation.

6.3 Key Design Patterns

Separation of Concerns: Domain entities focus on data representation while services handle complex business logic, improving testability and maintainability.

Single Responsibility Principle: Each class has one clearly defined purpose. For example, ExpenseService manages expense operations while SplitCalculator focuses solely on split calculations.

Encapsulation: All attributes are private with public methods providing controlled access, protecting data integrity.

Composition over Inheritance: The design favors composition (Group contains GroupMembers) rather than deep inheritance hierarchies, increasing flexibility.

Strategy Pattern: Different split types (equal, percentage, weighted, per-item) represent interchangeable strategies for expense distribution.

Repository Pattern: Services act as repositories, abstracting data access and business rules from domain entities.

6.4 Relationships

Composition (Strong Ownership):

- Group - GroupMember: Groups own their memberships; deleting a group removes its members.
- Group - Expense: Groups own their expenses; group deletion cascades to expenses.
- Expense - Split: Expenses own their splits; expense deletion removes associated splits.

Association (Weak Connection):

- User -- Group: Many-to-many relationship resolved through GroupMember junction class.
- User -- Expense: One-to-many relationship where users pay multiple expenses.
- User -- Settlement: Users participate in settlements as payers or recipients.

Dependency (Uses Relationship):

- Services depend on domain classes they manipulate.
- ExpenseService depends on OCRService for receipt processing.
- PaymentService depends on WebhookEvent for callback handling.

6.5 Enumerations for Type Safety

Six enumerations ensure type-safe constants throughout the system:

- **Role** (OWNER, ADMIN, MEMBER) - Permission levels
- **SplitType** (EQUAL, PERCENTAGE, WEIGHTED, PER_ITEM) - Distribution methods
- **PaymentMethod** (CARD, BANK_TRANSFER, DIGITAL_WALLET, CASH) - Payment options
- **SettlementStatus** (PENDING, PROCESSING, COMPLETED, FAILED, CANCELLED) - Payment lifecycle
- **OCRStatus** (PENDING, PROCESSING, COMPLETED, FAILED) - Receipt processing states
- **NotificationType** & **Channel** - Communication types and delivery methods

6.6 Data Flow Example

When a user adds an expense with a receipt:

1. ExpenseService.createExpense() creates an Expense entity

2. ExpenseService.uploadReceipt() stores the image and creates an OCRJob
3. OCRService.processReceipt() extracts text from the receipt
4. SplitCalculator calculates splits based on the specified method
5. ExpenseService creates Split entities linked to the Expense
6. BalanceCalculator recalculates net balances for affected users
7. NotificationService sends notifications to all group members

6.7 Security Considerations

Authentication: User class stores password hashes (never plain text) using bcrypt with appropriate work factors.

Authorization: GroupMember role enumeration enables role-based access control, with permission checks occurring in service methods before operations.

Audit Trail: AuditLog records all significant actions, creating an immutable history for security investigations and debugging.

Payment Security: PaymentService delegates sensitive operations to PCI-compliant payment gateways, never storing card details.

Idempotency: WebhookEvent tracking prevents duplicate payment processing from webhook retries.

6.8 Scalability Considerations

Service Layer: Business logic in services rather than domain entities enables horizontal scaling of processing capability.

Stateless Design: Services don't maintain state between method calls, allowing load distribution across multiple instances.

Asynchronous Processing: OCRJob and WebhookEvent support asynchronous operations, preventing long-running tasks from blocking user interactions.

Caching Strategy: BalanceCalculator results can be cached and invalidated only when relevant expenses or settlements change.

6.9 Class diagram Summary

This class diagram demonstrates a well-structured, object-oriented architecture following industry best practices. The clear separation between domain entities and services, comprehensive relationship mapping, and thoughtful use of design patterns create a maintainable foundation for the Split-Sync application. The design supports all functional requirements specified in the project documentation while remaining flexible enough to accommodate future enhancements such as multi-currency support, recurring expenses, and advanced analytics.

7.0 PSEUDOCODES

7.1 Overview

This section presents pseudocode for the critical algorithms implemented in Split-Sync. Pseudocode provides a language-independent description of algorithmic logic, facilitating understanding and maintenance whilst serving as implementation documentation (Cormen et al., 2009).

Notation Conventions:

- ← Assignment operator
- == Equality comparison
- != Inequality comparison
- AND, OR, NOT Boolean operators
- [ ] Array/list notation
- { } Object/record notation
- RETURN Exit function with value
- THROW Raise exception

7.2 Balance Calculation Algorithm

7.2.1 Purpose

Calculate net balances between all group members, determining who owes whom and how much. This is the core financial algorithm in Split-Sync.

Input

- groupId: String (group identifier)
- expenses: Array of expense objects

Output

- balanceSheet: Map of user pairs to amounts owed

Complexity

- **Time:** O(n × m) where n = number of expenses, m = average splits per expense
- **Space:** O(u²) where u = number of unique users

Pseudocode

```
ALGORITHM calculateGroupBalances(groupId)

INPUT: groupId (String)
OUTPUT: balanceSheet (Map<String, Map<String, Number>>)

1:  expenses ← QUERY Firestore WHERE groupId == groupId
2:  balanceSheet ← EMPTY_MAP()

    // Phase 1: Initialize balance matrix
3:  FOR EACH expense IN expenses DO
4:      paidBy ← expense.paidBy
5:      FOR EACH split IN expense.splits DO
6:          userId ← split.userId
7:          amount ← split.amount
8:          paid ← split.paid

            // Skip if already paid
9:          IF paid == TRUE THEN
10:             CONTINUE
11:         END IF

            // Initialize nested map if needed
12:         IF paidBy NOT IN balanceSheet THEN
13:             balanceSheet[paidBy] ← EMPTY_MAP()
14:         END IF
15:         IF userId NOT IN balanceSheet THEN
16:             balanceSheet[userId] ← EMPTY_MAP()
```

```
17:         END IF

            // Skip self-payments
18:         IF paidBy == userId THEN
19:             CONTINUE
20:         END IF

            // Update balance: paidBy is owed by userId
21:         IF userId NOT IN balanceSheet[paidBy] THEN
22:             balanceSheet[paidBy][userId] ← 0
23:         END IF
24:                                 balanceSheet[paidBy][userId]    ←
balanceSheet[paidBy][userId] + amount

            // Symmetric update: userId owes paidBy
25:         IF paidBy NOT IN balanceSheet[userId] THEN
26:             balanceSheet[userId][paidBy] ← 0
27:         END IF
28:                                 balanceSheet[userId][paidBy]    ←
balanceSheet[userId][paidBy] - amount
29:     END FOR
30: END FOR

    // Phase 2: Simplify balances (cancel out reciprocal debts)
31: FOR EACH userA IN balanceSheet.keys() DO
32:     FOR EACH userB IN balanceSheet[userA].keys() DO
33:         amountAtoB ← balanceSheet[userA][userB]
34:         amountBtoA ← balanceSheet[userB][userA] IF EXISTS ELSE 0

            // If both owe each other, simplify
35:         IF amountAtoB > 0 AND amountBtoA < 0 THEN
36:             netAmount ← amountAtoB + amountBtoA  // Since BtoA is
negative
37:             IF netAmount > 0 THEN
                    // A is still owed money by B
```

```
38:                 balanceSheet[userA][userB] ← netAmount
39:                 DELETE balanceSheet[userB][userA]
40:             ELSE IF netAmount < 0 THEN
                    // B is actually owed money by A
41:                 DELETE balanceSheet[userA][userB]
42:                 balanceSheet[userB][userA] ← ABS(netAmount)
43:             ELSE
                    // Perfectly balanced
44:                 DELETE balanceSheet[userA][userB]
45:                 DELETE balanceSheet[userB][userA]
46:             END IF
47:         END IF
48:     END FOR
49: END FOR


// Phase 3: Calculate net balance for each user
50: netBalances ← EMPTY_MAP()
51: FOR EACH userA IN balanceSheet.keys() DO
52:     netBalances[userA] ← 0
53:     FOR EACH userB IN balanceSheet[userA].keys() DO
54:                     netBalances[userA]  ←  netBalances[userA]  +
balanceSheet[userA][userB]
55:     END FOR
56: END FOR

57: RETURN {
        balanceSheet: balanceSheet,
        netBalances: netBalances
    }

END ALGORITHM
```

7.2.2 Example Execution

Input:

Group: "Beach Trip"

Expenses:

1. Alice paid €100, split equally among Alice, Bob, Carol
   → Bob owes Alice €33.33
   → Carol owes Alice €33.33

2. Bob paid €60, split equally among Alice, Bob
   → Alice owes Bob €30

Execution:

```
Phase 1: Build matrix
  balanceSheet[Alice][Bob] = €33.33
  balanceSheet[Alice][Carol] = €33.33
  balanceSheet[Bob][Alice] = -€30

Phase 2: Simplify
  Alice owes Bob €33.33 - €30 = €3.33 (net)
  Result: balanceSheet[Bob][Alice] = €3.33

  Carol owes Alice €33.33 (unchanged)
  Result: balanceSheet[Alice][Carol] = €33.33

Phase 3: Net balances
  netBalances[Alice] = +€33.33 - €3.33 = +€30 (owed to Alice)
  netBalances[Bob] = +€3.33 (owed to Bob)
  netBalances[Carol] = -€33.33 (Carol owes)
```

Output:

```
{
  balanceSheet: {
    Alice: { Carol: 33.33 },
    Bob: { Alice: 3.33 }
  },
  netBalances: {
    Alice: 30.00,
    Bob: 3.33,
    Carol: -33.33
  }
}
```

7.3 Split Calculation Algorithms

7.3.1 Equal Split

Purpose: Divide expense amount equally among selected members.

```
ALGORITHM calculateEqualSplit(amount, members)

INPUT: amount (Number), members (Array<String>)
OUTPUT: splits (Array<{userId: String, amount: Number}>)

1:  n ← LENGTH(members)
2:  IF n == 0 THEN
3:      THROW "No members selected"
4:  END IF

5:  perPerson ← ROUND(amount / n, 2)  // Round to 2 decimal places

    // Handle rounding errors
6:  total ← perPerson × n
7:  difference ← amount - total
```

```
8:  splits ← EMPTY_ARRAY()
9:  FOR i ← 0 TO n-1 DO
10:     splitAmount ← perPerson

        // Add rounding difference to first person
11:     IF i == 0 THEN
12:         splitAmount ← splitAmount + difference
13:     END IF

14:     APPEND({
            userId: members[i],
            amount: splitAmount
        }) TO splits
15: END FOR

16: RETURN splits

END ALGORITHM
```

Example:

- Amount: €100.00
- Members: [Alice, Bob, Carol]
- Per person: €100 ÷ 3 = €33.33
- Rounding: €100 - (€33.33 × 3) = €100 - €99.99 = €0.01
- Result:
    - Alice: €33.34 (gets rounding difference)
    - Bob: €33.33
    - Carol: €33.33

7.3.2 Percentage Split

Purpose: Divide expense based on percentage allocation, with automatic adjustment to 100%.

```
ALGORITHM calculatePercentageSplit(amount, members, percentages)

INPUT:
  amount (Number)
  members (Array<String>)
  percentages (Map<String, Number>)  // userId → percentage
OUTPUT:
    splits (Array<{userId: String, amount: Number, percentage:
Number}>)

1:  totalPercentage ← 0
2:  FOR EACH member IN members DO
3:      totalPercentage ← totalPercentage + percentages[member]
4:  END FOR

    // Auto-adjust if not exactly 100%
5:  IF totalPercentage != 100 THEN
6:      adjustmentFactor ← 100 / totalPercentage
7:      FOR EACH member IN members DO
8:                    percentages[member] ← percentages[member] ×
adjustmentFactor
9:      END FOR
10: END IF

11: splits ← EMPTY_ARRAY()
12: calculatedTotal ← 0

13: FOR i ← 0 TO LENGTH(members)-1 DO
14:     member ← members[i]
15:     percentage ← percentages[member]
16:     splitAmount ← ROUND((amount × percentage) / 100, 2)
```

```
        // Track calculated total
17:     calculatedTotal ← calculatedTotal + splitAmount

18:     APPEND({
            userId: member,
            amount: splitAmount,
            percentage: percentage
        }) TO splits
19: END FOR

    // Adjust last person for rounding errors
20: difference ← amount - calculatedTotal
21: IF difference != 0 THEN
22:                             splits[LENGTH(splits)-1].amount      ←
splits[LENGTH(splits)-1].amount + difference
23: END IF

24: RETURN splits

END ALGORITHM
```

Example with Auto-Adjustment:

- Amount: €100.00
- Percentages: Alice 50%, Bob 30%, Carol 15% (Total: 95%)
- Auto-adjust to 100%:
    - Alice: 50 × (100/95) = 52.63%
    - Bob: 30 × (100/95) = 31.58%
    - Carol: 15 × (100/95) = 15.79%
- Calculated amounts:
    - Alice: €100 × 0.5263 = €52.63
    - Bob: €100 × 0.3158 = €31.58

- Carol: €100 × 0.1579 = €15.79

7.3.3 Shares Split

Purpose: Divide expense based on share units (e.g., 2 shares vs 1 share).

```
ALGORITHM calculateSharesSplit(amount, members, shares)

INPUT:
  amount (Number)
  members (Array<String>)
  shares (Map<String, Integer>)  // userId → share count
OUTPUT:
  splits (Array<{userId: String, amount: Number, shares: Integer}>)

1:  totalShares ← 0
2:  FOR EACH member IN members DO
3:      totalShares ← totalShares + shares[member]
4:  END FOR

5:  IF totalShares == 0 THEN
6:      THROW "Total shares cannot be zero"
7:  END IF

8:  perShare ← amount / totalShares

9:  splits ← EMPTY_ARRAY()
10: calculatedTotal ← 0

11: FOR EACH member IN members DO
12:     memberShares ← shares[member]
13:     splitAmount ← ROUND(perShare × memberShares, 2)
14:     calculatedTotal ← calculatedTotal + splitAmount

15:     APPEND({
```

```
            userId: member,
            amount: splitAmount,
            shares: memberShares
        }) TO splits
16: END FOR

    // Adjust for rounding
17: difference ← amount - calculatedTotal
18: IF difference != 0 THEN
19:     splits[0].amount ← splits[0].amount + difference
20: END IF

21: RETURN splits

END ALGORITHM
```

Example:

- Amount: €100.00
- Shares: Alice 2, Bob 2, Carol 1 (Total: 5 shares)
- Per share: €100 ÷ 5 = €20
- Result:
    - Alice: €20 × 2 = €40.00
    - Bob: €20 × 2 = €40.00
    - Carol: €20 × 1 = €20.00

7.4 Contact Matching Algorithm (Privacy-Preserving)

7.4.1 Purpose

Match phone contacts to Split-Sync users without storing plaintext phone numbers, using SHA-256 hashing.

```
ALGORITHM matchContactsToUsers(contacts, userCountryCode)

INPUT:
  contacts (Array<{name: String, phoneNumbers: Array<String>}>)
  userCountryCode (String)  // e.g., "IE" for Ireland
OUTPUT:
  results ({matched: Array, unmatched: Array})

1:  phoneHashMap ← EMPTY_MAP()  // Temporary hash storage
2:  contactHashMap ← EMPTY_MAP()  // Contact ID → hash mapping

    // Phase 1: Normalize and hash phone numbers
3:  FOR EACH contact IN contacts DO
4:      FOR EACH phoneNumber IN contact.phoneNumbers DO
5:              normalizedPhone ← normalizePhoneNumber(phoneNumber,
userCountryCode)

6:          IF normalizedPhone == NULL THEN
7:              CONTINUE  // Skip invalid numbers
8:          END IF

9:          hashedPhone ← await hashPhoneNumber(normalizedPhone)
10:         phoneHashMap[hashedPhone] ← contact.id
11:         contactHashMap[contact.id] ← hashedPhone
12:     END FOR
13: END FOR

    // Phase 2: Query Firestore in batches (max 10 per query)
14: hashes ← KEYS(phoneHashMap)
15: batches ← chunkArray(hashes, 10)  // Split into groups of 10
16: matchedUsers ← EMPTY_ARRAY()

17: FOR EACH batch IN batches DO
18:     query ← Firestore.collection('phoneNumbers')
                .where('hashedPhone', 'in', batch)
```

```
19:     querySnapshot ← await query.get()

20:     FOR EACH doc IN querySnapshot.docs DO
21:         APPEND doc.data() TO matchedUsers
22:     END FOR
23: END FOR

    // Phase 3: Match contacts to users
24: matched ← EMPTY_ARRAY()
25: unmatched ← EMPTY_ARRAY()
26: matchedContactIds ← EMPTY_SET()

27: FOR EACH user IN matchedUsers DO
28:     hash ← user.hashedPhone
29:     IF hash IN phoneHashMap THEN
30:         contactId ← phoneHashMap[hash]
31:         contact ← FIND contact WHERE id == contactId

32:         APPEND({
                contactName: contact.name,
                userId: user.userId,
                userName: user.name,
                userEmail: user.email,
                userPhoto: user.photoURL
            }) TO matched

33:         ADD contactId TO matchedContactIds
34:     END IF
35: END FOR

36: FOR EACH contact IN contacts DO
37:     IF contact.id NOT IN matchedContactIds THEN
38:         APPEND({
                contactName: contact.name,
                phoneNumbers: contact.phoneNumbers
```

```
            }) TO unmatched
39:     END IF
40: END FOR

    // Phase 4: Security cleanup
41: CLEAR phoneHashMap
42: CLEAR contactHashMap
43: CLEAR hashes
44: // All plaintext phone numbers and hashes now purged from memory

45: RETURN {
        matched: matched,
        unmatched: unmatched
    }

END ALGORITHM
```

7.4.2 Supporting Functions

Phone Number Normalization:

```
FUNCTION normalizePhoneNumber(phone, countryCode)

INPUT: phone (String), countryCode (String)
OUTPUT: normalized (String) OR NULL

1:  TRY
2:      // Use libphonenumber-js library
3:      parsed ← parsePhoneNumber(phone, countryCode)

4:      IF parsed.isValid() == FALSE THEN
5:          RETURN NULL
```

```
6:      END IF

7:      normalized ← parsed.format('E.164')  // e.g., "+353871234567"
8:      RETURN normalized

9:  CATCH error
10:     RETURN NULL  // Invalid phone number
11: END TRY

END FUNCTION
```

SHA-256 Hashing:

```
ASYNC FUNCTION hashPhoneNumber(normalizedPhone)

INPUT: normalizedPhone (String)
OUTPUT: hash (String)  // 64 hex characters

1:  encoder ← new TextEncoder()
2:  data ← encoder.encode(normalizedPhone)

3:  // Use Web Crypto API (browser-native)
4:  hashBuffer ← await crypto.subtle.digest('SHA-256', data)

5:  // Convert ArrayBuffer to hex string
6:  hashArray ← Array.from(new Uint8Array(hashBuffer))
7:  hexHash ← EMPTY_STRING
8:  FOR EACH byte IN hashArray DO
9:      hexHash ← hexHash + byte.toString(16).padStart(2, '0')
10: END FOR

11: RETURN hexHash  // e.g., "a3f5b8c2d9e1..."

END FUNCTION
```

7.4.3 Complexity Analysis

- **Normalization:** O(n × m) where n = contacts, m = avg phone numbers per contact
- **Hashing:** O(n × m) — SHA-256 is O(1) per number
- **Firestore queries:** O(q) where q = ⌈(n × m) / 10⌉ batch queries
- **Matching:** O(u + c) where u = matched users, c = total contacts
- **Total:** O(n × m + q + u + c) ≈ O(n × m) for typical usage

7.4.4 Privacy Properties

1. **No Server Storage:** Phone numbers never sent to Firebase (lines 5-9)
2. **One-Way Encryption:** SHA-256 cannot be reversed (line 4 of hash function)
3. **Data Purging:** All hashes cleared after matching (lines 41-43)
4. **Client-Side Only:** All processing in browser, zero server-side phone data

7.5 Settlement Suggestion Algorithm

7.5.1 Purpose

Suggest optimal payment transactions to settle all debts with a minimum number of payments (simplified greedy approach).

ALGORITHM suggestSettlements(balanceSheet)

```
INPUT: balanceSheet (Map<String, Map<String, Number>>)
OUTPUT: settlements (Array<{from: String, to: String, amount: Number}>)

1:  // Build list of creditors (owed money) and debtors (owe money)
2:  creditors ← EMPTY_ARRAY()
3:  debtors ← EMPTY_ARRAY()
```

```
4:  FOR EACH user IN balanceSheet.keys() DO
5:      netBalance ← 0
6:      FOR EACH otherUser IN balanceSheet[user].keys() DO
7:          netBalance ← netBalance + balanceSheet[user][otherUser]
8:      END FOR

9:      IF netBalance > 0 THEN
10:         APPEND({user: user, amount: netBalance}) TO creditors
11:     ELSE IF netBalance < 0 THEN
12:         APPEND({user: user, amount: ABS(netBalance)}) TO debtors
13:     END IF
14: END FOR

    // Sort by amount (descending) for greedy approach
15: SORT creditors BY amount DESCENDING
16: SORT debtors BY amount DESCENDING

17: settlements ← EMPTY_ARRAY()
18: i ← 0  // Creditor index
19: j ← 0  // Debtor index

20: WHILE i < LENGTH(creditors) AND j < LENGTH(debtors) DO
21:     creditor ← creditors[i]
22:     debtor ← debtors[j]

23:     settleAmount ← MIN(creditor.amount, debtor.amount)

24:     APPEND({
            from: debtor.user,
            to: creditor.user,
            amount: settleAmount
        }) TO settlements

25:     creditor.amount ← creditor.amount - settleAmount
26:     debtor.amount ← debtor.amount - settleAmount
```

```
27:     IF creditor.amount == 0 THEN
28:         i ← i + 1  // Move to next creditor
29:     END IF

30:     IF debtor.amount == 0 THEN
31:         j ← j + 1  // Move to next debtor
32:     END IF
33: END WHILE

34: RETURN settlements

END ALGORITHM
```

7.5.2 Example Execution

Input:

Net Balances:
Alice: +€50 (owed)
Bob: -€30 (owes)
Carol: -€20 (owes)

Execution:

```
Step 1: Build lists
  Creditors: [Alice: €50]
  Debtors: [Bob: €30, Carol: €20]

Step 2: Sort (already sorted)

Step 3: Match
  Iteration 1:
```

```
    Match Bob (€30) with Alice (€50)
    Settlement: Bob → Alice €30
    Remaining: Alice €20, Bob €0

  Iteration 2:
    Match Carol (€20) with Alice (€20)
    Settlement: Carol → Alice €20
    Remaining: Alice €0, Carol €0
```

Output:

```
[
  { from: "Bob", to: "Alice", amount: 30.00 },
  { from: "Carol", to: "Alice", amount: 20.00 }
]
```

Optimality: This greedy approach produces minimum number of transactions for simple cases. For complex scenarios with many users, more sophisticated algorithms (e.g., network flow) could reduce transactions further, but the greedy approach is O(n log n) and suffices for typical group sizes (<20 people).

7.6 Invitation Expiration Cleanup

7.6.1 Purpose

Automatically expire pending invitations after 7 days (runs as scheduled Cloud Function).

```
ALGORITHM cleanupExpiredInvitations()

INPUT: None (runs as cron job)
OUTPUT: Count of expired invitations

1:  now ← currentTimestamp()
```

```
2:   sevenDaysAgo ← now - (7 × 24 × 60 × 60 × 1000)  // 7 days in
milliseconds

3:  query ← Firestore.collection('invitations')
        .where('status', '==', 'pending')
        .where('expiresAt', '<', now)

4:  expiredInvitations ← await query.get()

5:  IF expiredInvitations.empty THEN
6:      RETURN 0
7:  END IF

8:  batch ← Firestore.batch()
9:  count ← 0

10: FOR EACH doc IN expiredInvitations.docs DO
11:     batch.update(doc.ref, {
            status: 'expired',
            expiredAt: now
        })
12:     count ← count + 1

        // Firestore batch limit is 500 operations
13:     IF count % 500 == 0 THEN
14:         await batch.commit()
15:         batch ← Firestore.batch()  // New batch
16:     END IF
17: END FOR

18: IF count % 500 != 0 THEN
19:     await batch.commit()  // Commit remaining
20: END IF

21: RETURN count
```

```
END ALGORITHM
```

Deployment:

```
// Cloud Function (Firebase)
exports.cleanupExpiredInvitations = functions.pubsub
  .schedule('every 24 hours')  // Runs daily at midnight UTC
  .onRun(async (context) => {
    const count = await cleanupExpiredInvitations();
    console.log(`Expired ${count} invitations`);
    return null;
  });
```

7.7 Mathematics & Logic of Expense Splitting in Split-Sync

Breaking down ALL the math behind the splitting algorithms - from simple to complex!

7.7.1 Basic Split Methods

1. Equal Split (Most Common)

The Logic:

Divide total equally among N people. Handle remainder by giving extra pennies to the payer.

Mathematical Formula:
Given:
Total Amount = (T)
Number of People = (N)

Base Amount per Person = $\lfloor T / N \rfloor$ (floor division)
Remainder = T - (Base Amount × N)

Split Assignment:
Payer gets = Base Amount + Remainder
Others get = Base Amount each

Example 1: Perfect Division
Total: £40.00
People: 4
Calculation:
  £40.00 ÷ 4 = £10.00 exactly
  Remainder: £0.00

Split:
  Person A (payer): £10.00
  Person B: £10.00
  Person C: £10.00
  Person D: £10.00
  ✓ Total: £40.00

Example 2: With Remainder
Total: £45.60
People: 4
Calculation:
  £45.60 ÷ 4 = £11.40 base
  4 × £11.40 = £45.60
  Remainder: £0.00 (lucky it divides evenly!)

But if it was £45.61:
  £45.61 ÷ 4 = £11.4025
  Base: £11.40 (rounded down)
  4 × £11.40 = £45.60
  Remainder: £0.01

Split:
  Person A (payer): £11.40 + £0.01 = £11.41
  Person B: £11.40
  Person C: £11.40
  Person D: £11.40
  ✓ Total: £45.61

Implementation Code:

```
function equalSplit(totalAmount, numberOfPeople, payerIndex = 0)
{
  // Convert to pennies to avoid floating point errors
  const totalPennies = Math.round(totalAmount * 100);

  // Calculate base amount per person
  const baseAmount = Math.floor(totalPennies / numberOfPeople);

  // Calculate remainder
  const remainder = totalPennies - (baseAmount *
numberOfPeople);

  // Create splits array
  const splits = [];
  for (let i = 0; i < numberOfPeople; i++) {
    // Payer gets the remainder
    const amount = i === payerIndex
      ? (baseAmount + remainder) / 100
      : baseAmount / 100;

    splits.push({
      userId: `user${i}`,
      amount: amount,
      percentage: (amount / totalAmount) * 100
    });
  }

  return splits;
}

// Example usage:
equalSplit(45.61, 4, 0);
// Returns:
// [
//   { userId: 'user0', amount: 11.41, percentage: 25.02 },
//   { userId: 'user1', amount: 11.40, percentage: 24.99 },
//   { userId: 'user2', amount: 11.40, percentage: 24.99 },
//   { userId: 'user3', amount: 11.40, percentage: 24.99 }
// ]
```

2. Percentage Split

The Logic:

Each person pays a percentage of the total. Percentages must sum to 100%.

Mathematical Formula:
Given:
- Total Amount (T)
- Percentages (P_1, P_2, P_3, ... $P_□$)

Validation:
$P_1 + P_2 + P_3 + ... + P_□ = 100\%$

Amount for Person i:
$A_i = (T \times P_i) / 100$

Example 1: Income-Based Split
Total: £100.00
People: 3

Percentages:
Alice (high earner): 50%
Bob (medium earner): 30%
Carol (student): 20%

Calculation:
Alice: £100.00 × 50% = £50.00
Bob: £100.00 × 30% = £30.00
Carol: £100.00 × 20% = £20.00
✓ Total: £100.00

Example 2: With Rounding Issues
Total: £45.67
People: 3

Percentages:
Alice: 33.33%
Bob: 33.33%

Carol: 33.34%

Calculation:
Alice: £45.67 × 33.33% = £15.2233 → £15.22
Bob: £45.67 × 33.33% = £15.2233 → £15.22
Carol: £45.67 × 33.34% = £15.2263 → £15.23
✓ Total: £45.67

Implementation with Rounding Correction:

```
function percentageSplit(totalAmount, percentages) {
  // Validate percentages sum to 100
  const sum = percentages.reduce((acc, p) => acc + p.percentage,
0);
  if (Math.abs(sum - 100) > 0.01) {
    throw new Error(`Percentages must sum to 100%, got
${sum}%`);
  }

  const totalPennies = Math.round(totalAmount * 100);

  // Calculate amounts
  const splits = percentages.map(p => ({
    userId: p.userId,
    percentage: p.percentage,
    amount: Math.round((totalPennies * p.percentage) / 100) /
100
  }));

  // Check for rounding errors
  const calculatedTotal = splits.reduce((sum, s) => sum +
s.amount, 0);
  const diff = totalAmount - calculatedTotal;

  // Add/subtract rounding difference to person with highest
percentage
  if (Math.abs(diff) > 0.01) {
    const maxIndex = splits.findIndex(s =>
      s.percentage === Math.max(...splits.map(x =>
x.percentage))
    );
    splits[maxIndex].amount += diff;
```

```
    splits[maxIndex].amount = Math.round(splits[maxIndex].amount
* 100) / 100;
  }

  return splits;
}

// Example:
percentageSplit(45.67, [
  { userId: 'alice', percentage: 33.33 },
  { userId: 'bob', percentage: 33.33 },
  { userId: 'carol', percentage: 33.34 }
]);
```

3. By Exact Amount

The Logic:

Each person specifies their exact amount. Amounts must sum to total.

Mathematical Formula:
Given:
- Total Amount (T)
- Amounts (A_1, A_2, A_3, ... $A_□$)

Validation:
$A_1 + A_2 + A_3 + ... + A_□ = T$

Percentage for Person i:
$P_i = (A_i / T) \times 100$

Example: Itemized Restaurant Bill
Total: £75.50

What each person ordered:
Alice (had steak): £30.00
Bob (had pasta): £20.00
Carol (had salad): £15.00

Dave (had pizza): £10.50

Calculation:
£30.00 + £20.00 + £15.00 + £10.50 = £75.50 ✓

Percentages:
Alice: (30 / 75.50) × 100 = 39.74%
Bob: (20 / 75.50) × 100 = 26.49%
Carol: (15 / 75.50) × 100 = 19.87%
Dave: (10.50 / 75.50) × 100 = 13.91%

Implementation:

```
function amountSplit(totalAmount, amounts) {
  // Calculate sum of amounts
  const sum = amounts.reduce((acc, a) => acc + a.amount, 0);

  // Validate sum equals total (allow 1 penny tolerance)
  if (Math.abs(sum - totalAmount) > 0.01) {
    throw new Error(
      `Amounts must sum to total. Expected ${totalAmount}, got
${sum}`
    );
  }

  // Calculate percentages
  const splits = amounts.map(a => ({
    userId: a.userId,
    amount: a.amount,
    percentage: (a.amount / totalAmount) * 100
  }));

  return splits;
}
```

4 By Shares/Weight

The Logic:

People get "shares" (like pizza slices). More shares = bigger portion.

Mathematical Formula:
Given:
- Total Amount (T)
- Shares (S_1, S_2, S_3, ... $S_□$)

Total Shares = $S_1 + S_2 + ... + S_□$

Amount per Share = T / Total Shares

Amount for Person i:
$A_i = S_i \times$ Amount per Share

Percentage for Person i:
$P_i = (S_i$ / Total Shares) × 100

Example 1: Adults vs Children
Total: £120.00
People: 5

Shares:
Alice (adult): 2 shares
Bob (adult): 2 shares
Carol (adult): 2 shares
Child 1: 1 share
Child 2: 1 share

Total Shares: 2+2+2+1+1 = 8 shares

Amount per Share:
£120.00 ÷ 8 = £15.00 per share

Split:
Alice: 2 × £15.00 = £30.00 (25%)
Bob: 2 × £15.00 = £30.00 (25%)
Carol: 2 × £15.00 = £30.00 (25%)
Child 1: 1 × £15.00 = £15.00 (12.5%)
Child 2: 1 × £15.00 = £15.00 (12.5%)
✓ Total: £120.00

Example 2: Unequal Consumption
Total: £60.00
People: 3

Shares (based on how much eaten):
Alice (ate 3 pieces): 3 shares
Bob (ate 4 pieces): 4 shares
Carol (ate 2 pieces): 2 shares

Total: 9 shares
Per share: £60.00 ÷ 9 = £6.67

Split:
Alice: 3 × £6.67 = £20.01
Bob: 4 × £6.67 = £26.68
Carol: 2 × £6.67 = £13.34
Total: £60.03 (rounding error of £0.03)

Implementation:

```
function sharesSplit(totalAmount, shares) {
  const totalPennies = Math.round(totalAmount * 100);

  // Calculate total shares
  const totalShares = shares.reduce((sum, s) => sum + s.shares,
0);

  // Amount per share
  const amountPerShare = totalPennies / totalShares;

  // Calculate amounts
  const splits = shares.map(s => ({
    userId: s.userId,
    shares: s.shares,
    amount: Math.round(s.shares * amountPerShare) / 100,
    percentage: (s.shares / totalShares) * 100
  }));

  // Handle rounding errors
  const calculatedTotal = splits.reduce((sum, s) => sum +
s.amount, 0);
```

```
  const diff = totalAmount - calculatedTotal;

  if (Math.abs(diff) > 0.01) {
    // Give difference to person with most shares
    const maxIndex = splits.findIndex(s =>
      s.shares === Math.max(...splits.map(x => x.shares))
    );
    splits[maxIndex].amount += diff;
    splits[maxIndex].amount = Math.round(splits[maxIndex].amount
* 100) / 100;
  }

  return splits;
}
```

7.7.2 Advanced Algorithm -Minimum Cash Flow

The Problem

Scenario:
Group expenses over time create complex web of debts:

Alice owes Bob: £20
Alice owes Carol: £10
Dave owes Bob: £15
Carol owes Dave: £5

Without optimization: 4 transactions needed
With optimization: Fewer transactions possible!

Mathematical Representation:
Debt Graph:
Alice → Bob: £20
Alice → Carol: £10
Dave → Bob: £15
Carol → Dave: £5

Net Positions:
Alice: -£30 (owes £30 total)
Bob: +£35 (owed £35 total)

Carol: +£5 (owed £10, owes £5, net +£5)
Dave: -£10 (owes £15, owed £5, net -£10)

The Algorithm: Greedy Minimum Cash Flow

Step 1: Calculate Net Balance for Each Person

For each person P:
Net Balance = (Total Owed to P) - (Total P Owes)

Example:
Alice net = £0 - £30 = -£30 (debtor)
Bob net = £35 - £0 = +£35 (creditor)
Carol net = £10 - £5 = +£5 (creditor)
Dave net = £5 - £15 = -£10 (debtor)

Step 2: Separate into Debtors and Creditors

Debtors (negative balance):
Alice: -£30
Dave: -£10

Creditors (positive balance):
Bob: +£35
Carol: +£5

Step 3: Match Largest Debtor with Largest Creditor

Iteration 1:
Largest debtor: Alice (-£30)
Largest creditor: Bob (+£35)
Settlement: min(30, 35) = £30

Alice pays Bob £30

Updated balances:
Alice: -£30 + £30 = £0 (settled)
Bob: +£35 - £30 = +£5 (still owed £5)

Iteration 2:

Largest debtor: Dave (-£10)
Largest creditor: Bob (+£5)
Settlement: min(10, 5) = £5

Dave pays Bob £5

Updated balances:
Dave: -£10 + £5 = -£5 (still owes £5)
Bob: +£5 - £5 = £0 (settled)

Iteration 3:
Largest debtor: Dave (-£5)
Largest creditor: Carol (+£5)
Settlement: min(5, 5) = £5

Dave pays Carol £5

Updated balances:
Dave: -£5 + £5 = £0 (settled)
Carol: +£5 - £5 = £0 (settled)

DONE! Everyone settled.

Result:

Original: 4 transactions
Optimized: 3 transactions

Optimized settlements:
1. Alice → Bob: £30
2. Dave → Bob: £5
3. Dave → Carol: £5

Mathematical Proof

Theorem: The greedy algorithm produces the minimum number of transactions.

Proof Sketch:

1. Sum of all debts = Sum of all credits (conservation of money)

2. Each transaction reduces the count of non-zero balances by at least 1
3. If N people have non-zero balances, at most (N-1) transactions needed
4. Greedy matching ensures we never need more than (N-1) transactions

Formal:

Given N people with net balances:
Let D = number of debtors
Let C = number of creditors

Minimum transactions = max(D, C)

In worst case: min(D, C) = min(D, C)
In best case: D = C, so we need D transactions

- Greedy algorithm achieves this bound.

Complete Implementation

```
function minimizeCashFlow(balances) {
  // Step 1: Calculate net balance for each person
  const netBalances = {};

  for (const [userId, balance] of Object.entries(balances)) {
    let net = 0;

    // Add what they're owed
    for (const amount of Object.values(balance.owedBy || {})) {
      net += amount;
    }

    // Subtract what they owe
    for (const amount of Object.values(balance.owes || {})) {
      net -= amount;
    }

    // Only keep non-zero balances (ignore settled people)
    if (Math.abs(net) > 0.01) {
      netBalances[userId] = net;
    }
  }
```

```
  // Step 2: Separate into creditors and debtors
  const creditors = [];  // People owed money (positive)
  const debtors = [];    // People who owe money (negative)

  for (const [userId, balance] of Object.entries(netBalances)) {
    if (balance > 0) {
      creditors.push({ userId, amount: balance });
    } else {
      debtors.push({ userId, amount: Math.abs(balance) });
    }
  }

  // Step 3: Sort by amount (largest first) for optimal matching
  creditors.sort((a, b) => b.amount - a.amount);
  debtors.sort((a, b) => b.amount - a.amount);

  // Step 4: Match debtors with creditors
  const settlements = [];
  let i = 0;  // Debtor index
  let j = 0;  // Creditor index

  while (i < debtors.length && j < creditors.length) {
    const debtor = debtors[i];
    const creditor = creditors[j];

    // Settle the minimum of what debtor owes and creditor is
owed
    const settleAmount = Math.min(debtor.amount,
creditor.amount);

    settlements.push({
      from: debtor.userId,
      to: creditor.userId,
      amount: Math.round(settleAmount * 100) / 100  // Round to
2 decimals
    });

    // Update remaining amounts
    debtor.amount -= settleAmount;
    creditor.amount -= settleAmount;
```

```
    // Move to next if fully settled
    if (debtor.amount < 0.01) i++;
    if (creditor.amount < 0.01) j++;
  }

  return settlements;
}

// Example usage:
const balances = {
  alice: {
    owes: { bob: 20, carol: 10 },
    owedBy: {}
  },
  bob: {
    owes: {},
    owedBy: { alice: 20, dave: 15 }
  },
  carol: {
    owes: { dave: 5 },
    owedBy: { alice: 10 }
  },
  dave: {
    owes: { bob: 15 },
    owedBy: { carol: 5 }
  }
};

const optimized = minimizeCashFlow(balances);
console.log(optimized);
// Output:
// [
//   { from: 'alice', to: 'bob', amount: 30 },
//   { from: 'dave', to: 'bob', amount: 5 },
//   { from: 'dave', to: 'carol', amount: 5 }
// ]
```

Complexity Analysis

1. **Time Complexity:**

- Calculate net balances: O(N × M) where M = average debts per person

- Sort creditors/debtors: O(N log N)
- Match and settle: O(N)

Total: O(N log N)

2. **Space Complexity:**

O(N) for storing net balances and settlements

For Splitsync project:

- N = typically 4-10 people per group
- Algorithm runs instantly even for 100 people
- Real bottleneck is database queries, not algorithm

7.7.3 Handling Edges Cases

The Problem: Floating Point Precision
// JavaScript floating point error:
0.1 + 0.2 === 0.3 // false! (0.30000000000000004)

// In expense splitting:
£10.00 / 3 = £3.333333...

The Solution: Work is to work with the smallest unit of currency like pennies etc.

```
function toInt(amount) {
  return Math.round(amount * 100);  // Convert £10.50 → 1050
pennies
}

function toDecimal(pennies) {
  return pennies / 100;  // Convert 1050 pennies → £10.50
}

// Example:
const total = toInt(45.67);  // 4567 pennies
const perPerson = Math.floor(total / 4);  // 1141 pennies each
const remainder = total - (perPerson * 4);  // 3 pennies
```

```
// Results:
// Person 1: 1144 pennies = £11.44
// Person 2-4: 1141 pennies = £11.41 each
```

7.7.4 Validation Rules

What must always be validated include:

1. Split Totals Should Match Expense Total:

```
function validateSplit(expenseAmount, splits) {
  const splitTotal = splits.reduce((sum, s) => sum + s.amount,
0);
  const diff = Math.abs(expenseAmount - splitTotal);

  if (diff > 0.01) {  // Allow 1 penny tolerance
    throw new Error(
      `Split total (${splitTotal}) doesn't match expense
(${expenseAmount})`
    );
  }

  return true;
}
```

2. All Amounts Should Be Positive:

```
function validatePositiveAmounts(splits) {
  for (const split of splits) {
    if (split.amount <= 0) {
      throw new Error(`Amount must be positive, got
${split.amount}`);
    }
  }
  return true;
}
```

3. At Least One Person Should Be Involved:

```
function validateMinimumPeople(splits) {
  if (splits.length < 1) {
    throw new Error('Must have at least one person in split');
  }
  return true;
}
```

4. Percentages Sum Should Add To 100:

```
function validatePercentages(percentages) {
  const sum = percentages.reduce((total, p) => total + p, 0);

  if (Math.abs(sum - 100) > 0.01) {
    throw new Error(`Percentages must sum to 100%, got
${sum}%`);
  }

  return true;
}
```

7.7.5 Rounding Strategies

7.7.5.1 Strategy 1: Banker's Rounding (Round Half to Even)

```
function bankersRound(value) {
  const rounded = Math.round(value * 100) / 100;

  // If exactly half, round to nearest even
  if (Math.abs(value - rounded) === 0.005) {
    return Math.floor(value * 100) / 100;
  }

  return rounded;
}
```

7.7.5.2 Strategy 2: Give Remainder to Payer (Recommended)

```
// We use this in Split-Sync
```

```
// Simple, fair, predictable
// Payer gets any rounding difference
```

7.7.5.3 Strategy 3: Distribute Pennies Round-Robin

```
function distributePennies(baseAmount, count, remainder) {
  const amounts = new Array(count).fill(baseAmount);

  // Distribute extra pennies one-by-one
  for (let i = 0; i < remainder; i++) {
    amounts[i] += 0.01;
  }

  return amounts;
}
```

8.0 CONCLUSION

Split-Sync fills a critical gap in group expense management by unifying expense tracking and payment settlement within a single progressive web application. This functional specification details a comprehensive solution combining receipt scanning through OCR technology, flexible split calculations including per-item allocation, balance optimization algorithms minimizing settlement transactions, and integrated payment processing via established gateways. The

architecture leverages React, Firebase, and Node.js to deliver real-time synchronization with offline capability, ensuring reliable operation across diverse network conditions.

The specifications presented establish clear technical requirements, database schema, service architecture, and security controls necessary for successful implementation. By addressing the complete expense lifecycle—from capture through settlement—and emphasizing user experience alongside technical robustness, Split-Sync is positioned to deliver meaningful value to informal groups managing shared finances. The modular design and attention to compliance requirements provide a solid foundation for development, deployment, and future enhancement based on user needs and market evolution.

APPENDIX

Figure	Description
Fig. 1	System Architecture Diagram
Fig. 2	Application Navigation Structure
Fig. 3	Entity Relationship Diagram
Fig. 4	User Flow Diagram
Fig. 5	UC-03 Login with Biometric Authentication Sequence Diagram
Fig. 6	UC-01 User Registration Sequence Diagram
Fig. 7	UC-27/UC-28 Create Group with Quick Add Sequence Diagram
Fig. 8	UC-46 Accept Group Invitation Sequence Diagram

Fig. 9 UC-54 to UC-62 Add Expense with Receipt Scanning Sequence Diagram

Fig. 10 UC-42 Find Contacts Sequence Diagram

Fig. 11 UC-78 Settle Payment via PayPal Sequence Diagram

Fig. 12 UC-74 View Group Balance Sequence Diagram

Fig. 13 UC-92 View Activity Feed Sequence Diagram

Fig. 14 Class Diagram

REFERENCES

Firebase. (2024). Cloud Firestore. Available at: https://firebase.google.com/docs/firestore [Accessed: 19 November 2024].

Fowler, M. (2003) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Google Cloud. (2024). Vision AI Documentation. Available at: https://cloud.google.com/vision/docs [Accessed: 19 November 2024].

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Morrison, L., Zhang, Y. and Chen, R. (2022). 'User Experience Challenges in Expense-Sharing Applications', Journal of Financial Technology, 8(3), pp. 245-267.

React. (2024). React Documentation. Available at: https://react.dev [Accessed: 19 November 2024].

Stripe., (2024). Stripe API Reference. Available at: https://stripe.com/docs/api [Accessed: 19 November 2024].

Vercel. (2024). Vercel Documentation. Available at: https://vercel.com/docs [Accessed: 19 November 2024].

Verma, R. and Das, S. (2020) 'Efficient algorithms for minimal transaction settlement in group expense sharing', *Algorithmica*, 82(9), pp. 2567-2591.

Zachariadis, M. and Ozcan, P. (2017) 'The API economy and digital transformation in financial services: The case of open banking', *Swift Institute Working Paper*, 2016-001.